
O-RAN Work Group 3 (Near-Real-time RAN Intelligent Controller and E2 Interface)

Near-RT RIC APIs

Copyright © 2024 by the O-RAN ALLIANCE e.V.

The copying or incorporation into any other work of part or all of the material available in this specification in any form without the prior written permission of O-RAN ALLIANCE e.V. is prohibited, save that you may print or download extracts of the material of this specification for your personal use, or copy the material of this specification for the purpose of sending to individual third parties for their information provided that you acknowledge O-RAN ALLIANCE as the source of the material and that you inform the third party that these conditions apply to them and that they must comply with them.

O-RAN ALLIANCE e.V., Buschkauler Weg 27, 53347 Alfter, Germany

Register of Associations, Bonn VR 11238, VAT ID DE321720189

Contents

Foreword.....	4
Modal verbs terminology	4
1 Scope.....	5
2 References	5
2.1 Normative references	5
2.2 Informative references	5
3 Definition of terms, symbols and abbreviations	6
3.1 Terms	6
3.2 Symbols	6
3.3 Abbreviations	6
4 General	6
4.1 Introduction.....	6
4.2 General Principles.....	6
4.3 Version Control.....	6
5 A1-related API	7
5.1 Overview	7
5.2 A1-related API Procedures	8
5.2.1 A1 Policy Setup procedure.....	8
5.2.2 A1 Policy Update procedure	10
5.2.3 A1 Policy Delete procedure	11
5.3 Elements for A1-related API Communication.....	13
5.3.1 General	13
5.3.2 Message Functional Definition and Content.....	13
5.3.3 Information Element Definitions	15
5.3.4 Message and Information Element Abstract Syntax	17
5.3.5 Message Transfer Syntax	20
6 E2-related API.....	20
6.1 Overview	20
6.1.1 Protocol Stack	20
6.1.2 List of E2 related APIs, procedures and messages.....	21
6.2 E2-related API Procedures.....	22
6.2.1 E2_Subscription_API procedures	22
6.2.2 E2_Indication_API procedures	28
6.2.3 E2_Control_API procedures	29
6.3 Elements for E2-related API Communication	31
6.3.1 General	31
6.3.2 Message Functional Definition and Content.....	31
6.3.3 Information Element Definitions	40
6.3.4 Message and Information Element Abstract Syntax	43
6.3.5 Message Transfer Syntax	54
7 SDL API.....	54
7.1 Overview	54
7.1.1 Protocol Stack	54
7.1.2 List of SDL APIs, procedures and messages	54
7.2 SDL API Procedures.....	56
7.2.1 SDL Client Registration procedure	56
7.2.2 SDL Client Deregistration procedure.....	56
7.2.3 SDL Subscribe Information procedure	56
7.2.4 SDL Information Push procedure	57
7.2.5 SDL Information Update Notify procedure	58
7.2.6 SDL Information Fetch procedure	59
7.2.7 Store Data procedure	60

7.2.8	Modify Data procedure	60
7.3	Elements for SDL API Communication	65
7.3.1	General	65
7.3.2	Message Functional Definition and Content	66
7.3.3	Information Element Definitions	73
7.3.4	Message and Information Element Abstract Syntax	82
7.3.5	Message Transfer Syntax	93
8	Management API	93
8.1	Overview	93
8.1.1	Protocol Stack	93
8.1.2	List of Management APIs, procedures and messages	94
8.2	Management API Procedures	94
8.2.1	xApp Registration Procedure	94
8.3	Elements for Management API Communication	96
8.3.1	General	96
8.3.2	Message Functional Definition and Content	96
8.3.3	Information Element Definitions	97
8.3.4	Message and Information Element Abstract Syntax	98
8.3.5	Message Transfer Syntax	100
9	Enablement API	100
9.1	Overview	100
9.1.1	Protocol Stack	100
9.1.2	List of Enablement APIs, procedures and messages	101
9.2	Enablement API Procedures	102
9.2.1	Enabl_Discovery_API Procedures	102
9.2.2	Enabl_Events_API Procedures	104
9.3	Elements for Enablement API Communication	112
9.3.1	General	112
9.3.2	Message Functional Definition and Content	112
9.3.3	Information Element Definitions	119
9.3.4	Message and Information Element Abstract Syntax	122
9.3.5	Message Transfer Syntax	128
10	Common definitions across APIs	128
10.1	Information element definitions	128
10.1.1	PTI	128
10.1.2	xApp ID	129
10.1.3	API Name	129
10.1.4	API Version	129
10.1.5	A1 Policy Type ID	129
10.1.6	Global E2 Node ID	130
10.1.7	RAN Function ID	130
10.1.8	SCTP API PDU	130
10.1.9	Endpoint Information	130
10.2	Information element abstract syntax	130
10.2.1	Abstact syntax in Protocol Buffers	130
	Annex (informative): Change History	133

Foreword

This Technical Specification (TS) has been produced by O-RAN Alliance.

The content of the present document is subject to continuing work within O-RAN and may change following formal O-RAN approval. Should the O-RAN Alliance modify the contents of the present document, it will be re-released by O-RAN with an identifying change of version date and an increase in version number as follows:

version xx.yy.zz

where:

- xx: the first digit-group is incremented for all changes of substance, i.e. technical enhancements, corrections, updates, etc. (the initial approved document will have xx=01). Always 2 digits with leading zero if needed.
- yy: the second digit-group is incremented when editorial only changes have been incorporated in the document. Always 2 digits with leading zero if needed.
- zz: the third digit-group included only in working versions of the document indicating incremental changes during the editing process. External versions never include the third digit-group. Always 2 digits with leading zero if needed.

Modal verbs terminology

In the present document "**shall**", "**shall not**", "**should**", "**should not**", "**may**", "**need not**", "**will**", "**will not**", "**can**" and "**cannot**" are to be interpreted as described in clause 3.2 of the O-RAN Drafting Rules (Verbal forms for the expression of provisions).

"**must**" and "**must not**" are **NOT** allowed in O-RAN deliverables except when used in direct citation.

1 Scope

The present document specifies the signalling and data transport protocols for Near-RT RIC APIs. The protocols are developed in accordance with the functionalities and interactions stated in [2].

2 References

2.1 Normative references

References are either specific (identified by date of publication and/or edition number or version number) or non-specific. For specific references, only the cited version applies. For non-specific references, the latest version of the referenced document (including any amendments) applies.

NOTE: While any hyperlinks included in this clause were valid at the time of publication, O-RAN cannot guarantee their long term validity.

The following referenced documents are necessary for the application of the present document.

- [1] 3GPP TR 21.905: "Vocabulary for 3GPP Specifications".
- [2] O-RAN.WG3.RICARCH-v03.00.00: "O-RAN, Near-Real-time RAN Intelligent Controller, Near-RT RIC Architecture".
- [3] O-RAN.WG3.E2AP-v02.01.00: "O-RAN, Near-Real-time RAN Intelligent Controller, E2 Application Protocol(E2AP)".
- [4] Protocol buffers. Available at <https://developers.google.com/protocol-buffers/>.
- [5] gRPC. Available at <https://www.grpc.io/>.
- [6] O-RAN.WG11.SecReqSpecs: "O-RAN, Security Requirements Specifications".
- [7] O-RAN.WG2.A1TD-v04.00: "O-RAN, A1 interface: Type Definitions".
- [8] O-RAN.WG3.E2SM-v02.01.02: "O-RAN, Near-Real-time RAN Intelligent Controller, E2 Service Model (E2SM)".
- [9] IETF RFC 4122: "A Universally Unique Identifier (UUID) URN Namespace".
- [10] O-RAN.WG3.Y1AP-v01.00: "O-RAN Y1 interface: Application Protocol".
- [11] O-RAN.WG3.Y1TD-v01.00: "O-RAN Y1 interface: Type Definitions".

2.2 Informative references

References are either specific (identified by date of publication and/or edition number or version number) or non-specific. For specific references, only the cited version applies. For non-specific references, the latest version of the referenced document (including any amendments) applies.

NOTE: While any hyperlinks included in this clause were valid at the time of publication, O-RAN cannot guarantee their long term validity.

The following referenced documents are not necessary for the application of the present document but they assist the user with regard to a particular subject area.

- [i.1] Semantic Versioning Specification. Available at <https://semver.org>.

3 Definition of terms, symbols and abbreviations

3.1 Terms

For the purposes of the present document, the following terms apply:

Near-RT RIC: as defined in [2].

Near-RT RIC APIs: interfaces that connect xApp and Near-RT RIC platform.

Near-RT RIC Platform: a software platform to host xApps. The Near-RT RIC is comprised of the Near-RT RIC Platform and xApps.

xApp: as defined in [2].

3.2 Symbols

No Symbol is defined in this document.

3.3 Abbreviations

For the purposes of the present document, the abbreviations given in 3GPP TR 21.905 [1] and the following apply:

API	Application Programming Interface
Near-RT RIC	Near-real-time RAN Intelligent Controller
PTI	Procedure Transaction Identity

4 General

4.1 Introduction

Clause 4 specifies the principles and conventions commonly applied to Near-RT RIC API design with the Network API approach as described in clause 7.1.2 of [2].

4.2 General Principles

For each Near-RT RIC API, the following principles should be followed:

- A default solution that includes network protocol and message encoding should be defined to ensure interoperability. The default solution requires mandatory support by all Near-RT RIC platforms and xApps. The optional protocols available for a given APIs are discoverable by xApps.
- The IEs in the Near-RT RIC API which refer to different specifications shall be encoded as defined in the referred specification.

The common data types applicable to several API categories should be collected in clause 10.

In addition, the following design principles shall apply:

- Parallel transactions, i.e., multiple ongoing Near-RT RIC API procedures of the same types related to the same xApp, and platform functionality are supported.

4.3 Version Control

An API version shall consist of at least 3 fields, following a MAJOR.MINOR.PATCH pattern according to the Semantic Versioning Specification [i.1] unless specified otherwise.

EXAMPLE: "1.0.0".

The basic rules as reproduced from [i.1] shall apply for incrementing the versions:

- MAJOR version is incremented if backward incompatible change is made.
- MINOR version is incremented if functional change is made in a backward compatible way.
- PATCH version is incremented if correction is made in a backward compatible way.

5 A1-related API

5.1 Overview

5.1.1 Protocol Stack

In this version of specification, the default protocol stack for the A1 related APIs is shown on Figure 5.1.1-1.

Note: the network layer for security protection is FFS.

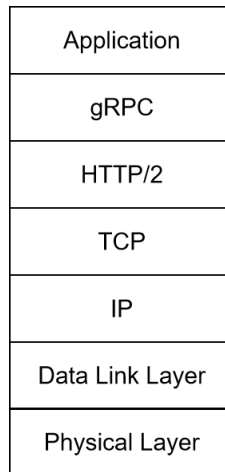


Figure 5.1.1-1: Protocol stack for A1 related APIs

The A1 related APIs use gRPC [5] with Protocol Buffers [4] as the application layer serialization protocol.

5.1.2 List of A1 related APIs, procedures and messages

The A1 related API procedures introduced in clause 9.2 of [2] are categorized under the A1 related APIs. These APIs include:

- A1 Policy API: enabling A1 policy related operations for the xApps.
- A1 Enrichment Information (A1-EI) API: enabling A1-EI related operations for the xApps.

These APIs are summarized in Table 5.1.2-1.

Table 5.1.2-1: List of A1 related APIs

API Name	Description
A1_Policy_API	Service for A1 policy related operations
A1_EI_API	Service for A1 enrichment information related operations

The table 5.1.2-2 lists the API procedures for each A1 related API.

Table 5.1.2-2: List of A1 related API procedures

API Name	API procedure	Communication type
A1_Policy_API	A1 Policy Setup	Request/Response
	A1 Policy Update	Request/Response
	A1 Policy Delete	Request/Response
	A1 Policy Query	Request/Response
	A1 Policy Status Update	Request/Response
A1_EI_API	A1 EI Subscription Setup	Request/Response
	A1 EI Subscription Update	Request/Response
	A1 EI Subscription Delete	Request/Response
	A1 EI Delivery	Notify
	A1 EI Query	Request/Response

The table 5.1.2-3 lists the messages for each A1 related API procedure.

Table 5.1.2-3: List of A1 related API messages

API Procedure	Initiated by	Initiating Message	Outcome Message	
			Successful Outcome	Unsuccessful Outcome
A1 Policy Setup	Near-RT RIC platform	A1 POLICY SETUP REQUEST	A1 POLICY SETUP RESULT (SUCCESS)	A1 POLICY SETUP RESULT (FAILURE)
A1 Policy Update	Near-RT RIC platform	A1 POLICY UPDATE REQUEST	A1 POLICY UPDATE RESULT (SUCCESS)	A1 POLICY UPDATE RESULT (FAILURE)
A1 Policy Delete	Near-RT RIC platform	A1 POLICY DELETE REQUEST	A1 POLICY DELETE RESULT (SUCCESS)	A1 POLICY DELETE RESULT (FAILURE)
A1 Policy Query	Near-RT RIC platform	A1 POLICY QUERY REQUEST	A1 POLICY QUERY RESULT (SUCCESS)	A1 POLICY QUERY RESULT (FAILURE)
A1 Policy Status Update	xApp	A1 POLICY STATUS UPDATE	A1 POLICY STATUS UPDATE ACK	A1 POLICY STATUS UPDATE ERROR
A1 EI Subscription Setup	xApp	A1 EI SUBSCRIPTION SETUP REQUEST	A1 EI SUBSCRIPTION SETUP RESULT (SUCCESS)	A1 EI SUBSCRIPTION SETUP RESULT (FAILURE)
A1 EI Subscription Update	xApp	A1 EI SUBSCRIPTION UPDATE REQUEST	A1 EI SUBSCRIPTION UPDATE RESULT (SUCCESS)	A1 EI SUBSCRIPTION UPDATE RESULT (FAILURE)
A1 EI Subscription Delete	xApp	A1 EI SUBSCRIPTION DELETE REQUEST	A1 EI SUBSCRIPTION DELETE RESULT (SUCCESS)	A1 EI SUBSCRIPTION DELETE RESULT (FAILURE)
A1 EI Delivery	Near-RT RIC platform	A1 EI Delivery	-	-
A1 EI Query	xApp	A1 EI QUERY REQUEST	A1 EI QUERY RESULT (SUCCESS)	A1 EI QUERY RESULT (FAILURE)

5.2 A1-related API Procedures

5.2.1 A1 Policy Setup procedure

5.2.1.1 General

The purpose of the A1 Policy Setup procedure is to enable the execution of the transferred A1 policy.

5.2.1.2 Successful operation

```
@startuml
Skin rose
Participant rp as "Near-RT RIC Platform"
Participant ap as "xApp"

rp->>ap: A1 POLICY SETUP REQUEST
ap->>rp: A1 POLICY SETUP RESULT\n[A1 Policy Setup Result=success]

@enduml
```

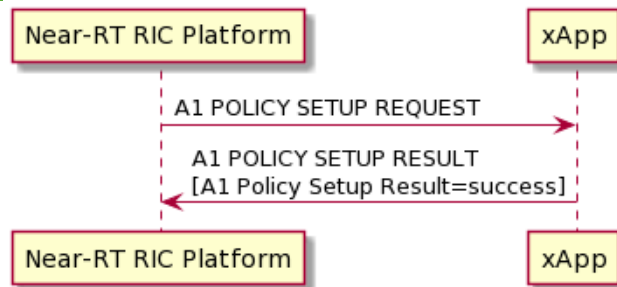


Figure 0-1 : A1 Policy Setup procedure, successful operation

The Near-RT RIC Platform initiates the procedure by sending A1 POLICY SETUP REQUEST message to xApp.

The A1 POLICY SETUP REQUEST message shall contain the information required by the xApp to setup one A1 policy including *A1 Policy Type ID IE*, *A1 Policy ID IE*, and *A1 Policy Object IE*.

Upon reception of the A1 POLICY SETUP REQUEST message, the xApp shall transform the received A1 policy into appropriate E2 / A1-EI operation(s) requirements, and shall trigger Near-RT RIC Platform to accommodate E2 / A1-EI operation(s) requirements.

If Near-RT RIC Platform is able to accommodate E2 / A1-EI operation(s) requirements, xApp replies A1 POLICY SETUP RESULT message with *A1 Policy Setup Result IE* set to *success* to Near-RT RIC Platform. And xApp should store the received *A1 Policy Type ID IE*, *A1 Policy ID IE*, *A1 Policy Object IE* as basic elements of A1 policy profile, and label the status of corresponding A1 policy as “enforced”.

5.2.1.3 Unsuccessful operation

```
@startuml
Skin rose
Participant rp as "Near-RT RIC Platform"
Participant ap as "xApp"

rp->>ap: A1 POLICY SETUP REQUEST
ap->>rp: A1 POLICY SETUP RESULT\n[A1 Policy Setup Result=failure]

@enduml
```

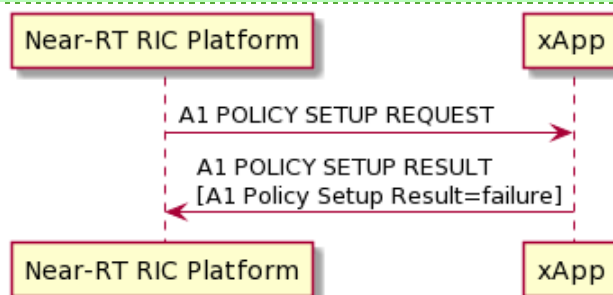


Figure 5.2.1.3-1 : A1 Policy Setup procedure, unsuccessful operation

If xApp cannot transform the received A1 policy into appropriate E2 / A1-EI operation(s) requirements due to unsupported A1 policy type, xApp replies A1 POLICY SETUP RESULT message with *A1 Policy Setup Result* IE set to *failure* to Near-RT RIC Platform, possible with *Cause* IE set to *Not supported A1 policy type*.

If xApp cannot transform the received A1 policy into appropriate E2 / A1-EI operation(s) requirements due to violation of of the requirements as stated in 5.2.1.2.2, 5.2.2.2.2, 5.2.3.2.2, 5.2.4.2.2, 5.2.5.2.2, 5.2.6.2.2, 5.2.7.2.2 and 5.2.8.2.2 of [7], xApp replies A1 POLICY SETUP RESULT message with *A1 Policy Setup Result* IE set to *failure* to Near-RT RIC Platform, possible with *Cause* IE set to *Not allowed statements, restrictions and extensions*.

If Near-RT RIC Platform cannot accommodate E2 / A1-EI operation(s) requirements, xApp replies A1 POLICY SETUP RESULT message with *A1 Policy Setup Result* IE set to *failure* to Near-RT RIC Platform, possible with *Cause* IE set to *Near-RT RIC Platform cannot accommodate E2 / A1-EI operations requirements*.

5.2.1.4 Abnormal conditions

If A1 policy indicated by *A1 Policy Type ID* IE, *A1 Policy ID* IE contained in A1 POLICY SETUP REQUEST message is being enforced by xApp, xApp replies A1 POLICY SETUP RESULT message with *A1 Policy Setup Result* IE set to *failure* to Near-RT RIC Platform, possible with *Cause* IE set to *A1 policy already enabled at xApp*.

5.2.2 A1 Policy Update procedure

5.2.2.1 General

The purpose of the A1 Policy Update procedure is to update the execution of the transferred A1 policy.

5.2.2.2 Successful operation

```
@startuml
Skin rose
Participant rp as "Near-RT RIC Platform"
Participant ap as "xApp"

rp->>ap: A1 POLICY UPDATE REQUEST
ap->>rp: A1 POLICY UPDATE RESULT\n[A1 Policy Update Result=success]

@enduml
```

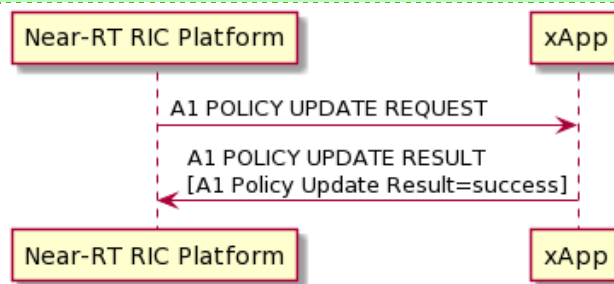


Figure 5.2.2.2-1 : A1 Policy Update procedure, successful operation

The Near-RT RIC Platform initiates the procedure by sending A1 POLICY UPDATE REQUEST message to xApp.

The A1 POLICY UPDATE REQUEST message shall contain the information required by the xApp to update one A1 policy including *A1 Policy Type ID* IE, *A1 Policy ID* IE, and *A1 Policy Object* IE.

Upon reception of the A1 POLICY SETUP REQUEST message, the xApp shall transform the received A1 policy into appropriate E2 / A1-EI operation(s) requirements. If the transformed E2 / A1-EI operation(s) requirements are the same as before, xApp replies A1 POLICY UPDATE RESULT message with *A1 Policy Update Result* IE set to *success* to Near-RT RIC Platform; else, xApp shall trigger Near-RT RIC Platform to accommodate E2 / A1-EI operation(s) requirements.

If Near-RT RIC Platform to accommodate E2 / A1-EI operation(s) requirements succeeds, xApp replies A1 POLICY UPDATE RESULT message with *A1 Policy Update Result* IE set to *success* to Near-RT RIC Platform. And xApp should

update the stored information of *A1 Policy Object* IE of A1 policy profile, and label the status of corresponding A1 policy as “enforced”.

5.2.2.3 Unsuccessful operation

```
@startuml
Skin rose
Participant rp as "Near-RT RIC Platform"
Participant ap as "xApp"

rp->>ap: A1 POLICY UPDATE REQUEST
ap->>rp: A1 POLICY UPDATE RESULT\n[A1 Policy Update Result=failure]

@enduml
```

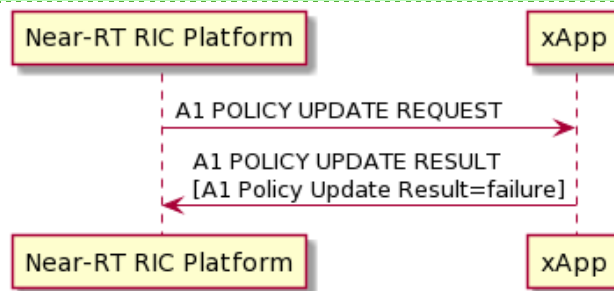


Figure 5.2.2.3-1 : A1 Policy Update procedure, unsuccessful operation

If xApp cannot transform the received A1 policy into appropriate E2 / A1-EI operation(s) requirements due to not support the corresponding A1 policy type, xApp replies A1 POLICY UPDATE RESULT message with *A1 Policy Update Result* IE set to *failure* to Near-RT RIC Platform, possible with *Cause* IE set to *Not supported A1 policy type*.

If xApp cannot transform the received A1 policy into appropriate E2 / A1-EI operation(s) requirements due to violation of the requirements as stated in 5.2.1.2.2, 5.2.2.2.2, 5.2.3.2.2, 5.2.4.2.2, 5.2.5.2.2, 5.2.6.2.2, 5.2.7.2.2 and 5.2.8.2.2 of [7], xApp replies A1 POLICY UPDATE RESULT message with *A1 Policy Update Result* IE set to *failure* to Near-RT RIC Platform, possible with *Cause* IE set to *Not allowed statements, restrictions and extensions*.

If Near-RT RIC Platform cannot accommodate E2 / A1-EI operation(s) requirements, xApp replies A1 POLICY UPDATE RESULT message with *A1 Policy Update Result* IE set to *failure* to Near-RT RIC Platform, possible with *Cause* IE set to *Near-RT RIC Platform cannot accommodate E2 / A1-EI operations requirements*.

5.2.2.4 Abnormal conditions

If A1 policy indicated by *A1 Policy Type ID* IE, *A1 Policy ID* IE contained in A1 POLICY UPDATE REQUEST message is not enabled by xApp, xApp replies A1 POLICY UPDATE RESULT message with *A1 Policy Update Result* IE set to *failure* to Near-RT RIC Platform, possible with *Cause* IE set to *Not enabled A1 policy at xApp*.

5.2.3 A1 Policy Delete procedure

5.2.3.1 General

The purpose of the A1 Policy Delete procedure is to terminate the execution of the transferred A1 policy.

5.2.3.2 Successful operation

```
@startuml
Skin rose
Participant rp as "Near-RT RIC Platform"
Participant ap as "xApp"

rp->>ap: A1 POLICY DELETE REQUEST
ap->>rp: A1 POLICY DELETE RESULT\n[A1 Policy Delete Result=success]

@enduml
```

```
@endum1
```

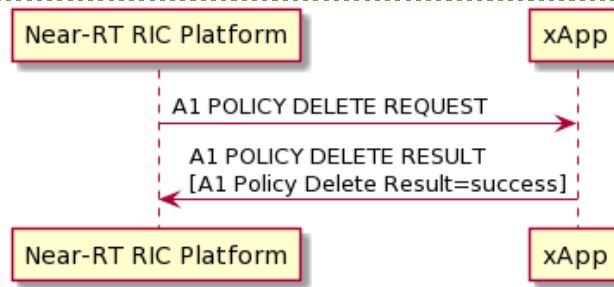


Figure 5.2.3.2-1 : A1 Policy Delete procedure, successful operation

The Near-RT RIC Platform initiates the procedure by sending A1 POLICY DELETE REQUEST message to xApp.

The A1 POLICY DELETE REQUEST message shall contain the information required by the xApp to delete one A1 policy including *A1 Policy Type ID IE*, *A1 Policy ID IE*.

Upon reception of the A1 POLICY DELETE REQUEST message, the xApp shall locate the E2 / A1-EI operation(s) associated with the A1 policy indicated by *A1 Policy Type ID IE*, *A1 Policy ID IE* contained in A1 POLICY DELETE REQUEST message, and shall trigger Near-RT RIC Platform to terminate the accommodation of located E2 / A1-EI operation(s).

No matter Near-RT RIC Platform to terminate the accommodation of located E2 / A1-EI operation(s) succeeds or not, xApp replies A1 POLICY DELETE RESULT message with *A1 Policy Delete Result IE* set to *success* to Near-RT RIC Platform. And xApp should delete the stored A1 policy profile associated.

5.2.3.3 Unsuccessful operation

```
@startuml
Skin rose
Participant rp as "Near-RT RIC Platform"
Participant ap as "xApp"

rp->>ap: A1 POLICY DELETE REQUEST
ap->>rp: A1 POLICY DELETE RESULT\n[A1 Policy Delete Result=failure]

@enduml
```

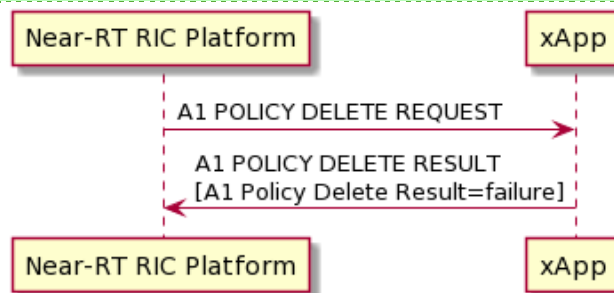


Figure 0-1 : A1 Policy Delete procedure, unsuccessful operation

If xApp cannot locate the E2 / A1-EI operation(s) associated with the A1 policy indicated by *A1 Policy Type ID IE*, *A1 Policy ID IE* contained in A1 POLICY DELETE REQUEST message, xApp replies A1 POLICY DELETE RESULT message with *A1 Policy Delete Result IE* set to *failure* to Near-RT RIC Platform, possible with *Cause IE* set to *Not enabled A1 policy at xApp*.

5.2.3.4 Abnormal conditions

Not applicable.

5.3 Elements for A1-related API Communication

5.3.1 General

5.3.2 Message Functional Definition and Content

5.3.2.1 A1 POLICY SETUP REQUEST

This message is sent by the RIC Platform and is used to request the destined xApp to enable the execution of the nested A1 policy.

Direction: RIC Platform → xApp

IE/Group Name	Presence	Range	IE type and reference	Semantics description	Criticality	Assigned Criticality
Message Type	M		5.3.3.1		YES	reject
A1 Policy Type ID	M		5.3.3.2		YES	reject
A1 Policy ID	M		5.3.3.3		YES	reject
A1 Policy Object	M		OCTET SRTING	Defined in [7].	YES	reject

5.3.2.2 A1 POLICY SETUP RESULT

This message is sent by the xApp as response to the request to enable the execution of the nested A1 policy.

Direction: xApp → RIC Platform

IE/Group Name	Presence	Range	IE type and reference	Semantics description	Criticality	Assigned Criticality
Message Type	M		5.3.3.1		YES	reject
A1 Policy Type ID	M		5.3.3.2		YES	reject
A1 Policy ID	M		5.3.3.3		YES	reject
A1 Policy Setup Result	M		5.3.3.4		YES	reject
Cause	O		5.3.3.5		YES	ignore
A1 Policy Object	C		OCTET SRTING	Defined in [7]. This IE shall present if the A1 Policy Setup Result IE is set to success. When present, this IE is copied from the corresponding Request message.	YES	ignore

5.3.2.3 A1 POLICY UPDATE REQUEST

This message is sent by the RIC Platform and is used to request the destined xApp to update the execution of the nested A1 policy.

Direction: RIC Platform → xApp

IE/Group Name	Presence	Range	IE type and reference	Semantics description	Criticality	Assigned Criticality
Message Type	M		5.3.3.1		YES	reject
A1 Policy Type ID	M		5.3.3.2		YES	reject

A1 Policy ID	M		5.3.3.3		YES	reject
A1 Policy Object	M		OCTET SRTING	Defined in [7].	YES	reject

5.3.2.4 A1 POLICY UPDATE RESULT

This message is sent by xApp as response to the request to update the execution of the nested A1 policy.

Direction: xApp → RIC Platform

IE/Group Name	Presence	Range	IE type and reference	Semantics description	Criticality	Assigned Criticality
Message Type	M		5.3.3.1		YES	reject
A1 Policy Type ID	M		5.3.3.2		YES	reject
A1 Policy ID	M		5.3.3.3		YES	reject
A1 Policy Update Result	M		5.3.3.4		YES	reject
Cause	O		5.3.3.5		YES	ignore
A1 Policy Object	C		OCTET SRTING	Defined in [7]. This IE shall present if the A1 Policy Update Result IE is set to success. When present, this IE is copied from the corresponding Request message.	YES	ignore

5.3.2.5 A1 POLICY DELETE REQUEST

This message is sent by the RIC Platform and is used to request the destined xApp to terminate the execution of the nested A1 policy.

Direction: RIC Platform → xApp

IE/Group Name	Presence	Range	IE type and reference	Semantics description	Criticality	Assigned Criticality
Message Type	M		5.3.3.1		YES	reject
A1 Policy Type ID	M		5.3.3.2		YES	reject
A1 Policy ID	M		5.3.3.3		YES	reject

5.3.2.6 A1 POLICY DELETE RESULT

This message is sent by xApp as response to the request to terminate the execution of the nested A1 policy.

Direction: xApp → RIC Platform

IE/Group Name	Presence	Range	IE type and reference	Semantics description	Criticality	Assigned Criticality
Message Type	M		5.3.3.1		YES	reject
A1 Policy Type ID	M		5.3.3.2		YES	reject
A1 Policy ID	M		5.3.3.3		YES	reject
A1 Policy Delete Result	M		5.3.3.4		YES	reject
Cause	O		5.3.3.5		YES	ignore

- 5.3.2.7 A1 POLICY QUERY REQUEST
- 5.3.2.8 A1 POLICY QUERY RESULT
- 5.3.2.9 A1 POLICY STATUS UPDATE
- 5.3.2.10 A1 POLICY STATUS UPDATE ACK
- 5.3.2.11 A1 POLICY STATUS UPDATE ERROR

- 5.3.2.12 A1-EI SUBSCRIPTION SETUP REQUEST
- 5.3.2.13 A1-EI SUBSCRIPTION SETUP RESULT
- 5.3.2.14 A1-EI SUBSCRIPTION UPDATE REQUEST
- 5.3.2.15 A1-EI SUBSCRIPTION UPDATE RESULT
- 5.3.2.16 A1-EI SUBSCRIPTION DELETE REQUEST
- 5.3.2.17 A1-EI SUBSCRIPTION DELETE RESULT
- 5.3.2.18 A1-EI DELIVERY

- 5.3.2.19 A1-EI QUERY REQUEST
- 5.3.2.20 A1-EI QUERY RESULT

5.3.3 Information Element Definitions

5.3.3.1 Message Type

The Message Type IE identifies the message being sent. It is mandatory for all messages.

IE/Group Name	Presence	Range	IE Type and Reference	Semantics Description
Message Type				
>Procedure Type	M		CHOICE (A1 policy setup, A1 policy update, A1 policy delete, ...)	
>Type of Message	M		CHOICE (Request, Result, ...)	Refer to Table 5.1.2-3 for applicable choices for each procedure type

5.3.3.2 A1 Policy Type ID

Defined in 10.1.5.

5.3.3.3 A1 Policy ID

The A1 Policy ID IE identifies one A1 policy under one A1 policy type.

IE/Group Name	Presence	Range	IE Type and Reference	Semantics Description
A1 Policy ID	M		STRING	

5.3.3.4 Result

The Result IE indicates the procedure succeeds or not.

IE/Group Name	Presence	Range	IE Type and Reference	Semantics Description
Result	M		ENUMERATED (success, failure, ...)	

5.3.3.5 Cause

The Cause IE indicates the reason for a particular event in A1-related API.

IE/Group Name	Presence	Range	IE Type and Reference	Semantics Description
CHOICE <i>Cause Group</i>	M			
> <i>A1-related API Layer</i>				
>>A1-related API Layer Cause	M		ENUMERATED (Unspecified A1-related API layer reason, Not supported A1 policy type, Not allowed statements, restrictions and extensions, Near-RT RIC Platform cannot accommodate E2 / A1-EI operations requirements, A1 policy already enabled at xApp, Not enabled A1 policy at xApp, ...)	
> <i>Transport Layer</i>				
>>Transport Layer Cause	M		ENUMERATED (Unspecified transport layer reason, Transport Resource Unavailable, ...)	
> <i>Protocol</i>				
>>Protocol Cause	M		ENUMERATED (Unspecified protocol reason, Transfer Syntax Error, Abstract Syntax Error (Reject), Abstract Syntax Error (Ignore and Notify), Message not Compatible with Receiver State, Semantic Error, Abstract Syntax Error (Falsely Constructed Message), ...)	
> <i>Misc</i>				
>>Miscellaneous Cause	M		ENUMERATED (Unspecified miscellaneous reason, Control Processing Overload Hardware failure, O&M intervention, ...)	

The meaning of the different cause values is described in the following table. In general, “not supported” cause values indicate that the related capability is missing. On the other hand, “not available” cause values indicate that the related capability is present, but insufficient resources were available to perform the requested action.

A1-related API Layer cause	Meaning
Unspecified A1-related API layer reason	Sent for A1-related API layer cause when none of the specified cause values applies.
Not supported A1 policy type	The A1 policy type is not supported.
Not allowed statements, restrictions and extensions	The A1 policy object is illegal due to violation of statements, restrictions and extensions required for corresponding A1 policy type.
Near-RT RIC Platform cannot accommodate E2 / A1-EI operations requirements	The RIC platform failed to setup xApp’s requested E2 / A1-EI operation(s).
A1 policy already enabled at xApp	The A1 policy indicated by A1 Policy Type ID and A1 Policy ID is enforced already.
Not enabled A1 policy at xApp	The A1 policy indicated by A1 Policy Type ID and A1 Policy ID is not enforced.

Transport Layer cause	Meaning
Unspecified transport layer reason	Sent for transport layer cause when none of the specified cause values applies.
Transport Resource Unavailable	The required transport resources are not available.

Protocol cause	Meaning
Unspecified protocol reason	Sent for protocol cause when none of the specified cause values applies.
Transfer Syntax Error	The received message included a transfer syntax error.
Abstract Syntax Error (Reject)	The received message included an abstract syntax error and the concerning criticality indicated “reject”.
Abstract Syntax Error (Ignore And Notify)	The received message included an abstract syntax error and the concerning criticality indicated “ignore and notify”.
Message Not Compatible With Receiver State	The received message was not compatible with the receiver state.
Semantic Error	The received message included a semantic error.
Abstract Syntax Error (Falsely Constructed Message)	The received message contained IEs or IE groups in wrong order or with too many occurrences.

Miscellaneous cause	Meaning
Unspecified miscellaneous reason	Sent for miscellaneous cause when none of the specified cause values applies.
Control Processing Overload	Control processing overload
Hardware Failure	Action related to hardware failure
O&M Intervention	The action is due to O&M intervention.

5.3.4 Message and Information Element Abstract Syntax

5.3.4.1 Solution 1: gRPC with Protocol Buffers

5.3.4.1.1 Usage of gRPC with Protocol Buffers

This solution uses Protocol Buffers as message encoding, and also as the Interface Definition Language (IDL) for gRPC [5].

Table 5.3.4.1.1-1 lists the mapping of A1-related APIs and corresponding gRPC services.

Table 5.3.4.1.1-1: Mapping of A1-related APIs and gRPC services

API Name	gRPC service(s)
A1_Policy_API	A1_Policy_API

A procedure of an A1-related API is mapped to a *gRPC method*.

Table 5.3.4.1.1-2 lists the A1-related APIs abstract syntax included in this document.

Table 5.3.4.1.1-2: List of A1-related APIs abstract syntax

API Name / Contents	Clause	Version	File Name
A1_Policy_API	5.3.4.1.2.1	1.0.0	a1_policy_api_ver_1_0_0.proto
A1-related API common information elements	5.3.4.1.3	1.0.0	a1_related_apis_common_ies_ver_1_0_0.proto
Near-RT RIC API common information elements	10.2.1	2.0.0	near_rt_ric_apis_common_ies_ver_2_0_0.proto

5.3.4.1.2 API Definitions

5.3.4.1.2.1 A1_Policy_API

```
syntax = "proto3";

package ricapi.a1_policy_api.v1;

import "near_rt_ric_apis_common_ies_ver_2_0_0.proto";
import "a1_related_apis_common_ies_ver_1_0_0.proto";

// gRPC service and rpc definition
service A1_Policy_API {
    rpc AlPolicySetupProc(AlPolicySetupInitMsg) returns (AlPolicySetupOutMsg) {};
    rpc AlPolicyUpdateProc(AlPolicyUpdateInitMsg) returns (AlPolicyUpdateOutMsg) {};
    rpc AlPolicyDeleteProc(AlPolicyDeleteInitMsg) returns (AlPolicyDeleteOutMsg) {};
}

message AlPolicySetupInitMsg {
    optional AlPolicySetupRequest al_policy_setup_request = 1;
}

message AlPolicySetupOutMsg {
    oneof type_of_message {
        AlPolicySetupResult al_policy_setup_result = 1;
    }
}

message AlPolicyUpdateInitMsg {
    optional AlPolicyUpdateRequest al_policy_update_request = 1;
}

message AlPolicyUpdateOutMsg {
    oneof type_of_message {
        AlPolicyUpdateResult al_policy_update_result = 1;
    }
}

message AlPolicyDeleteInitMsg {
    optional AlPolicyDeleteRequest al_policy_delete_request = 1;
}

message AlPolicyDeleteOutMsg {
    oneof type_of_message {
        AlPolicyDeleteResult al_policy_delete_result = 1;
    }
}
```

```
// ----- messages of al_policy_API -----
// ----- messages of Al Policy Setup procedure -----
message AlPolicySetupRequest {
    optional near_rt_ric_apis_common_ies.v2.AlPolicyTypeId al_policy_type_id = 1;
    optional al_related_apis_common_ies.v1.AlPolicyId al_policy_id = 2;
    optional al_related_apis_common_ies.v1.AlPolicyObject al_policy_object = 3;
}

message AlPolicySetupResult {
    optional near_rt_ric_apis_common_ies.v2.AlPolicyTypeId al_policy_type_id = 1;
    optional al_related_apis_common_ies.v1.AlPolicyId al_policy_id = 2;
    optional al_related_apis_common_ies.v1.Result al_policy_setup_result = 3;
    optional al_related_apis_common_ies.v1.Cause cause = 4;
    optional al_related_apis_common_ies.v1.AlPolicyObject al_policy_object = 5;
}

// ----- messages of Al Policy Update procedure -----
message AlPolicyUpdateRequest {
    optional near_rt_ric_apis_common_ies.v2.AlPolicyTypeId al_policy_type_id = 1;
    optional al_related_apis_common_ies.v1.AlPolicyId al_policy_id = 2;
    optional al_related_apis_common_ies.v1.AlPolicyObject al_policy_object = 3;
}

message AlPolicyUpdateResult {
    optional near_rt_ric_apis_common_ies.v2.AlPolicyTypeId al_policy_type_id = 1;
    optional al_related_apis_common_ies.v1.AlPolicyId al_policy_id = 2;
    optional al_related_apis_common_ies.v1.Result al_policy_update_result = 3;
    optional al_related_apis_common_ies.v1.Cause cause = 4;
    optional al_related_apis_common_ies.v1.AlPolicyObject al_policy_object = 5;
}

// ----- messages of Al Policy Delete procedure -----
message AlPolicyDeleteRequest {
    optional near_rt_ric_apis_common_ies.v2.AlPolicyTypeId al_policy_type_id = 1;
    optional al_related_apis_common_ies.v1.AlPolicyId al_policy_id = 2;
}

message AlPolicyDeleteResult {
    optional near_rt_ric_apis_common_ies.v2.AlPolicyTypeId al_policy_type_id = 1;
    optional al_related_apis_common_ies.v1.AlPolicyId al_policy_id = 2;
    optional al_related_apis_common_ies.v1.Result al_policy_delete_result = 3;
    optional al_related_apis_common_ies.v1.Cause cause = 4;
}
```

5.3.4.1.3 Information Element Definitions

```
syntax = "proto3";

package ricapi.al_related_apis_common_ies.v1;

import "near_rt_ric_apis_common_ies_ver_2_0_0.proto";
// ----- Information Elements -----
// -----A-----
message AlPolicyObject {
    optional string value = 1;
}

message AlPolicyId {
    optional string value = 1;
}

// -----C-----
message Cause {
    oneof cause_group {
        AlRelatedApiLayerCause al_related_api_layer_cause = 1;
        TransportLayerCause transport_layer_cause = 2;
        ProtocolLayerCause protocol_layer_cause = 3;
        MiscCause misc_cause = 4;
    }

    enum AlRelatedApiLayerCause {
```

```

    A1_RELATED_API_LAYER_UNSPECIFIED = 0;
    NOT_SUPPORTED_A1_POLICY_TYPE = 1;
    NOT_ALLOWED_STATEMENTS_RESTRICTIONS_AND_EXTENSIONS = 2;
    NEAR_RT_RIC_PLATFORM_CANNOT_ACCOMMODATE_E2_A1_EI_OPERATIONS_REQUIREMENTS = 3;
    A1_POLICY_ALREADY_ENABLED_AT_XAPP = 4;
    NOT_ENABLED_A1_POLICY_AT_XAPP = 5;
}

enum TransportLayerCause {
    TRANSPORT_UNSPECIFIED = 0;
    TRANSPORT_RESOURCE_UNAVAILABLE = 1;
}

enum ProtocolLayerCause {
    PROTOCOL_UNSPECIFIED = 0;
    TRANSFER_SYNTAX_ERROR = 1;
    ABSTRACT_SYNTAX_ERROR_REJECT = 2;
    ABSTRACT_SYNTAX_ERROR_IGNORE_AND_NOTIFY = 3;
    MESSAGE_NOT_COMPATIBLE_WITH_RECEIVER_STATE = 4;
    SEMANTIC_ERROR = 5;
    ABSTRACT_SYNTAX_ERROR_FALSEELY_CONSTRUCTED_MESSAGE = 6;
}

enum MiscCause {
    MISC_UNSPECIFIED = 0;
    CONTROL_PROCESSING_OVERLOAD = 1;
    HARDWARE_FAILURE = 2;
    OAM_INTERVENTION = 3;
}
}

// -----R-----
enum Result {
    RESULT_SUCCESS = 0;
    RESULT_FAILURE = 1;
}

```

5.3.5 Message Transfer Syntax

5.3.5.1 Solution 1: gRPC with Protocol Buffers

The message transfer syntax for Solution 1 is serialized Protocol Buffers [4].

6 E2-related API

6.1 Overview

6.1.1 Protocol Stack

In this version of specification, the default and optional protocol stacks for the E2 related APIs are shown on Figure 6.1.1-1.

Note: the network layer for security protection is FFS.

	Application		Application	
	SCTP		gRPC	
	IP		HTTP/2	
	Data Link Layer		TCP	
	Physical Layer		IP	
Default protocol stack : SCTP			Optional protocol stack : gRPC	

Figure 6.1.1-1: Protocol stacks for E2 related APIs

The E2 related APIs may use SCTP or gRPC with Protocol Buffers [4] as the application layer serialization protocol. The supported protocol by the platform APIs is presented to the the xApps during the xApp Discovery Procedure in clause 9.2.1.

6.1.2 List of E2 related APIs, procedures and messages

The E2 related API procedures introduced in clause 9.3 of [2] are categorized under the E2 related APIs. These APIs include:

- E2 Subscription API: enabling xApp to setup, modify and delete E2 subscriptions with the Near-RT RIC platform.
- E2 Indication API: enabling xApp to receive E2 indications associated with an E2 subscription.
- E2 Control API: enabling xApp to consume E2 control service from the Near-RT RIC platform.
- E2 Guidance API: enabling xApp to acquire E2 related guidance from the Near-RT RIC platform and mitigate potential conflicts with other xApps.

These APIs are summarized in Table 6.1.2-1.

Table 6.1.2-1: List of E2 related APIs

API Name	Description
E2_Subscription_API	Near-RT RIC platform service for subscription of E2 REPORT, INSERT and POLICY services
E2_Indication_API	Near-RT RIC platform service for delivery of subscribed information associated with E2 REPORT and INSERT services
E2_Control_API	Near-RT RIC platform service for E2 CONTROL
E2_Guidance_API	Near-RT RIC platform service for E2 related guidance

The table 6.1.2-2 lists the API procedures for each E2 related API.

Table 6.1.2-2: List of E2 related API procedures

API Name	API procedure	Communication type
E2_Subscription_API	E2 Subscription	Request/Response
	E2 Subscription Delete	Request/Response
	E2 Subscription Delete Query	Request/Response
	E2 Subscription Delete Notification	Notify
E2_Indication_API	E2 Indication	Notify
E2_Control_API	E2 Control	Request/Response
E2_Guidance_API	E2 Guidance	Request/Response
	E2 Guidance Modification	Notify

The table 6.1.2-3 lists the messages for each E2 related API procedure.

Table 6.1.2-3: List of E2 related API messages

API Procedure	Initiated by	Initiating Message	Outcome Message		
			Successful Outcome	Unsuccessful Outcome 1	Unsuccessful Outcome 2
E2 Subscription	xApp	E2 SUBSCRIPTION REQUEST	E2 SUBSCRIPTION SUCCESS	E2 SUBSCRIPTION FAILURE	E2 SUBSCRIPTION REJECT
E2 Subscription Delete	xApp	E2 SUBSCRIPTION DELETE REQUEST	E2 SUBSCRIPTION DELETE SUCCESS	E2 SUBSCRIPTION DELETE FAILURE	E2 SUBSCRIPTION DELETE REJECT
E2 Subscription Delete Query	Near-RT RIC platform	E2 SUBSCRIPTION DELETE REQUIRED	E2 SUBSCRIPTION DELETE ACCEPT	E2 SUBSCRIPTION DELETE DECLINE	-
E2 Subscription Delete Notification	Near-RT RIC platform	E2 SUBSCRIPTION DELETE NOTIFICATION	-	-	-
E2 Indication	Near-RT RIC platform	E2 INDICATION PUSH	-	E2 INDICATION FAILURE	-
E2 Control	xApp	E2 CONTROL REQUEST	E2 CONTROL SUCCESS	E2 CONTROL FAILURE	E2 CONTROL REJECT
E2 Guidance	xApp	E2 GUIDANCE REQUEST	E2 GUIDANCE SUCCESS	-	-
E2 Guidance Modification	Near-RT RIC platform	E2 GUIDANCE MODIFICATION	-	-	-

6.2 E2-related API Procedures

6.2.1 E2_Subscription_API procedures

6.2.1.1 E2 Subscription Procedure

6.2.1.1.1 General

This procedure is used by an xApp to request an E2 subscription to Near-RT RIC platform.

6.2.1.1.2 Successful Operation

```
@startuml
Skin rose
skinparam ParticipantPadding 50
skinparam BoxPadding 10
skinparam lifelineStrategy solid
participant "xApp" as app
participant "Near-RT RIC Platform" as near
app -> near: E2 SUBSCRIPTION (Request)
near -> app: E2 SUBSCRIPTION (Success)
@enduml
```

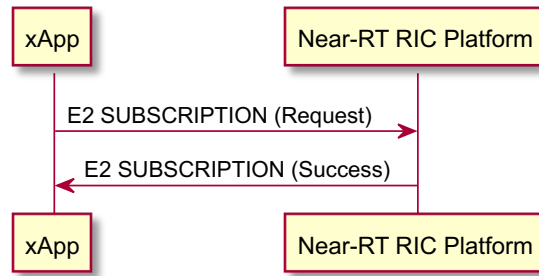


Figure 6.2.1.1.2-1: E2 Subscription procedure, successful operation

The xApp initiates the procedure by sending an E2 SUBSCRIPTION (Request) message to the Near-RT-RIC platform. The message contains a unique *Procedure Transaction ID* IE assigned by the initiating xApp, the *Global E2 Node ID* IE, the *RAN Function ID* IE, and the *E2 Subscription Details* IE including one or more requested E2 actions and optionally the *E2 Subscription Timer* IE. The message may include the *E2 Indication Destination* IE which specifies the xApp's endpoint to receive E2 indications (see clause 6.2.2.1), and the *E2 Subscription Delete Notification Destination* IE which specifies the xApp's endpoint to receive query/notification from the Near-RT RIC platform for E2 subscription deletion (see clause 6.2.1.3/6.2.1.4).

Note: When execution order of multiple actions is not concerned, it is recommended that an E2 Subscription procedure contains a single action, to allow efficient reuse of E2 subscriptions.

At reception of the E2 SUBSCRIPTION (Request) message, the Near-RT-RIC platform shall:

- Search existing E2 subscription(s) for matched *Global E2 Node ID* IE, *RAN Function ID* IE and matched contents in *E2 Subscription Details* IE. If a matched E2 subscription exists, add the initiating xApp as a new subscriber of the E2 subscription.
- Initiate appropriate E2AP procedure(s) to establish RIC subscription with the target E2 Node, if existing E2 subscriptions are insufficient to fulfill the request.

The Near-RT-RIC platform may use the value in the *E2 Subscription Timer* IE, if present, as a one-time value of the E2AP timer $T_{RICEVENTcreate}$ in clause 9.5 of [3], which is valid only for this transaction.

If the subscription request is successfully fulfilled, the Near-RT RIC platform shall send an E2 SUBSCRIPTION (Success) message back to the xApp. Similar to [3], a successfully fulfilled request shall be sent if the E2 event trigger and at least one requested E2 action are admitted.

In the E2 SUBSCRIPTION (Success) message, Near-RT RIC platform shall include the associated *Procedure Transaction ID* IE, *Global E2 Node ID* IE, *RAN Function ID* IE and an *E2 Request ID* IE assigned by the Near-RT RIC platform. The message shall also indicate the admitted E2 action(s) in the *Actions Admitted List* IE, each with the associated *E2 Action ID* IE assigned by the Near-RT RIC platform. The same *E2 Request ID* IE and *E2 Action ID* IE shall be present in any subsequent E2 INDICATION message(s). The message shall indicate, if exist, the E2 action(s) not admitted in the *Actions Not Admitted List* IE with appropriate cause(s).

6.2.1.1.3 Unsuccessful Operation

```

@startuml
Skin rose
skinparam ParticipantPadding 50
skinparam BoxPadding 10
skinparam lifelineStrategy solid
participant "xApp" as app
participant "Near-RT RIC Platform" as near
app -> near: E2 SUBSCRIPTION (Request)
near -> app: E2 SUBSCRIPTION (Reject)
@enduml

```

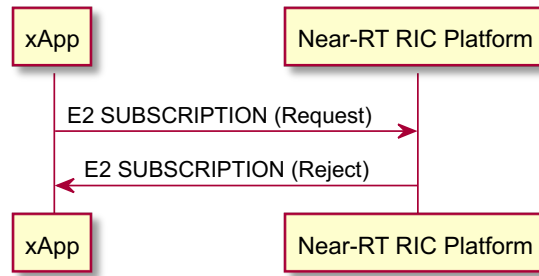


Figure 6.2.1.1.3-1: E2 Subscription procedure, unsuccessful operation (Reject)

If the subscription request is rejected by the Near-RT RIC platform, an E2 SUBSCRIPTION (Reject) message shall be sent back to the xApp. The message shall include associated *Procedure Transaction ID* IE which is replicated from the corresponding request, the *Global E2 Node ID* IE, the *RAN Function ID* IE and appropriate *Cause* IE.

```
@startuml
Skin rose
skinparam ParticipantPadding 50
skinparam BoxPadding 10
skinparam lifelineStrategy solid
participant "xApp" as app
participant "Near-RT RIC Platform" as near
app -> near: E2 SUBSCRIPTION (Request)
near -> app: E2 SUBSCRIPTION (Failure)
@enduml
```

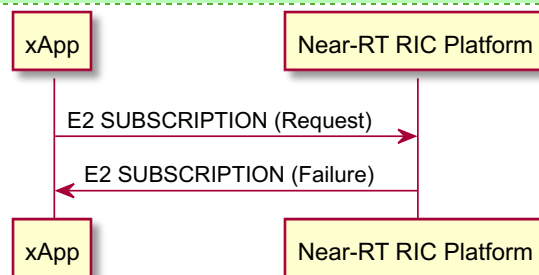


Figure 6.2.1.1.3-2: E2 Subscription procedure, unsuccessful operation (Failure)

If the subscription request fails due to failure in E2AP procedure, the Near-RT RIC platform shall send an E2 SUBSCRIPTION (Failure) message back to the xApp. The message shall include the associated *Procedure Transaction ID* IE which is replicated from the corresponding request, *Global E2 Node ID* IE, *RAN Function ID* IE and appropriate cause.

6.2.1.1.4 Abnormal Conditions

Not applicable.

6.2.1.2 E2 Subscription Delete Procedure, xApp initiated

6.2.1.2.1 General

This procedure is used by an xApp to unsubscribe from an E2 subscription on Near-RT RIC platform.

6.2.1.2.2 Successful Operation

```
@startuml
Skin rose
skinparam ParticipantPadding 50
skinparam BoxPadding 10
skinparam lifelineStrategy solid
participant "xApp" as app
participant "Near-RT RIC Platform" as near
app -> near: E2 SUBSCRIPTION DELETE (Request)
@enduml
```



```

near -> app: E2 SUBSCRIPTION DELETE (Success)
@enduml

```

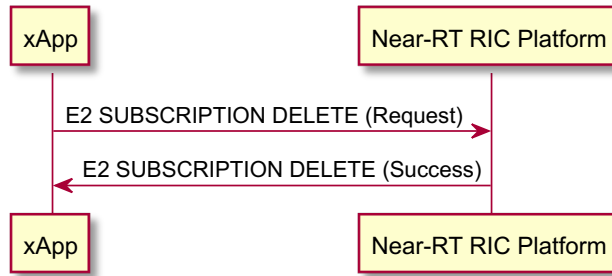


Figure 6.2.1.2.2-1: E2 Subscription Delete procedure (xApp initiated), successful operation

The xApp initiates the procedure by sending the E2 SUBSCRIPTION DELETE (Request) message to the Near-RT-RIC platform. The message contains the *Global E2 Node ID* IE and the *E2 Request ID* IE to indicate the target E2 subscription and optionally the *E2 Subscription Delete Timer* IE.

At reception of the E2 SUBSCRIPTION DELETE (Request) message, the Near-RT-RIC platform shall, if the delete request is accepted by the Near-RT-RIC platform:

- Check for other subscriber(s) of the target E2 subscription. If such subscriber(s) exists, the E2 subscription shall be maintained by the Near-RT RIC platform but no longer serve the initiating xApp; if not, the E2 subscription shall be deleted.
- Initiate appropriate E2AP procedure(s) with the target E2 Node, if necessary, to map the change in the target E2 subscription to its corresponding RIC subscription.

The Near-RT-RIC platform may use the value in the *E2 Subscription Delete Timer* IE, if present, as a one-time value of the E2AP timer $T_{\text{RICEVENTdelete}}$ in clause 9.5 of [3], which is valid only for this transaction.

Upon success of the above processing for the target E2 subscription, the Near-RT RIC platform shall stop delivering its associated E2 INDICATION message(s) to the initiating xApp and respond the xApp with an E2 SUBSCRIPTION DELETE(Success) message. The E2 SUBSCRIPTION DELETE (Success) message shall include the same *Global E2 Node ID* IE and *E2 Request ID* IE from the request message.

6.2.1.2.3 Unsuccessful Operation

```

@startuml
Skin rose
skinparam ParticipantPadding 50
skinparam BoxPadding 10
skinparam lifelineStrategy solid
participant "xApp" as app
participant "Near-RT RIC Platform" as near
app -> near: E2 SUBSCRIPTION DELETE (Request)
near -> app: E2 SUBSCRIPTION DELETE (Reject)
@enduml

```

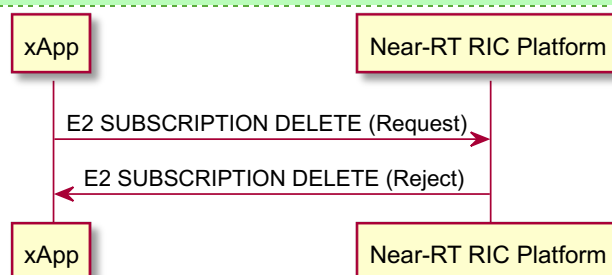


Figure 6.2.1.2.3-1: E2 Subscription Delete procedure (xApp initiated), unsuccessful operation (Reject)

If the Near-RT RIC platform rejects the request message, an E2 SUBSCRIPTION (Reject) message shall be sent back to the initiating xApp. The message shall include associated *Procedure Transaction ID* IE which is replicated from the corresponding request, the *Global E2 Node ID* IE, the *RAN Function ID* IE and appropriate *Cause* IE.

```
@startuml
Skin rose
skinparam ParticipantPadding 50
skinparam BoxPadding 10
skinparam lifelineStrategy solid
participant "xApp" as app
participant "Near-RT RIC Platform" as near
app -> near: E2 SUBSCRIPTION DELETE (Request)
near -> app: E2 SUBSCRIPTION DELETE (Failure)
@enduml
```

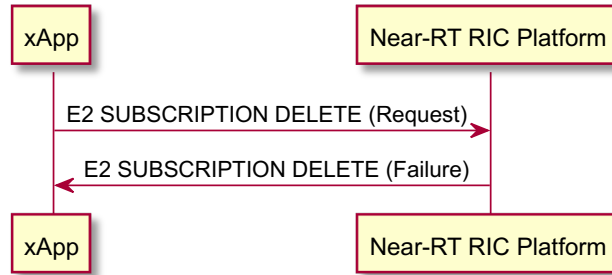


Figure 6.2.1.2.3-2: E2 Subscription Delete procedure (xApp initiated), unsuccessful operation (Failure)

If the associated E2AP procedure(s) fails for the E2 subscription, the Near-RT RIC platform shall send an E2 SUBSCRIPTION DELETE (Failure) message back to the initiating xApp. The message shall include associated *Procedure Transaction ID* IE which is replicated from the corresponding request, the *Global E2 Node ID* IE, the *RAN Function ID* IE and appropriate *Cause* IE.

6.2.1.2.4 Abnormal Conditions

Not applicable.

6.2.1.3 E2 Subscription Delete Query procedure

6.2.1.3.1 General

This procedure is used by Near-RT RIC platform to consult the xApp about deleting an existing E2 subscription.

6.2.1.3.2 Successful Operations

```
@startuml
Skin rose
skinparam ParticipantPadding 50
skinparam BoxPadding 10
skinparam lifelineStrategy solid
participant "xApp" as app
participant "Near-RT RIC Platform" as near
near -> app: E2 SUBSCRIPTION DELETE QUERY (Required)
app -> near: E2 SUBSCRIPTION DELETE QUERY (Accept)
@enduml
```

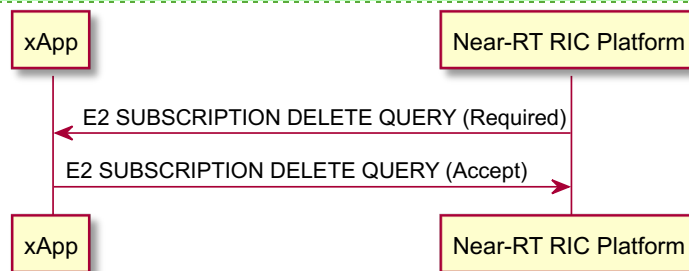


Figure 6.2.1.3.2-1: E2 Subscription Delete Query procedure, successful operation

The Near-RT RIC platform may consult each concerned xApp before deciding to delete an E2 subscription. The Near-RT RIC platform initiates the procedure by sending an E2 SUBSCRIPTION DELETE Query (Required) message to the xApp. The message contains the *Global E2 Node ID* IE and the *E2 Request ID* IE to identify the target E2 subscription.

At reception of the E2 SUBSCRIPTION DELETE Query (Required) message, the xApp shall respond by an E2 SUBSCRIPTION DELETE Query (Accept) message, if the xApp agrees to delete the E2 subscription based on its own evaluation. The reply message shall contain the same *Global E2 Node ID* IE, *E2 Request ID* IE, and *RAN Function ID* IE as in the corresponding E2 SUBSCRIPTION DELETE (Required) message. The Near-RT RIC platform should consider the suggestions from all concerned xApps, but the decision for the E2 subscription is up to the Near-RT RIC platform.

6.2.1.3.3 Unsuccessful Operation

If the xApp disagrees to delete the E2 subscription proposed in the E2 SUBSCRIPTION DELETE Query (Required) message, it shall reply an E2 SUBSCRIPTION DELETE Query (Decline) message to the Near-RT RIC platform.

```
@startuml
Skin rose
skinparam ParticipantPadding 50
skinparam BoxPadding 10
skinparam lifelineStrategy solid
participant "xApp" as app
participant "Near-RT RIC Platform" as near
near -> app: E2 SUBSCRIPTION DELETE QUERY (Required)
app -> near: E2 SUBSCRIPTION DELETE QUERY (Decline)
@enduml
```

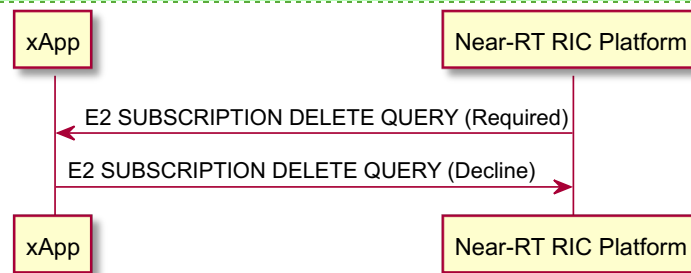


Figure 6.2.1.3.3-2: E2 Subscription Delete Query procedure, unsuccessful operation

6.2.1.3.4 Abnormal Conditions

Not applicable.

6.2.1.4 E2 Subscription Delete Notification procedure

6.2.1.4.1 General

This procedure is used by Near-RT RIC platform to notify the xApp that an existing E2 subscription has been deleted as required by the E2 Node.

6.2.1.4.2 Successful Operations

```
@startuml
Skin rose
skinparam ParticipantPadding 50
skinparam BoxPadding 10
skinparam lifelineStrategy solid
participant "xApp" as app
participant "Near-RT RIC Platform" as near
near -> app: E2 SUBSCRIPTION DELETE NOTIFICATION (Deleted)
@enduml
```

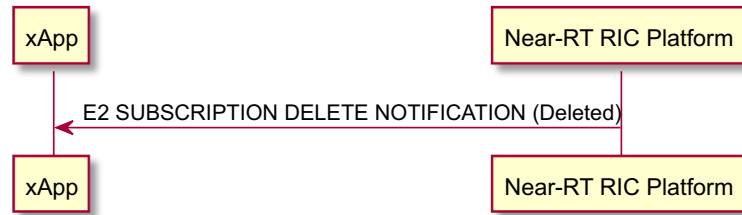


Figure 6.2.1.4.2-1: E2 Subscription Delete Notification procedure, successful operation

After successfully deleting the E2 subscription required by the E2 Node (including removal of the RIC subscription over E2 interface and removal of corresponding subscription record in Near-RT RIC platform), the Near-RT RIC platform shall send an E2 SUBSCRIPTION DELETE NOTIFICATION (Deleted) message to the xApp to notify the deletion.

The message shall contain the *Global E2 Node ID* IE and the *E2 Request ID* IE to identify the target E2 subscription, and the *Cause IE* with appropriate value.

At reception of the message, the xApp shall remove its local subscription record of the E2 subscription.

6.2.1.4.3 Unsuccessful Operations

Not applicable.

6.2.1.4.4 Abnormal Conditions

Not applicable.

6.2.2 E2_Indication_API procedures

6.2.2.1 E2 Indication Procedure

6.2.2.1.1 General

This procedure is used by the Near-RT RIC platform to deliver to xApp the subscribed information associated with an E2 subscription. The procedure is triggered by the successful operation of an E2AP RIC indication procedure.

6.2.2.1.2 Successful Operation

```

@startuml
Skin rose
skinparam ParticipantPadding 50
skinparam BoxPadding 10
skinparam lifelineStrategy solid
participant "xApp" as app
participant "Near-RT RIC Platform" as near
near -> app: E2 INDICATION (Push)
@enduml
  
```

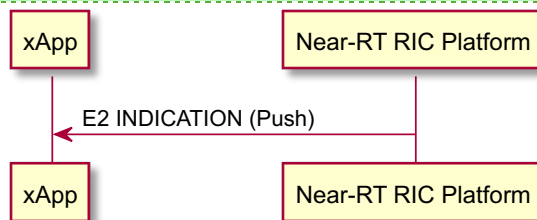


Figure 6.2.2.1.2-1: E2 Indication procedure, successful operation

The Near-RT-RIC platform initiates the procedure by sending the E2 INDICATION (Push) message containing the *Global E2 Node ID* IE, the *E2 Request ID* IE, the *E2 Action ID* IE to the xApp. The message shall also contain the *E2 Indication Type* IE, the *E2 Indication Header* IE and the *E2 Indication Message* IE to carry the information from corresponding fields, i.e. the *RIC*

Indication Type IE, the RIC Indication Header IE, and the RIC Indication Message IE, respectively, in the E2AP RIC INDICATION message. The E2 Indication SN IE and the E2 Call Process ID IE shall also be included, if corresponding fields, i.e. the RIC Indication SN IE, and the RIC Call Process ID IE, respectively, are present in the E2AP RIC INDICATION message.

At reception of the E2 INDICATION (Push) message an xApp shall:

- Use the *Global E2 Node ID* IE, the *E2 Request ID* IE and the *E2 Action ID* IE to associate the received message to the right E2 subscription. The *E2 Request ID* IE and the *E2 Action ID* IE have been assigned in the E2 SUBSCRIPTION (Success) message for each E2 subscription.
- Process the information carried in the *E2 Indication SN* IE, the *E2 Indication Type* IE, the *E2 Indication Header* IE, the *E2 Indication Message* IE and the *E2 Call Process ID* IE.

6.2.2.1.3 Unsuccessful Operation

```
@startuml
Skin rose
skinparam ParticipantPadding 50
skinparam BoxPadding 10
skinparam lifelineStrategy solid
participant "xApp" as app
participant "Near-RT RIC Platform" as near
near -> app: E2 INDICATION (Push)
app --> near: E2 INDICATION (Failure)
@enduml
```

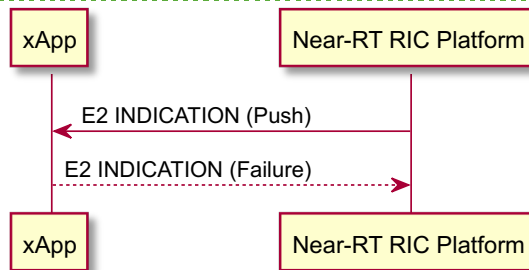


Figure 6.2.2.1.3-1: E2 Indication procedure, unsuccessful operation (Failure)

If the received E2 INDICATION (Push) message cannot be handled successfully as specified in 6.2.2.1.2, the xApp may, according to operator's OM configuration, send E2 INDICATION (Failure) message back to the Near-RT RIC platform. The message shall include the associated *Global E2 Node ID* IE, *E2 Request ID* IE, *RAN Function ID* IE, *E2 Action ID* IE, and appropriate *Cause* IE. The *E2 Indication SN* IE shall also be included if the same name IE is present in the corresponding E2 INDICATION (Push) message.

6.2.2.1.4 Abnormal Conditions

Not applicable.

6.2.3 E2_Control_API procedures

6.2.3.1 E2 Control procedure

6.2.3.1.1 General

This procedure is used by an xApp to request for E2 control service to Near-RT RIC platform.

6.2.3.1.2 Successful Operation

```
@startuml
Skin rose
skinparam ParticipantPadding 50
```

```

skinparam BoxPadding 10
skinparam lifelineStrategy solid
participant "xApp" as app
participant "Near-RT RIC Platform" as near
app -> near: E2 CONTROL (Request)
near -> app: E2 CONTROL (Success)
@enduml

```

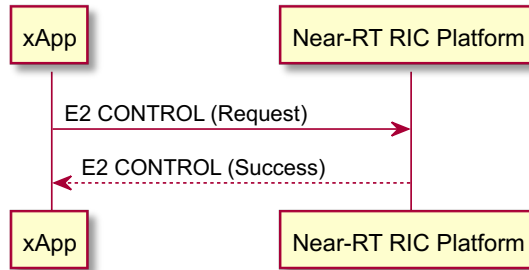


Figure 6.2.3.1.2-1: E2 Control procedure, successful operation

The xApp initiates the procedure by sending the E2 CONTROL (Request) message to the Near-RT-RIC platform, containing the *Global E2 Node ID* IE, the *RAN Function ID* IE, optionally the *E2 Call Process ID* IE, *E2 Control Header* IE, *E2 Control Message* IE, optionally the *E2 Control Ack Request* IE and optionally the *E2 Control Timer* IE.

At reception of the E2 CONTROL (Request) message, the Near-RT-RIC platform shall:

- Evaluate whether to reject the request due to potential control conflict, when such evaluation is required by conflict mitigation related configuration.
- Initiate appropriate E2AP RIC Control procedure with the target E2 Node, if the request is not rejected by the Near-RT-RIC platform.

The Near-RT-RIC platform may use the value in the *E2 Control Timer* IE, if present, as a one-time value of the E2AP timer $T_{RICcontrol}$ in Clause 9.5 of [3], which is valid only for this transaction.

If the control request is successfully performed, and if the optional *E2 Control Ack Request* IE in the request is absent or set to “Ack”, an E2 CONTROL (Success) message shall be responded to the initiating xApp. The E2 CONTROL (Success) message shall include the *Global E2 Node ID* IE, the *RAN Function ID* IE, optionally the *E2 Call Process ID* IE, and optionally the *E2 Control Outcome* IE. The xApp may use the information contained in the optional *E2 Control Outcome* IE to determine subsequent actions.

If the control request is successfully performed, and if the optional *E2 Control Ack Request* IE in the request is set to “NoAck”, an E2 CONTROL (Success) message is not needed and xApp may consider it success.

6.2.3.1.3 Unsuccessful Operations

```

@startuml
Skin rose
skinparam ParticipantPadding 50
skinparam BoxPadding 10
skinparam lifelineStrategy solid
participant "xApp" as app
participant "Near-RT RIC Platform" as near
app -> near: E2 CONTROL (Request)
near -> app: E2 CONTROL (Reject)
@enduml

```

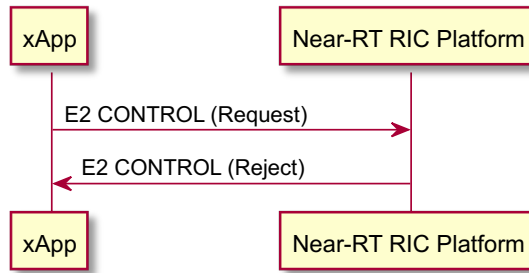


Figure 6.2.3.1.3-1: E2 Control procedure, unsuccessful operation (Reject)

If the request is rejected by the Near-RT RIC platform, an E2 CONTROL (Reject) message shall be responded to the initiating xApp with appropriate *Cause* IE. The xApp may use the information contained in the *Cause* IE to determine subsequent actions.

```

@startuml
Skin rose
skinparam ParticipantPadding 50
skinparam BoxPadding 10
skinparam lifelineStrategy solid
participant "xApp" as app
participant "Near-RT RIC Platform" as near
app -> near: E2 CONTROL (Request)
near -> app: E2 CONTROL (Failure)
@enduml
  
```

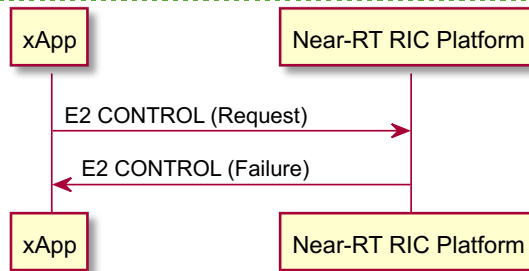


Figure 6.2.3.1.3-2: E2 Control procedure, unsuccessful operation (Failure)

If the request is accepted by the Near-RT RIC platform but failed in the E2AP RIC Control procedure, the Near-RT RIC platform shall send an E2 CONTROL (Failure) message back to the xApp with appropriate *Cause* IE. The xApp may use the information contained in the *Cause* IE and the optional *E2 Control Outcome* IE to determine subsequent actions.

6.2.3.1.4 Abnormal Conditions

Not applicable.

6.3 Elements for E2-related API Communication

6.3.1 General

6.3.2 Message Functional Definition and Content

6.3.2.1 E2 SUBSCRIPTION (Request)

This message is sent by an xApp to the Near-RT RIC platform for an E2 Subscription.

Direction: xApp -> Near-RT RIC platform

IE/Group Name	Presence	Range	IE type and reference	Semantics description	Criticality	Assigned Criticality
Message Type	M		6.3.3.1		YES	Reject
Procedure Transaction ID	M		10.1.1	Value assigned by the initiating xApp.	YES	Reject
Global E2 Node ID	M		10.1.6		YES	Reject
RAN Function ID	M		10.1.7		YES	Reject
E2 Subscription Details					YES	Reject
>E2 Event Trigger Definition	M		RIC Event Trigger Definition, Clause 9.2.9 of [3]		-	
>Sequence of Actions		1.. <maxnofE2Actions>			EACH	Ignore
>>Requested E2 Action ID	M		RIC Action ID, Clause 9.2.10 of [3]	Assigned by initiating xApp to uniquely identify an action within the sequence of actions. It should not be assumed that the E2 Action ID in the subsequent E2 SUBSCRIPTION (Success) message or RIC Action ID in the corresponding E2AP message has the same value.	-	
>>E2 Action Type	M		RIC Action Type, Clause 9.2.11 of [3]		-	
>>E2 Action Definition	O		RIC Action Definition, Clause 9.2.12 of [3]		-	
>>E2 Subsequent Action	O		RIC Subscription Action, Clause 9.2.13 of [3]		-	
E2 Subscription Timer	O		6.3.3.7			
E2 Indication Destination	O		10.1.9	When present, this IE specifies the destination address for the Near-RT RIC platform to send E2 INDICATION (Push) messages.	YES	Reject
E2 Subscription Delete Notification Destination	O		10.1.9	When present, this IE specifies the destination address for the Near-RT RIC platform to send E2 SUBSCRIPTION DELETE QUERY (Required) and E2 SUBSCRIPTION	YES	Reject

				DELETE NOTIFICATION (Deleted) messages.		
--	--	--	--	--	--	--

Range bound	Explanation
maxnoofE2Actions	Maximum no. of E2 Actions requested in the E2 subscription. Value is 16.

6.3.2.2 E2 SUBSCRIPTION (Success)

This message is sent by the Near-RT RIC platform after the xApp's subscription request is successfully accepted with respect to the target E2 Node.

Direction: Near-RT RIC platform -> xApp

IE/Group Name	Presence	Range	IE type and reference	Semantics description	Criticality	Assigned Criticality
Message Type	M		6.3.3.1		YES	Reject
Procedure Transaction ID	M		10.1.1	Value replicated from corresponding request.	YES	Reject
Global E2 Node ID	M		10.1.6	Value replicated from corresponding request.	YES	Reject
E2 Request ID	M		6.3.3.2	Value assigned by the Near-RT RIC platform to indicate the associated E2 subscription in Near-RT RIC.	YES	Reject
RAN Function ID	M		10.1.7	Value replicated from corresponding request.	YES	Reject
Actions Admitted List		1.. <maxnoofE2Actions>			YES	Reject
>Requested E2 Action ID	M		RIC Action ID, Clause 9.2.10 of [3]	Value replicated from corresponding request message	-	-
>E2 Action ID	M		RIC Action ID, Clause 9.2.10 of [3]	Value assigned by Near-RT RIC platform to indicate in the associated E2 subscription the action ID that corresponds to the <i>Requested E2 Action ID</i> IE in the E2 SUBSCRIPTION REQUEST message.	-	-
Actions Not Admitted List		0.. <maxnoofE2Actions>			YES	Reject
>Requested E2 Action ID	M		RIC Action ID, Clause 9.2.10 of [3]	Value replicated from corresponding request message	-	-
>Cause	M		6.3.3.4		-	-

Range bound	Explanation
maxnoofE2Actions	Maximum no. of E2 Actions requested in the E2 subscription. Value is 16.

6.3.2.3 E2 SUBSCRIPTION (Reject)

This message is sent by the Near-RT RIC platform after the xApp's subscription request is rejected by Near-RT RIC platform with respect to the target E2 Node.

Direction: Near-RT RIC platform -> xApp

IE/Group Name	Presence	Range	IE type and reference	Semantics description	Criticality	Assigned Criticality
Message Type	M		6.3.3.1		YES	Reject
Procedure Transaction ID	M		10.1.1	Value replicated from corresponding request.	YES	Reject
Global E2 Node ID	M		10.1.6	Value replicated from corresponding request.	YES	Reject
RAN Function ID	M		10.1.7	Value replicated from corresponding request.	YES	Reject
Cause	M		6.3.3.4		YES	Reject
Criticality Diagnostics	O		6.3.3.5		YES	Ignore

6.3.2.4 E2 SUBSCRIPTION (Failure)

This message is sent by the Near-RT RIC platform after the xApp's subscription request is accepted by Near-RT RIC platform but failed by E2 Node, with respect to the target E2 Node.

Direction: Near-RT RIC platform -> xApp

IE/Group Name	Presence	Range	IE type and reference	Semantics description	Criticality	Assigned Criticality
Message Type	M		6.3.3.1		YES	Reject
Procedure Transaction ID	M		10.1.1	Value replicated from corresponding request.	YES	Reject
Global E2 Node ID	M		10.1.6	Value replicated from corresponding request.	YES	Reject
RAN Function ID	M		10.1.7	Value replicated from corresponding request.	YES	Reject
Cause	M		6.3.3.4		YES	Reject
Criticality Diagnostics	O		6.3.3.5		YES	Ignore

6.3.2.5 E2 SUBSCRIPTION DELETE (Request)

This message is sent by the xApp to the Near-RT RIC platform to request deletion of one target E2 subscription.

Direction: xApp -> Near-RT RIC platform

IE/Group Name	Presence	Range	IE type and reference	Semantics description	Criticality	Assigned Criticality
Message Type	M		6.3.3.1		YES	Reject

Procedure Transaction ID	M		10.1.1		YES	Reject
Global E2 Node ID	M		10.1.6		YES	Reject
E2 Request ID	M		6.3.3.2		YES	Reject
RAN Function ID	M		10.1.7		YES	Reject
E2 Subscription Delete Timer	O		6.3.3.8			

6.3.2.6 E2 SUBSCRIPTION DELETE (Success)

This message is sent by the Near-RT RIC platform to the xApp to indicate success for the delete request of the target E2 subscription.

Direction: Near-RT RIC platform -> xApp

IE/Group Name	Presence	Range	IE type and reference	Semantics description	Criticality	Assigned Criticality
Message Type	M		6.3.3.1		YES	Reject
Procedure Transaction ID	M		10.1.1	Value replicated from corresponding request.	YES	Reject
Global E2 Node ID	M		10.1.6	Value replicated from corresponding request.	YES	Reject
E2 Request ID	M		6.3.3.2	Value replicated from corresponding request.	YES	Reject
RAN Function ID	M		10.1.7	Value replicated from corresponding request.	YES	Reject

6.3.2.7 E2 SUBSCRIPTION DELETE (Reject)

This message is sent by the Near-RT RIC platform to the xApp when the Near-RT RIC platform rejects the delete request for the target E2 subscription.

Direction: Near-RT RIC platform -> xApp

IE/Group Name	Presence	Range	IE type and reference	Semantics description	Criticality	Assigned Criticality
Message Type	M		6.3.3.1		YES	Reject
Procedure Transaction ID	M		10.1.1	Value replicated from corresponding request.	YES	Reject
Global E2 Node ID	M		10.1.6	Value replicated from corresponding request.	YES	Reject
E2 Request ID	M		6.3.3.2	Value replicated from corresponding request.	YES	Reject
RAN Function ID	M		10.1.7	Value replicated from corresponding request.	YES	Reject
Cause	M		6.3.3.4		YES	Ignore
Criticality Diagnostics	O		6.3.3.5		YES	Ignore

6.3.2.8 E2 SUBSCRIPTION DELETE (Failure)

This message is sent by the Near-RT RIC platform to the xApp when the delete request for a target E2 subscription fails due to failure in the associated E2AP procedure(s).

Direction: Near-RT RIC platform -> xApp

IE/Group Name	Presence	Range	IE type and reference	Semantics description	Criticality	Assigned Criticality
Message Type	M		6.3.3.1		YES	Reject
Procedure Transaction ID	M		10.1.1	Value replicated from corresponding request.	YES	Reject
Global E2 Node ID	M		10.1.6	Value replicated from corresponding request.	YES	Reject
E2 Request ID	M		6.3.3.2	Value replicated from corresponding request.	YES	Reject
RAN Function ID	M		10.1.7	Value replicated from corresponding request.	YES	Reject
Cause	M		6.3.3.4		YES	Ignore
Criticality Diagnostics	O		6.3.3.5		YES	Ignore

6.3.2.9 E2 SUBSCRIPTION DELETE QUERY (Required)

This message is sent by the Near-RT RIC platform to the xApp to consult the deletion of a target E2 subscription.

Direction: Near-RT RIC platform -> xApp

IE/Group Name	Presence	Range	IE type and reference	Semantics description	Criticality	Assigned Criticality
Message Type	M		6.3.3.1		YES	Reject
Procedure Transaction ID	M		10.1.1		YES	Reject
Global E2 Node ID	M		10.1.6		YES	Reject
E2 Request ID	M		6.3.3.2		YES	Reject
RAN Function ID	M		10.1.7		YES	Reject
Cause	M		6.3.3.4		YES	Ignore

6.3.2.10 E2 SUBSCRIPTION DELETE QUERY (Accept)

This message is sent by the xApp to the Near-RT RIC platform to agree to deletion of a target E2 subscription.

Direction: xApp -> Near-RT RIC platform

IE/Group Name	Presence	Range	IE type and reference	Semantics description	Criticality	Assigned Criticality
Message Type	M		6.3.3.1		YES	Reject
Procedure Transaction ID	M		10.1.1		YES	Reject
Global E2 Node ID	M		10.1.6		YES	Reject
E2 Request ID	M		6.3.3.2		YES	Reject
RAN Function ID	M		10.1.7		YES	Reject

6.3.2.11 E2 SUBSCRIPTION DELETE QUERY (Decline)

This message is sent by the xApp to the Near-RT RIC platform to decline deletion of a target E2 subscription.

Direction: xApp -> Near-RT RIC platform

IE/Group Name	Presence	Range	IE type and reference	Semantics description	Criticality	Assigned Criticality
Message Type	M		6.3.3.1		YES	Reject
Procedure Transaction ID	M		10.1.1		YES	Reject
Global E2 Node ID	M		10.1.6		YES	Reject
E2 Request ID	M		6.3.3.2		YES	Reject
RAN Function ID	M		10.1.7		YES	Reject
Cause	M		6.3.3.4		YES	Ignore

6.3.2.12 E2 SUBSCRIPTION DELETE NOTIFICATION (Deleted)

This message is sent by the Near-RT RIC platform to the xApp to notify the deletion of a target E2 subscription.

Direction: Near-RT RIC platform -> xApp

IE/Group Name	Presence	Range	IE type and reference	Semantics description	Criticality	Assigned Criticality
Message Type	M		6.3.3.1		YES	Reject
Procedure Transaction ID	M		10.1.1		YES	Reject
Global E2 Node ID	M		10.1.6		YES	Reject
RAN Function ID	M		10.1.7		YES	Reject
E2 Request ID	M		6.3.3.2		YES	Reject
Cause	M		6.3.3.4		YES	Ignore

6.3.2.13 E2 INDICATION (Push)

This message is sent by the Near-RT RIC platform to the xApp for delivery of subscribed information associated with an E2 subscription.

Direction: Near-RT RIC platform -> xApp

IE/Group Name	Presence	Range	IE type and reference	Semantics description	Criticality	Assigned Criticality
Message Type	M		6.3.3.1		YES	Reject
Global E2 Node ID	M		10.1.6		YES	Reject
E2 Request ID	M		6.3.3.2		YES	Reject
RAN Function ID	M		10.1.7		YES	Reject
E2 Action ID	M		RIC Action ID, Clause 9.2.10 of [3].			
E2 Indication SN	O		RIC Indication SN, Clause 9.2.14 of [3].		YES	Reject
E2 Indication Type	M		RIC Indication Type, Clause 9.2.15 of [3].		YES	Reject
E2 Indication Header	M		RIC Indication Header, Clause 9.2.17 of [3].		YES	Reject
E2 Indication Message	M		RIC Indication Message,		YES	Reject

			Clause 9.2.16 of [3].			
E2 Call Process ID	O		RIC Call Process ID, Clause 9.2.18 of [3].		YES	Reject

6.3.2.14 E2 INDICATION (Failure)

This message is sent by the xApp to the Near-RT RIC platform if an error is detected in E2 INDICATION (Push) message.

Direction: xApp -> Near-RT RIC platform

IE/Group Name	Presence	Range	IE type and reference	Semantics description	Criticality	Assigned Criticality
Message Type	M		6.3.3.1		YES	Reject
Global E2 Node ID	M		10.1.6		YES	Reject
E2 Request ID	M		6.3.3.2		YES	Reject
RAN Function ID	M		10.1.7		YES	Reject
E2 Action ID	M		RIC Action ID, Clause 9.2.10 of [3].			
E2 Indication SN	O		RIC Indication SN, Clause 9.2.14 of [3].		YES	Reject
Cause	M		6.3.3.4		YES	Reject
Criticality Diagnostics	O		6.3.3.5		YES	Ignore

6.3.2.15 E2 CONTROL (Request)

This message is sent by the xApp to the Near-RT RIC platform for an E2 control request.

Direction: xApp -> Near-RT RIC platform

IE/Group Name	Presence	Range	IE type and reference	Semantics description	Criticality	Assigned Criticality
Message Type	M		6.3.3.1		YES	Reject
Procedure Transaction ID	M		10.1.1	Value assigned by the initiating xApp.	YES	Reject
Global E2 Node ID	M		10.1.6		YES	Reject
RAN Function ID	M		10.1.7		YES	Reject
E2 Call Process ID	O		RIC Call Process ID, Clause 9.2.18 of [3].		YES	Reject
E2 Control Header	M		RIC Control Header, Clause 9.2.20 of [3].		YES	Reject
E2 Control Message	M		RIC Control Message, Clause 9.2.19 of [3].		YES	Reject
E2 Control Ack Request	O		RIC Control Ack Request, Clause 9.2.21 of [3].		YES	Reject
E2 Control Timer	O		6.3.3.6		YES	Ignore

6.3.2.16 E2 CONTROL (Success)

This message is sent by the Near-RT RIC platform to the xApp for a successfully performed E2 control request.

Direction: Near-RT RIC platform -> xApp

IE/Group Name	Presence	Range	IE type and reference	Semantics description	Criticality	Assigned Criticality
Message Type	M		6.3.3.1		YES	Reject
Procedure Transaction ID	M		10.1.1	Value replicated from corresponding request.	YES	Reject
Global E2 Node ID	M		10.1.6		YES	Reject
RAN Function ID	M		10.1.7		YES	Reject
E2 Call Process ID	O		RIC Call Process ID, Clause 9.2.18 of [3].		YES	Reject
E2 Control Outcome	O		RIC Control Outcome, Clause 9.2.25 of [3].		YES	Reject

6.3.2.17 E2 CONTROL (Reject)

This message is sent by the Near-RT RIC platform to the xApp if the corresponding E2 control request is rejected by the Near-RT RIC platform.

Direction: Near-RT RIC platform -> xApp

IE/Group Name	Presence	Range	IE type and reference	Semantics description	Criticality	Assigned Criticality
Message Type	M		6.3.3.1		YES	Reject
Procedure Transaction ID	M		10.1.1	Value replicated from corresponding request.	YES	Reject
Global E2 Node ID	M		10.1.6		YES	Reject
RAN Function ID	M		10.1.7		YES	Reject
Cause	M		6.3.3.4		YES	Ignore
Criticality Diagnostics	O		6.3.3.5		YES	Ignore

6.3.2.18 E2 CONTROL (Failure)

This message is sent by the Near-RT RIC platform to the xApp if the corresponding E2 control request is accepted by the Near-RT RIC platform but fails in the E2 Node.

Direction: Near-RT RIC platform -> xApp

IE/Group Name	Presence	Range	IE type and reference	Semantics description	Criticality	Assigned Criticality
Message Type	M		6.3.3.1		YES	Reject
Procedure Transaction ID	M		10.1.1	Value replicated from corresponding request.	YES	Reject
Global E2 Node ID	M		10.1.6		YES	Reject
RAN Function ID	M		10.1.7		YES	Reject
E2 Call Process ID	O		RIC Call Process ID, Clause 9.2.18 of [3].		YES	Reject

E2 Control Outcome	O		RIC Control Outcome, Clause 9.2.25 of [3].		YES	Reject
Cause	M		6.3.3.4		YES	Ignore
Criticality Diagnostics	O		6.3.3.5		YES	Ignore

6.3.2.19 E2 GUIDANCE (Request)

6.3.2.20 E2 GUIDANCE (Response)

6.3.2.21 E2 GUIDANCE (Modification)

6.3.3 Information Element Definitions

6.3.3.1 Message Type

This IE uniquely identifies the message being sent. It is mandatory for all messages.

IE/Group Name	Presence	Range	IE Type and Reference	Semantics Description
Message Type				
>Procedure Type	M		CHOICE (E2 Subscription, E2 Subscription Delete, E2 Subscription Delete Query, E2 Subscription Delete Notification, E2 Indication, E2 Control, ...)	
>Type of Message	M		CHOICE (Request, Success, Reject, Failure, ...)	Refer to Table 6.1.2-3 for applicable choices for each procedure type

6.3.3.2 E2 Request ID

This IE is assigned by Near-RT RIC platform to uniquely identify an E2 subscription among all the E2 subscriptions associated with the same E2 Node.

IE/Group Name	Presence	Range	IE type and reference	Semantics description
E2 Request ID	M		INTEGER (0 ... 4294967295)	

6.3.3.3 Void

6.3.3.4 Cause

This IE is used to indicate the reason for a particular event for the E2-related APIs.

Note: Not all causes from E2AP are applicable.

IE/Group Name	Presence	Range	IE type and reference	Semantics description
CHOICE Cause Group	M			
>E2-related API Cause				

>>E2-related API Cause	M		6.3.3.4a.	
>E2AP Cause				
>>E2AP Cause	M		Cause, Clause 9.2.1 of [3].	

6.3.3.4a E2-related API Cause

The purpose of the E2-related API Cause IE is to indicate the reason for a particular event for the E2-related APIs.

IE/Group Name	Presence	Range	IE type and reference	Semantics description
CHOICE E2-related API Cause Group	M			
>Application Layer				
>>Application Layer Cause	M		ENUMERATED (Unspecified, Global E2Node ID invalid, E2 Node Reset or Disconnected, RAN Function ID invalid, Action not supported, Excessive actions, Duplicate action, E2 Request ID unknown, Control conflict, E2 Indication unknown, E2AP Procedure timeout, Essential for operation, ...)	
>Transport Layer				
>>Transport Layer Cause	M		ENUMERATED (Unspecified, Transport Resource Unavailable, ...)	
>Protocol				
>>Protocol Cause	M		ENUMERATED (Unspecified, Transfer Syntax Error, Abstract Syntax Error (Reject), Abstract Syntax Error (Ignore and Notify), Message not Compatible with Receiver State, Semantic Error, Abstract Syntax Error (Falsely Constructed Message), ...)	
>Misc				
>>Miscellaneous Cause	M		ENUMERATED (Unspecified, Control Processing Overload, Hardware Failure, O&M Intervention, ...)	

The meaning of the different cause values is described in the following table. In general, "not supported" cause values indicate that the related capability is missing. On the other hand, "not available" cause values indicate that the related capability is present, but insufficient resources were available to perform the requested action.

Application Layer cause	Meaning
Unspecified	Sent when none of the cause values below applies but still the cause is Application Layer related.
Global E2Node ID invalid	Global E2Node ID invalid.
E2 Node Reset or Disconnected	E2Node Status is Reset or Disconnected.
RAN Function ID invalid	Requested RAN Function ID invalid.
Action not supported	Requested Action not supported by RAN function.
Excessive actions	Excessive number of actions requested for RAN Function.
Duplicate action	Same action requested more than once in same subscription request.
E2 Request ID unknown	E2 Request ID sent to Near-RT RIC is unknown.

Control Conflict	Requested Control command causes conflict.
E2 Indication unknown	The E2 Indication message is unknown, i.e. The received indication message is not subscribed to by the xApp.
E2AP Procedure timeout	An E2AP procedure triggered by the E2-related API procedure is time-out.
Essential for operation	xApp is unwilling to delete an E2 subscription essential for its operation.

Transport Layer cause	Meaning
Unspecified	Sent when none of the cause values below applies but still the cause is Transport Layer related.
Transport Resource Unavailable	The required transport resources are not available.

Protocol cause	Meaning
Unspecified	Sent when none of the below cause values applies but still the cause is Protocol related.
Transfer Syntax Error	The received message included a transfer syntax error.
Abstract Syntax Error (Reject)	The received message included an abstract syntax error and the concerning criticality indicated "reject".
Abstract Syntax Error (Ignore And Notify)	The received message included an abstract syntax error and the concerning criticality indicated "ignore and notify".
Message Not Compatible With Receiver State	The received message was not compatible with the receiver state.
Semantic Error	The received message included a semantic error.
Abstract Syntax Error (Falsely Constructed Message)	The received message contained IEs or IE groups in wrong order or with too many occurrences.

Miscellaneous cause	Meaning
Unspecified Failure	Sent when none of the below cause values applies and the cause is not related to any of the categories Application Layer, Transport Layer, Protocol.
Control Processing Overload	Control processing overload.
Hardware Failure	Action related to hardware failure.
O&M Intervention	The action is due to O&M intervention.

6.3.3.5 Criticality Diagnostics

The Criticality Diagnostics IE is sent by the xApp or the Near-RT RIC Platform when parts of a received message have not been comprehended or were missing. When applicable, it contains information about which IEs were not comprehended or were missing.

IE/Group Name	Presence	Range	IE type and reference	Semantics description
Information Element Criticality Diagnostics		<i>0 .. <maxnofErrors></i>		
>IE Criticality	M		ENUMERATED (reject, ignore, notify)	The IE Criticality is used for reporting the criticality of the triggering IE. The value 'ignore' shall not be used.
>IE ID	M		ENUMERATED (Message Type, Procedure Transaction ID, Global E2 Node ID, RAN Function ID...)	The IE ID of the not understood or missing IE.
>Type of Error	M		ENUMERATED (not understood, missing, ...)	

Range bound	Explanation
-------------	-------------

<i>maxnoofErrors</i>	Maximum no. of IE errors allowed to be reported with a single message. Value is 256.
----------------------	---

6.3.3.6 E2 Control Timer

This IE specifies the suggested time to wait before timeout for the RIC Control procedure in E2AP[3].

IE/Group Name	Presence	Range	IE type and reference	Semantics description
E2 Control Timer	M		INTEGER (10..10000, ...)	Unit in milliseconds.

6.3.3.7 E2 Subscription Timer

This IE specifies the suggested time to wait before timeout for the RIC subscription procedure in E2AP[3].

IE/Group Name	Presence	Range	IE type and reference	Semantics description
E2 Subscription Timer	M		INTEGER (10..10000, ...)	Unit in milliseconds.

6.3.3.8 E2 Subscription Delete Timer

This IE specifies the suggested time to wait before timeout for the RIC subscription deletion procedure in E2AP[3].

IE/Group Name	Presence	Range	IE type and reference	Semantics description
E2 Subscription Delete Timer	M		INTEGER (10..10000, ...)	Unit in milliseconds.

6.3.4 Message and Information Element Abstract Syntax

6.3.4.1 Solution 1: SCTP with Protocol Buffers

6.3.4.1.1 Usage of SCTP with Protocol Buffers

This solution uses Protocol Buffers as message encoding, and SCTP as the transport protocol.

The PDU on the SCTP connection uses the structure as defined in clause 10.1.8 where its API Payload IE serves as a container for individual API messages in clause 6.3.4.1.2 which can be E2SubscriptionApiPayload or E2IndicationApiPayload or E2ControlApiPayload.

Table 6.3.4.1.1-1 lists the E2 related APIs abstract syntax included in this document.

Table 6.3.4.1.1-1: List of E2 related APIs abstract syntax

API Name / Contents	Clause	Version	File Name
E2_Subscription_API	6.3.4.1.2.1	2.0.0	e2_subscription_api_ver_2_0_0.proto
E2_Indication_API	6.3.4.1.2.2	2.0.0	e2_indication_api_ver_2_0_0.proto
E2_Control_API	6.3.4.1.2.3	2.0.0	e2_control_api_ver_2_0_0.proto
E2 related API common information elements	6.3.4.1.3	1.1.0	e2_related_apis_common_ies_ver_1_1_0.proto
Near-RT RIC API common information elements	10.2.1	2.0.0	near_rt_ric_apis_common_ies_ver_2_0_0.proto

6.3.4.1.2 API Definitions

6.3.4.1.2.1 E2_Subscription_API

```

syntax = "proto3";

package ricapi.e2_subscription_api.v2;

import "near_rt_ric_apis_common_ies_ver_2_0_0.proto";
import "e2_related_apis_common_ies_ver_1_1_0.proto";

// top-level PDU
message E2SubscriptionApiPayload {
    oneof procedure_type {
        E2SubscriptionProc e2_subscription_proc = 1;
        E2SubscriptionDeleteProc e2_subscription_delete_proc = 2;
        E2SubscriptionDeleteQueryProc e2_subscription_delete_query_proc = 3;
        E2SubscriptionDeleteNotificationProc e2_subscription_delete_notification_proc = 4;
    }
}

// ----- procedures of E2_Subscription_API -----
message E2SubscriptionProc {
    oneof type_of_message {
        E2SubscriptionRequest e2_subscription_request = 1 ;
        E2SubscriptionSuccess e2_subscription_success = 2 ;
        E2SubscriptionReject e2_subscription_reject = 3 ;
        E2SubscriptionFailure e2_subscription_failure = 4 ;
    }
}

message E2SubscriptionDeleteProc {
    oneof type_of_message {
        E2SubscriptionDeleteRequest e2_subscription_delete_request = 1 ;
        E2SubscriptionDeleteSuccess e2_subscription_delete_success = 2 ;
        E2SubscriptionDeleteReject e2_subscription_delete_reject = 3 ;
        E2SubscriptionDeleteFailure e2_subscription_delete_failure = 4 ;
    }
}

message E2SubscriptionDeleteQueryProc {
    oneof type_of_message {
        E2SubscriptionDeleteQueryRequired e2_subscription_delete_query_required = 1 ;
        E2SubscriptionDeleteQueryAccept e2_subscription_delete_query_accept = 2 ;
        E2SubscriptionDeleteQueryDecline e2_subscription_delete_query_reject = 3 ;
    }
}

message E2SubscriptionDeleteNotificationProc {
    oneof type_of_message {
        E2SubscriptionDeleteNotificationDeleted e2_subscription_delete_notification_deleted = 1 ;
    }
}

// ----- messages of E2_Subscription_API -----
// ----- messages of E2 Subscription procedure -----
message E2SubscriptionRequest {
    optional near_rt_ric_apis_common_ies.v2.ProcedureTransactionId procedure_transaction_id = 1;
    optional near_rt_ric_apis_common_ies.v2.GlobalE2NodeId global_e2_node_id = 2;
    optional near_rt_ric_apis_common_ies.v2.RanFunctionId ran_function_id = 3;
    optional E2SubscriptionDetails e2_subscription_details = 4;
    optional uint32 e2_subscription_timer = 5;
    optional near_rt_ric_apis_common_ies.v2.EndpointInformation e2_indication_destination = 6;
    optional near_rt_ric_apis_common_ies.v2.EndpointInformation
e2_subscription_delete_notification_destination = 7;

    message E2SubscriptionDetails {
        optional e2_related_apis_common_ies.v1.E2EventTriggerDefinition e2_event_trigger_definition = 1;
        repeated E2ActionToBeSetupItem sequence_of_actions = 2 ;

        message E2ActionToBeSetupItem {
            optional e2_related_apis_common_ies.v1.E2ActionId requested_e2_action_id = 1 ;

```

```

        optional e2_related_apis_common_ies.v1.E2ActionType e2_action_type = 2 ;
        optional e2_related_apis_common_ies.v1.E2ActionDefinition e2_action_definition = 3 ;
        optional e2_related_apis_common_ies.v1.E2SubsequentAction e2_subsequent_action = 4 ;
    }
}

message E2SubscriptionSuccess {
    optional near_rt_ric_apis_common_ies.v2.ProcedureTransactionId procedure_transaction_id = 1;
    optional near_rt_ric_apis_common_ies.v2.GlobalE2NodeId global_e2_node_id = 2;
    optional e2_related_apis_common_ies.v1.E2RequestId e2_request_id = 3;
    optional near_rt_ric_apis_common_ies.v2.RanFunctionId ran_function_id = 4;
    repeated E2ActionAdmittedItem e2_actions_admitted_list = 5;
    repeated E2ActionNotAdmittedItem e2_actions_not_admitted_list = 6;

    message E2ActionAdmittedItem {
        optional e2_related_apis_common_ies.v1.E2ActionId requested_e2_action_id = 1 ;
        optional e2_related_apis_common_ies.v1.E2ActionId e2_action_id = 2 ;
    }

    message E2ActionNotAdmittedItem {
        optional e2_related_apis_common_ies.v1.E2ActionId requested_e2_action_id = 1 ;
        optional e2_related_apis_common_ies.v1.Cause cause = 2 ;
    }
}

message E2SubscriptionFailure {
    optional near_rt_ric_apis_common_ies.v2.ProcedureTransactionId procedure_transaction_id = 1;
    optional near_rt_ric_apis_common_ies.v2.GlobalE2NodeId global_e2_node_id = 2;
    optional near_rt_ric_apis_common_ies.v2.RanFunctionId ran_function_id = 3;
    optional e2_related_apis_common_ies.v1.Cause cause = 4;
    optional e2_related_apis_common_ies.v1.CriticalityDiagnostics criticality_diagnostics = 5;
}

message E2SubscriptionReject {
    optional near_rt_ric_apis_common_ies.v2.ProcedureTransactionId procedure_transaction_id = 1;
    optional near_rt_ric_apis_common_ies.v2.GlobalE2NodeId global_e2_node_id = 2;
    optional near_rt_ric_apis_common_ies.v2.RanFunctionId ran_function_id = 3;
    optional e2_related_apis_common_ies.v1.Cause cause = 4;
    optional e2_related_apis_common_ies.v1.CriticalityDiagnostics criticality_diagnostics = 5;
}

// ----- messages of E2 Subscription Delete procedure -----
message E2SubscriptionDeleteRequest {
    optional near_rt_ric_apis_common_ies.v2.ProcedureTransactionId procedure_transaction_id = 1;
    optional near_rt_ric_apis_common_ies.v2.GlobalE2NodeId global_e2_node_id = 2;
    optional e2_related_apis_common_ies.v1.E2RequestId e2_request_id = 3;
    optional near_rt_ric_apis_common_ies.v2.RanFunctionId ran_function_id = 4;
    optional uint32 e2_subscription_delete_timer = 5;
}

message E2SubscriptionDeleteSuccess {
    optional near_rt_ric_apis_common_ies.v2.ProcedureTransactionId procedure_transaction_id = 1;
    optional near_rt_ric_apis_common_ies.v2.GlobalE2NodeId global_e2_node_id = 2;
    optional e2_related_apis_common_ies.v1.E2RequestId e2_request_id = 3;
    optional near_rt_ric_apis_common_ies.v2.RanFunctionId ran_function_id = 4;
}

message E2SubscriptionDeleteFailure {
    optional near_rt_ric_apis_common_ies.v2.ProcedureTransactionId procedure_transaction_id = 1;
    optional near_rt_ric_apis_common_ies.v2.GlobalE2NodeId global_e2_node_id = 2;
    optional e2_related_apis_common_ies.v1.E2RequestId e2_request_id = 3;
    optional near_rt_ric_apis_common_ies.v2.RanFunctionId ran_function_id = 4;
    optional e2_related_apis_common_ies.v1.Cause cause = 5;
    optional e2_related_apis_common_ies.v1.CriticalityDiagnostics criticality_diagnostics = 6;
}

message E2SubscriptionDeleteReject {
    optional near_rt_ric_apis_common_ies.v2.ProcedureTransactionId procedure_transaction_id = 1;
    optional near_rt_ric_apis_common_ies.v2.GlobalE2NodeId global_e2_node_id = 2;
    optional e2_related_apis_common_ies.v1.E2RequestId e2_request_id = 3;
    optional near_rt_ric_apis_common_ies.v2.RanFunctionId ran_function_id = 4;
}

```

```

    optional e2_related_apis_common_ies.v1.Cause cause = 5;
    optional e2_related_apis_common_ies.v1.CriticalityDiagnostics criticality_diagnostics = 6;
}

// ----- messages of E2 Subscription Delete Query procedure -----
message E2SubscriptionDeleteQueryRequired {
    optional near_rt_ric_apis_common_ies.v2.ProcedureTransactionId procedure_transaction_id = 1;
    optional near_rt_ric_apis_common_ies.v2.GlobalE2NodeId global_e2_node_id = 2;
    optional e2_related_apis_common_ies.v1.E2RequestId e2_request_id = 3;
    optional near_rt_ric_apis_common_ies.v2.RanFunctionId ran_function_id = 4;
    optional e2_related_apis_common_ies.v1.Cause cause = 5;
}

message E2SubscriptionDeleteQueryAccept {
    optional near_rt_ric_apis_common_ies.v2.ProcedureTransactionId procedure_transaction_id = 1;
    optional near_rt_ric_apis_common_ies.v2.GlobalE2NodeId global_e2_node_id = 2;
    optional e2_related_apis_common_ies.v1.E2RequestId e2_request_id = 3;
    optional near_rt_ric_apis_common_ies.v2.RanFunctionId ran_function_id = 4;
}

message E2SubscriptionDeleteQueryDecline {
    optional near_rt_ric_apis_common_ies.v2.ProcedureTransactionId procedure_transaction_id = 1;
    optional near_rt_ric_apis_common_ies.v2.GlobalE2NodeId global_e2_node_id = 2;
    optional e2_related_apis_common_ies.v1.E2RequestId e2_request_id = 3;
    optional near_rt_ric_apis_common_ies.v2.RanFunctionId ran_function_id = 4;
    optional e2_related_apis_common_ies.v1.Cause cause = 5;
}

// ----- messages of E2 Subscription Delete Notification procedure -----
message E2SubscriptionDeleteNotificationDeleted {
    optional near_rt_ric_apis_common_ies.v2.ProcedureTransactionId procedure_transaction_id = 1;
    optional near_rt_ric_apis_common_ies.v2.GlobalE2NodeId global_e2_node_id = 2;
    optional e2_related_apis_common_ies.v1.E2RequestId e2_request_id = 3;
    optional near_rt_ric_apis_common_ies.v2.RanFunctionId ran_function_id = 4;
    optional e2_related_apis_common_ies.v1.Cause cause = 5;
}

```

6.3.4.1.2.2 E2_Indication_API

```

syntax = "proto3";

package ricapi.e2_indication_api.v2;

import "near_rt_ric_apis_common_ies_ver_2_0_0.proto";
import "e2_related_apis_common_ies_ver_1_1_0.proto";

// top-level PDU
message E2IndicationApiPayload {
    oneof procedure_type {
        E2IndicationProc e2_indication_proc = 1;
    }
}

// ----- procedures of E2_Subscription_API -----
message E2IndicationProc {
    oneof type_of_message {
        E2IndicationPush e2_indication_push = 1 ;
        E2IndicationFailure e2_indication_failure = 2 ;
    }
}

// ----- messages of E2_Indication_API -----
// ----- messages of E2 Indication procedure -----
message E2IndicationPush {
    optional near_rt_ric_apis_common_ies.v2.GlobalE2NodeId global_e2_node_id = 1;
    optional e2_related_apis_common_ies.v1.E2RequestId e2_request_id = 2;
    optional near_rt_ric_apis_common_ies.v2.RanFunctionId ran_function_id = 3;
    optional e2_related_apis_common_ies.v1.E2ActionId e2_action_id = 4;
    optional e2_related_apis_common_ies.v1.E2IndicationSN e2_indication_sn = 5;
    optional e2_related_apis_common_ies.v1.E2IndicationType e2_indication_type = 6;
    optional e2_related_apis_common_ies.v1.E2IndicationHeader e2_indication_header = 7;
    optional e2_related_apis_common_ies.v1.E2IndicationMessage e2_indication_message = 8;
}

```

```

    optional e2_related_apis_common_ies.v1.E2CallProcessId e2_call_process_id = 9;
}

message E2IndicationFailure {
    optional near_rt_ric_apis_common_ies.v2.GlobalE2NodeId global_e2_node_id = 1;
    optional e2_related_apis_common_ies.v1.E2RequestId e2_request_id = 2;
    optional near_rt_ric_apis_common_ies.v2.RanFunctionId ran_function_id = 3;
    optional e2_related_apis_common_ies.v1.E2ActionId e2_action_id = 4;
    optional e2_related_apis_common_ies.v1.E2IndicationSN e2_indication_sn = 5;
    optional e2_related_apis_common_ies.v1.Cause cause = 6;
    optional e2_related_apis_common_ies.v1.CriticalityDiagnostics criticality_diagnostics = 7;
}

```

6.3.4.1.2.3 E2_Control_API

```

syntax = "proto3";

package ricapi.e2_control_api.v2;

import "near_rt_ric_apis_common_ies_ver_2_0_0.proto";
import "e2_related_apis_common_ies_ver_1_1_0.proto";

// top-level PDU
message E2ControlApiPayload {
    oneof procedure_type {
        E2ControlProc e2_control_proc = 1;
    }
}

// ----- procedures of E2_Control_API -----
message E2ControlProc {
    oneof type_of_message {
        E2ControlRequest e2_control_request = 1;
        E2ControlSuccess e2_control_success = 2;
        E2ControlReject e2_control_reject = 3;
        E2ControlFailure e2_control_failure = 4;
    }
}

// ----- messages of E2_Control_API -----
// ----- messages of E2 Control procedure -----
message E2ControlRequest {
    optional near_rt_ric_apis_common_ies.v2.ProcedureTransactionId procedure_transaction_id = 1;
    optional near_rt_ric_apis_common_ies.v2.GlobalE2NodeId global_e2_node_id = 2;
    optional near_rt_ric_apis_common_ies.v2.RanFunctionId ran_function_id = 3;
    optional e2_related_apis_common_ies.v1.E2CallProcessId e2_call_process_id = 4;
    optional e2_related_apis_common_ies.v1.E2ControlHeader e2_control_header = 5;
    optional e2_related_apis_common_ies.v1.E2ControlMessage e2_control_message = 6;
    optional e2_related_apis_common_ies.v1.E2ControlAckRequest e2_control_ack_request = 7;
    optional uint32 e2_control_timer = 8;
}

message E2ControlSuccess {
    optional near_rt_ric_apis_common_ies.v2.ProcedureTransactionId procedure_transaction_id = 1;
    optional near_rt_ric_apis_common_ies.v2.GlobalE2NodeId global_e2_node_id = 2;
    optional near_rt_ric_apis_common_ies.v2.RanFunctionId ran_function_id = 3;
    optional e2_related_apis_common_ies.v1.E2CallProcessId e2_call_process_id = 4;
    optional e2_related_apis_common_ies.v1.E2ControlOutcome e2_control_outcome = 5;
}

message E2ControlReject {
    optional near_rt_ric_apis_common_ies.v2.ProcedureTransactionId procedure_transaction_id = 1;
    optional near_rt_ric_apis_common_ies.v2.GlobalE2NodeId global_e2_node_id = 2;
    optional near_rt_ric_apis_common_ies.v2.RanFunctionId ran_function_id = 3;
    optional e2_related_apis_common_ies.v1.Cause cause = 4;
    optional e2_related_apis_common_ies.v1.CriticalityDiagnostics criticality_diagnostics = 5;
}

message E2ControlFailure {
    optional near_rt_ric_apis_common_ies.v2.ProcedureTransactionId procedure_transaction_id = 1;
    optional near_rt_ric_apis_common_ies.v2.GlobalE2NodeId global_e2_node_id = 2;
}

```

```

optional near_rt_ric_apis_common_ies.v2.RanFunctionId ran_function_id = 3;
optional e2_related_apis_common_ies.v1.E2CallProcessId e2_call_process_id = 4;
optional e2_related_apis_common_ies.v1.E2ControlOutcome e2_control_outcome = 5;
optional e2_related_apis_common_ies.v1.Cause cause = 6;
optional e2_related_apis_common_ies.v1.CriticalityDiagnostics criticality_diagnostics = 7;
}

```

6.3.4.1.3 Information Element Definitions

```

syntax = "proto3";

package ricapi.e2_related_apis_common_ies.v1;

import "near_rt_ric_apis_common_ies_ver_2_0_0.proto";

// ----- Information Elements -----
// -----E-----
// From E2AP Clause 9.2.11
enum E2ActionType {
    E2ACTION_TYPE_REPORT = 0;
    E2ACTION_TYPE_INSERT = 1;
    E2ACTION_TYPE_POLICY = 2;
};

// From E2AP Clause 9.2.21
enum E2ControlAckRequest {
    E2_CONTROL_ACK_REQUEST_NO_ACK = 0;
    E2_CONTROL_ACK_REQUEST_ACK = 1;
}

// From E2AP Clause 9.2.15
enum E2IndicationType {
    E2_INDICATION_TYPE_REPORT = 0;
    E2_INDICATION_TYPE_INSERT = 1;
}

// -----C-----
message Cause {
    oneof cause_group {
        E2RelatedApiCause e2_related_api_cause = 1;
        E2apCause e2ap_cause = 2;
    }

    message E2apCause {
        oneof cause_group {
            RicRequest ric_request = 1;
            RicService ric_service = 2;
            E2Node e2_node = 3;
            Transport transport = 4;
            Protocol protocol = 5;
            Misc misc = 6;
        }

        enum E2Node {
            E2NODE_COMPONENT_UNKNOWN = 0;
        }

        enum Misc {
            CONTROL_PROCESSING_OVERLOAD = 0;
            HARDWARE_FAILURE = 1;
            OM_INTERVENTION = 2;
            MISC_UNSPECIFIED = 3;
        }

        enum Protocol {
            TRANSFER_SYNTAX_ERROR = 0;
            ABSTRACT_SYNTAX_ERROR_REJECT = 1;
            ABSTRACT_SYNTAX_ERROR_IGNORE_AND_NOTIFY = 2;
            MESSAGE_NOT_COMPATIBLE_WITH_RECEIVER_STATE = 3;
            SEMANTIC_ERROR = 4;
            ABSTRACT_SYNTAX_ERROR_FALSEELY_CONSTRUCTED_MESSAGE = 5;
        }
    }
}

```



```

        PROTOCOL_UNSPECIFIED = 6;
    }

    enum RicRequest {
        RAN_FUNCTION_ID_INVALID = 0;
        ACTION_NOT_SUPPORTED = 1;
        EXCESSIVE_ACTIONS = 2;
        DUPLICATE_ACTION = 3;
        DUPLICATE_EVENT_TRIGGER = 4;
        FUNCTION_RESOURCE_LIMIT = 5;
        REQUEST_ID_UNKNOWN = 6;
        INCONSISTENT_ACTION_SUBSEQUENT_ACTION_SEQUENCE = 7;
        CONTROL_MESSAGE_INVALID = 8;
        RIC_CALL_PROCESS_ID_INVALID = 9;
        CONTROL_TIMER_EXPIRED = 10;
        CONTROL_FAILED_TO_EXECUTE = 11;
        SYSTEM_NOT_READY = 12;
        RIC_REQUEST_UNSPECIFIED = 13;
    }

    enum RicService {
        RAN_FUNCTION_NOT_SUPPORTED = 0;
        EXCESSIVE_FUNCTIONS = 1;
        RIC_RESOURCE_LIMIT = 2;
    }

    enum Transport {
        TRANSPORT_UNSPECIFIED = 0;
        TRANSPORT_RESOURCE_UNAVAILABLE = 1;
    }
}

message E2RelatedApiCause {
    oneof cause_group {
        ApplicationLayer application_layer = 1;
        Transport transport = 2;
        Protocol protocol = 3;
        Misc misc = 4;
    }

    enum ApplicationLayer {
        APPLICATION_LAYER_UNSPECIFIED = 0;
        GLOBAL_E2_NODE_ID_INVALID = 1;
        E2_NODE_RESET_OR_DISCONNECTED = 2;
        RAN_FUNCTION_ID_INVALID = 3;
        ACTION_NOT_SUPPORTED = 4;
        EXCESSIVE_ACTIONS = 5;
        DUPLICATE_ACTION = 6;
        E2_REQUEST_ID_UNKNOWN = 7;
        CONTROL_CONFLICT = 8;
        E2_INDICATION_UNKNOWN = 9;
        E2AP_PROCEDURE_TIMEOUT = 10;
        ESSENTIAL_FOR_OPERATION = 11;
    }

    enum Misc {
        MISC_UNSPECIFIED = 0;
        CONTROL_PROCESSING_OVERLOAD = 1;
        HARDWARE_FAILURE = 2;
        OM_INTERVENTION = 3;
    }

    enum Protocol {
        PROTOCOL_UNSPECIFIED = 0;
        TRANSFER_SYNTAX_ERROR = 1;
        ABSTRACT_SYNTAX_ERROR_REJECT = 2;
        ABSTRACT_SYNTAX_ERROR_IGNORE_AND_NOTIFY = 3;
        MESSAGE_NOT_COMPATIBLE_WITH_RECEIVER_STATE = 4;
        SEMANTIC_ERROR = 5;
        ABSTRACT_SYNTAX_ERROR_FALSEELY_CONSTRUCTED_MESSAGE = 6;
    }

    enum Transport {
        TRANSPORT_UNSPECIFIED = 0;
    }
}

```

```

        TRANSPORT_RESOURCE_UNAVAILABLE = 1;
    }
}

message CriticalityDiagnostics {
    repeated IECriticalityDiagnosticsItem ie_criticality_diagnostics_list = 1;

    message IECriticalityDiagnosticsItem {
        optional IECriticality ie_criticality = 1;
        optional IEId ie_id = 2;
        optional TypeOfError type_of_error = 3;
    }

    enum IECriticality {
        IE_CRITICALITY_REJECT = 0;
        IE_CRITICALITY_IGNORE = 1;
        IE_CRITICALITY_NOTIFY = 2;
    }

    enum IEId {
        IE_ID_CAUSE = 0;
        IE_ID_CRITICALITY_DIAGNOSTICS = 1;
        IE_ID_E2_ACTION_ID = 2;
        IE_ID_E2_ACTIONS_ADMITTED_LIST = 3;
        IE_ID_E2_ACTIONS_NOT_ADMITTED_LIST = 4;
        IE_ID_E2_CALL_PROCESS_ID = 5;
        IE_ID_E2_CONTROL_ACK_REQUEST = 6;
        IE_ID_E2_CONTROL_HEADER = 7;
        IE_ID_E2_CONTROL_MESSAGE = 8;
        IE_ID_E2_CONTROL_OUTCOME = 9;
        IE_ID_E2_CONTROL_TIMER = 10;
        IE_ID_E2_INDICATION_HEADER = 11;
        IE_ID_E2_INDICATION_MESSAGE = 12;
        IE_ID_E2_INDICATION_SN = 13;
        IE_ID_E2_INDICATION_TYPE = 14;
        IE_ID_E2_REQUEST_ID = 15;
        IE_ID_E2_SUBSCRIPTION_DETAILS = 16;
        IE_ID_GLOBAL_E2_NODE_ID = 17;
        IE_ID_PROCEDURE_TRANSACTION_ID = 18;
        IE_ID_RAN_FUNCTION_ID = 19;
        IE_ID_SEQUENCE_OF_ACTIONS = 20;
        IE_ID_E2_SUBSCRIPTION_TIMER = 21;
        IE_ID_E2_SUBSCRIPTION_DELETE_TIMER = 22;
    }

    enum TypeOfError {
        TYPE_OF_ERROR_NOT_UNDERSTOOD = 0;
        TYPE_OF_ERROR_MISSING = 1;
    }
}

// -----E-----
message E2ActionDefinition {
    optional bytes value = 1;
};

message E2ActionId {
    optional uint32 value = 1;
};

message E2CallProcessId {
    optional bytes value = 1;
};

message E2ControlHeader {
    optional bytes value = 1;
};

message E2ControlMessage {
    optional bytes value = 1;
};

```

```

message E2ControlOutcome {
    optional bytes value = 1;
};

message E2EventTriggerDefinition {
    optional bytes value = 1;
};

message E2IndicationHeader {
    optional bytes value = 1;
};

message E2IndicationMessage {
    optional bytes value = 1;
};

message E2IndicationSN {
    optional uint32 value = 1;
};

message E2RequestId {
    optional uint32 value = 1;
};

// From E2AP Clause 9.2.13
message E2SubsequentAction {
    optional SubsequentActionType subsequent_action_type = 1;
    optional TimeToWait e2_time_to_wait = 2;

    enum SubsequentActionType {
        SUBSEQUENT_ACTION_TYPE_CONTINUE = 0;
        SUBSEQUENT_ACTION_TYPE_WAIT = 1;
    };

    enum TimeToWait {
        TIME_TO_WAIT_W1MS = 0;
        TIME_TO_WAIT_W2MS = 1;
        TIME_TO_WAIT_W5MS = 2;
        TIME_TO_WAIT_W10MS = 3;
        TIME_TO_WAIT_W20MS = 4;
        TIME_TO_WAIT_W30MS = 5;
        TIME_TO_WAIT_W40MS = 6;
        TIME_TO_WAIT_W50MS = 7;
        TIME_TO_WAIT_W100MS = 8;
        TIME_TO_WAIT_W200MS = 9;
        TIME_TO_WAIT_W500MS = 10;
        TIME_TO_WAIT_W1S = 11;
        TIME_TO_WAIT_W2S = 12;
        TIME_TO_WAIT_W5S = 13;
        TIME_TO_WAIT_W10S = 14;
        TIME_TO_WAIT_W20S = 15;
        TIME_TO_WAIT_W60S = 16;
    }
};

```

6.3.4.2 Solution 2: gRPC with Protocol Buffers

6.3.4.2.1 Usage of gRPC with Protocol Buffers

This solution uses Protocol Buffers as message encoding, and also as the Interface Definition Language (IDL) for gRPC [5].

Table 6.3.4.2.1-1 lists the mapping of enablement APIs and corresponding gRPC services.

Table 6.3.4.2.1-1: Mapping of E2 related APIs and gRPC services

API Name	gRPC service(s)
E2_Subscription_API	E2_Subscription_API, E2_Subscription_API_Callback
E2_Indication_API	E2_Indication_API
E2_Control_API	E2_Control_API

A procedure of an E2 related API is mapped to a *gRPC method*.

Table 6.3.4.2.1-1 lists the E2 related APIs abstract syntax included in this document.

Table 6.3.4.2.1-1: List of E2 related APIs abstract syntax

API Name / Contents	Clause	Version	File Name
E2_Subscription_API	6.3.4.2.2.1	1.0.0	e2_subscription_api_grpc_ver_1_0_0.proto
E2_Indication_API	6.3.4.2.2.2	1.0.0	e2_indication_api_grpc_ver_1_0_0.proto
E2_Control_API	6.3.4.2.2.3	1.0.0	e2_control_api_ver_grpc_1_0_0.proto
E2 related API common information elements	6.3.4.1.3	1.1.0	e2_related_apis_common_ies_ver_1_1_0.proto
Near-RT RIC API common information elements	10.2.1	2.0.0	near_rt_ric_apis_common_ies_ver_2_0_0.proto

6.3.4.2.2 API Definitions

6.3.4.2.2.1 E2_Subscription_API

```
syntax = "proto3";

package ricapi.e2_subscription_api_grpc.v1;

import "e2_subscription_api_ver_2_0_0.proto";

// gRPC service and rpc definition
service E2_Subscription_API {
    rpc E2SubscriptionProc(E2SubscriptionInitMsg) returns (E2SubscriptionOutMsg) {};
    rpc E2SubscriptionDeleteProc(E2SubscriptionDeleteInitMsg) returns (E2SubscriptionDeleteOutMsg) {};
}

message E2SubscriptionInitMsg {
    optional ricapi.e2_subscription_api.v2.E2SubscriptionRequest e2_subscription_request = 1;
}

message E2SubscriptionOutMsg {
    oneof type_of_message {
        ricapi.e2_subscription_api.v2.E2SubscriptionSuccess e2_subscription_success = 1;
        ricapi.e2_subscription_api.v2.E2SubscriptionReject e2_subscription_reject = 2;
        ricapi.e2_subscription_api.v2.E2SubscriptionFailure e2_subscription_failure = 3;
    }
}

message E2SubscriptionDeleteInitMsg {
    optional ricapi.e2_subscription_api.v2.E2SubscriptionDeleteRequest e2_subscription_delete_request = 1;
}

message E2SubscriptionDeleteOutMsg {
    oneof type_of_message {
        ricapi.e2_subscription_api.v2.E2SubscriptionDeleteSuccess e2_subscription_delete_success = 1;
        ricapi.e2_subscription_api.v2.E2SubscriptionDeleteReject e2_subscription_delete_reject = 2;
        ricapi.e2_subscription_api.v2.E2SubscriptionDeleteFailure e2_subscription_delete_failure = 3;
    }
}

service E2_Subscription_API_Callback {
    rpc E2SubscriptionDeleteQueryProc(E2SubscriptionDeleteQueryInitMsg) returns
(E2SubscriptionDeleteQueryOutMsg) {};
    rpc E2SubscriptionDeleteNotificationProc(E2SubscriptionDeleteNotificationInitMsg) returns
(E2SubscriptionDeleteNotificationOutMsg) {};
}

message E2SubscriptionDeleteQueryInitMsg {
```

```

    optional ricapi.e2_subscription_api.v2.E2SubscriptionDeleteQueryRequired
e2_subscription_delete_query_required = 1;
}

message E2SubscriptionDeleteQueryOutMsg {
    oneof type_of_message {
        ricapi.e2_subscription_api.v2.E2SubscriptionDeleteQueryAccept e2_subscription_delete_query_accept
= 1;
        ricapi.e2_subscription_api.v2.E2SubscriptionDeleteQueryDecline e2_subscription_delete_query_reject
= 2;
    }
}

message E2SubscriptionDeleteNotificationInitMsg {
    optional ricapi.e2_subscription_api.v2.E2SubscriptionDeleteNotificationDeleted
e2_subscription_delete_notification_deleted = 1;
}

message E2SubscriptionDeleteNotificationOutMsg {
}

```

6.3.4.2.2.2 E2_Indication_API

```

syntax = "proto3";

package ricapi.e2_indication_api_grpc.v1;

import "e2_indication_api_ver_2_0_0.proto";

// gRPC service and rpc definition
service E2_Indication_API {
    rpc E2IndicationProc(E2IndicationInitMsg) returns (E2IndicationOutMsg) {};
}

message E2IndicationInitMsg {
    optional ricapi.e2_indication_api.v2.E2IndicationPush e2_indication_push = 1;
}

message E2IndicationOutMsg {
    oneof type_of_message {
        EmptyMessage empty_message = 1;
        ricapi.e2_indication_api.v2.E2IndicationFailure e2_indication_failure = 2;
    }

    message EmptyMessage {
    }
}

```

6.3.4.2.2.3 E2_Control_API

```

syntax = "proto3";

package ricapi.e2_control_api.v2;

import "e2_control_api_ver_2_0_0.proto";

// gRPC service and rpc definition
service E2_Control_API {
    rpc E2ControlProc(E2ControlInitMsg) returns (E2ControlOutMsg) {};
}

message E2ControlInitMsg {
    optional ricapi.e2_control_api.v2.E2ControlRequest e2_control_request = 1;
}

message E2ControlOutMsg {
    oneof type_of_message {
        ricapi.e2_control_api.v2.E2ControlSuccess e2_control_success = 1;
        ricapi.e2_control_api.v2.E2ControlReject e2_control_reject = 2;
        ricapi.e2_control_api.v2.E2ControlFailure e2_control_failure = 3;
    }
}

```

6.3.5 Message Transfer Syntax

6.3.5.1 Solution 1: SCTP with Protocol Buffers

The message transfer syntax for Solution 1 is serialized Protocol Buffers [4].

6.3.5.2 Solution 2: gRPC with Protocol Buffers

The message transfer syntax for Solution 2 is serialized Protocol Buffers [4].

7 SDL API

7.1 Overview

7.1.1 Protocol Stack

In this version of specification, the default and optional protocol stacks for the SDL APIs are shown on Figure 7.1.1-1.

Note: The security protocols are specified in [6].

	Application			Application	
	SCTP			gRPC	
	IP			HTTP/2	
	Data Link Layer			TCP	
	Physical Layer			IP	
Default protocol stack : SCTP			Optional protocol stack : gRPC		

Figure 7.1.1-1: Protocol stack for SDL APIs

The SDL APIs may use the SCTP or gRPC with Protocol Buffers [4] as the application layer serialization protocol. The supported protocol by the platform API is presented to the xApps during the xApp Discovery Procedure in clause 9.2.1.

7.1.2 List of SDL APIs, procedures and messages

The SDL API procedures introduced in clause 9.5 of [2] are categorized under the SDL APIs. These APIs include:

- SDL Information API: enabling subscription and retrieval of information by xApps.
- xApp Data Sharing API: enabling xApp to share specific data with Near-RT RIC platform and/or other data consumer xApps.

These APIs are summarized in Table 7.1.2-1.

Table 7.1.2-1: List of SDL APIs

API Name	Description
SDL_Information_API	Near-RT RIC platform service for SDL information operation
Xapp_Data_Sharing_API	xApp service to share SDL data produced by the xApp

The table 7.1.2-2 lists the API procedures for each SDL API.

Table 7.1.2-2: List of SDL API procedures

API Name	API procedure	Communication type
SDL_Information_API	SDL Information Subscribe	Request/Response
	SDL Information Push	Notify
	SDL Information Update Notify	Notify
	SDL Information Fetch	Request/Response
Xapp_Data_Sharing_API	Xapp Shared Data Fetch	Request/Response
	Xapp Shared Data Subscrbe	Request/Response
	Xapp Shared Data Subscription Delete	Request/Response
	Xapp Shared Data Push	Notify

The table 7.1.2-3 lists the messages for each SDL API procedure.

Table 7.1.2-3: List of SDL API messages

API Procedure	Initiated by	Initiating Message	Outcome Message	
			Successful Outcome	Unsuccessful Outcome
SDL Information Subscribe	xApp	SUBSCRIBE INFORMATION REQUEST	SUBSCRIBE INFORMATION RESPONSE	SUBSCRIBE INFORMATION FAILURE
SDL Information Push	Near-RT RIC platform	INFORMATION PUSH	-	-
SDL Information Update Notify	Near-RT RIC platform	INFORMATION UPDATE NOTIFY	-	-
SDL Information Fetch	xApp	INFORMATION FETCH REQUEST	INFORMATION FETCH RESPONSE	INFORMATION FETCH FAILURE
Xapp Shared Data Fetch	Near-RT RIC platform or data consumer xApp	XAPP SHARED DATA FETCH REQUEST	XAPP SHARED DATA FETCH RESPONSE	XAPP SHARED DATA FETCH FAILURE
Xapp Shared Data Subscribe	Near-RT RIC platform or data consumer xApp	XAPP SHARED DATA SUBSCRIPTION REQUEST	XAPP SHARED DATA SUBSCRIPTION RESPONSE	XAPP SHARED DATA SUBSCRIPTION FAILURE
Xapp Shared Data Subscription Delete	Near-RT RIC platform or data consumer xApp	XAPP SHARED DATA SUBSCRIPTION DELETE REQUEST	XAPP SHARED DATA SUBSCRIPTION DELETE RESPONSE	XAPP SHARED DATA SUBSCRIPTION DELETE FAILURE
Xapp Shared Data Push	xApp	XAPP SHARED DATA PUSH	-	-

The table 7.1.2-4 lists the supported data categories for each SDL API in this release.

Table 7.1.2-4: Supported Data Categories

API Name	Supported Data Categories
SDL_Information_API	E2 Node Information
Xapp_Data_Sharing_API	RAN Analytics Information

7.2 SDL API Procedures

7.2.1 SDL Client Registration procedure

7.2.2 SDL Client Deregistration procedure

7.2.3 SDL Subscribe Information procedure

7.2.3.1 General

This procedure is used by the xApp to subscribe to the SDL for the information that it is interested to obtain.

7.2.3.2 Successful Operation

```

@startuml
Skin rose
skinparam ParticipantPadding 50
skinparam BoxPadding 10
skinparam lifelineStrategy solid
participant "Near-RT RIC Platform" as near
participant "xApp" as app
app -> near: SUBSCRIBE INFORMATION REQUEST
near -> app: SUBSCRIBE INFORMATION RESPONSE
@enduml

```


Figure 7.2.3.2-1: Subscribe Information procedure, successful operation

The xApp initiates the procedure by sending the SUBSCRIBE INFORMATION REQUEST message.

The message shall include:

- the *SDL Data Type ID* IE, which indicates the requested SDL data type;
- the *Delivery Method* IE, which indicates the method of data delivery;
- the *Notification Destination* IE, which specifies the endpoint to receive associated notification(s);
- the *Notification Criteria* IE, which specifies the timing related to the notification(s). It shall include:
 - the *Notification Method* IE, which indicates the notification(s) are periodic or event triggered;

- the *Notification Period* IE if the *Notification Method* IE is set to "PERIODIC", which indicates the time interval between periodic notifications;
- the *Trigger Event Definition* IE if the *Notification Method* IE is set to "EVENT_TRIGGERED", which contains the description of the triggering event;

The message may include:

- the *SDL Data Description* IE, which contains the detailed descriptions for the requested data.

When the Near-RT RIC platform receives the request, it searches the local database or data sources for the requested information.

If the Near-RT RIC platform successfully processes the request, it shall prepare necessary resource for the subscription, assign a subscription ID, and responds to the xApp with the SUBSCRIBE INFORMATION RESPONSE message.

7.2.3.3 Unsuccessful Operation

```
@startuml
Skin rose
skinparam ParticipantPadding 50
skinparam BoxPadding 10
skinparam lifelineStrategy solid
participant "Near-RT RIC Platform" as near
participant "xApp" as app
app -> near: SUBSCRIBE INFORMATION REQUEST
near -> app: SUBSCRIBE INFORMATION FAILURE
@enduml
```



Figure 7.2.3.3-1: Subscribe Information procedure, unsuccessful operation

If the Near-RT RIC platform can not accept the request it shall respond with the SUBSCRIBE INFORMATION FAILURE message and the proper cause. If the message contains *Time To Wait* IE the xApp shall wait at least the indicated time before reinitiating the request to the Near-RT RIC platform.

7.2.3.4 Abnormal Conditions

If the Near-RT RIC platform receives a SUBSCRIBE INFORMATION REQUEST message containing *Information Type* IE that is not supported by the Near-RT RIC platform, or the optional filter definition is invalid, the Near-RT RIC platform shall send the SUBSCRIBE INFORMATION FAILURE message to the xApp with an appropriate cause value.

7.2.4 SDL Information Push procedure

7.2.4.1 General

This procedure is used by the Near-RT RIC platform to send the latest information from the Near-RT RIC platform to the xApp based on what the xApp has subscribed when the *Delivery Method* IE set to "PUSH_DELTA" or "PUSH_FULL" in the subscribe information request.

7.2.4.2 Successful Operation

```
@startuml
Skin rose
skinparam ParticipantPadding 50
```

```
skinparam BoxPadding 10
skinparam lifelineStrategy solid
participant "Near-RT RIC Platform" as near
participant "xApp" as app
near -> app: INFORMATION PUSH
@enduml
```

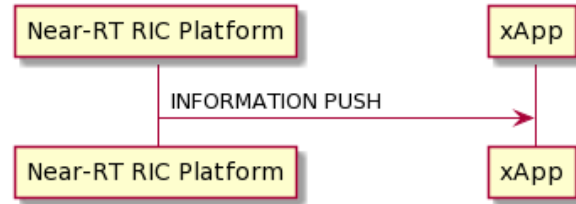


Figure 7.2.4.2-1: Information Push procedure, successful operation

The Near-RT RIC initiates the procedure by sending the INFORMATION PUSH message.

The message shall include:

- the *Subscription ID* IE, which indicates the associated subscription;
- the *SDL Data Type ID* IE, which indicates the SDL data Type carried in the message;
- the *SDL Data Block* IE, which carries the SDL data and shall include:
 - the *SDL Data Unit* IE, which contains a single item of the SDL data;
 - the *Modification Type* IE, which indicates type of change (added, modified, removed, etc.) associated with the SDL data in the *SDL Data Unit* IE.

7.2.4.3 Unsuccessful Operation

Not applicable.

7.2.4.4 Abnormal Conditions

Not applicable.

7.2.5 SDL Information Update Notify procedure

7.2.5.1 General

This procedure is used by the Near-RT RIC platform to notify the xApp that certain information subscribed by the xApp has been updated and available to be fetched. The procedure is used when the Delivery Method IE set to "UPDATE_NOTIFY" in the subscribe information request.

7.2.5.2 Successful Operation

```
@startuml
Skin rose
skinparam ParticipantPadding 50
skinparam BoxPadding 10
skinparam lifelineStrategy solid
participant "Near-RT RIC Platform" as near
participant "xApp" as app
near -> app: INFORMATION UPDATE NOTIFY
@enduml
```

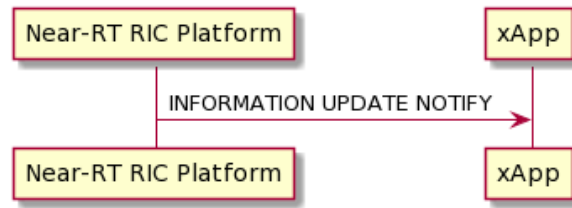


Figure 7.2.5.2-1: Information Update Notify procedure, successful operation

The Near-RT RIC platform initiates the procedure by sending the INFORMATION UPDATE NOTIFY message.

The message shall include:

- the *Subscription ID* IE, which indicates the associated subscription;
- the *SDL Data Type ID* IE, which indicates the SDL data Type carried in the message;
- the *SDL Data Block* IE, which carries the SDL data and shall include:
 - the *SDL Update Notify Unit* IE, which contains a single item of the SDL data;
 - the *Modification Type* IE, which indicates type of change (added, modified, removed, etc.) associated with the SDL data in the *SDL Update Notify Unit* IE.

Once the xApp has received the message, xApp can determine to retrieve the updated information through the Information Fetch procedure described in clause 7.2.6.

7.2.5.3 Unsuccessful Operation

Not applicable.

7.2.5.4 Abnormal Conditions

Not applicable.

7.2.6 SDL Information Fetch procedure

7.2.6.1 General

This procedure is used by the xApp to fetch certain information from the SDL.

7.2.6.2 Successful Operation

```

@startuml
Skin rose
skinparam ParticipantPadding 50
skinparam BoxPadding 10
skinparam lifelineStrategy solid
participant "Near-RT RIC Platform" as near
participant "xApp" as app
app -> near: INFORMATION FETCH REQUEST
near --> app: INFORMATION FETCH RESPONSE
@enduml
  
```

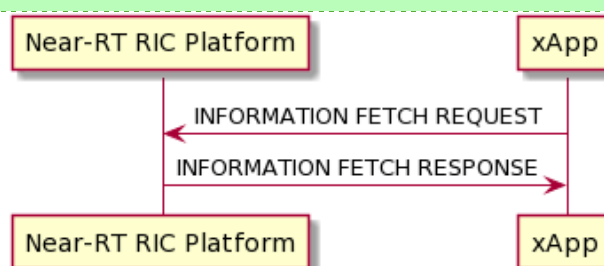


Figure 7.2.6.2-1: SDL Information Fetch procedure, successful operation

The xApp initiates the procedure by sending the INFORMATION FETCH REQUEST message.

The message shall include:

- the *SDL Data Type ID* IE, which indicates the requested SDL data type.

The message may include:

- the *SDL Data Description* IE, which contains the detailed descriptions for the requested data.

When the Near-RT RIC platform receives the request, it searches the local database or data sources for the fetched information.

If the SDL contains matching data for the fetch request, the Near-RT RIC platform sends the requested data to the xApp in the INFORMATION FETCH RESPONSE message.

7.2.6.3 Unsuccessful Operation

If the data corresponding to a fetch request is not found, the Near-RT RIC platform sends an INFORMATION FETCH FAILURE message with an appropriate failure cause.

7.2.6.4 Abnormal Conditions

If the Near-RT RIC platform receives a INFORMATION FETCH REQUEST message containing *SDL Data Type ID* IE that is not supported by the Near-RT RIC platform, or the optional *SDL Data Description* IE definition is invalid, the Near-RT RIC platform shall send the INFORMATION FETCH FAILURE message to the xApp with an appropriate cause value.

7.2.7 Store Data procedure

7.2.8 Modify Data procedure

7.2.9 Xapp Shared Data Fetch procedure

7.2.9.1 General

This procedure is used by the Near-RT RIC platform or the data consumer xApp to fetch specific types of data shared by the data producer xApp.

7.2.9.2 Successful Operation

```
@startuml
Skin rose
skinparam ParticipantPadding 50
skinparam BoxPadding 10
skinparam lifelineStrategy solid
participant "Data consumer \n(Near-RT RIC platform or xApp)" as consumer
participant "Data producer xApp" as producerxApp
consumer -> producerxApp: XAPP SHARED DATA FETCH REQUEST
producerxApp -> consumer: XAPP SHARED DATA FETCH RESPONSE
@enduml
```

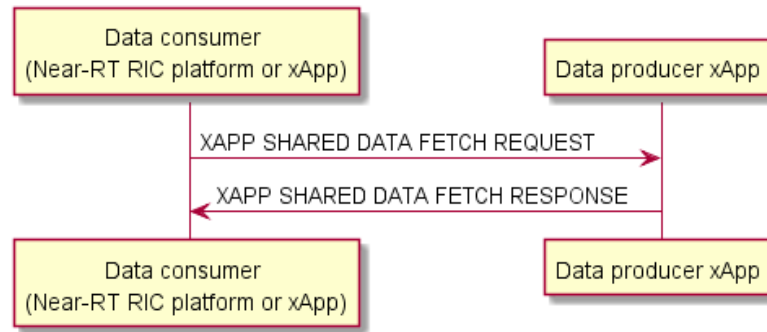


Figure 7.2.9.2-1: xApp Shared Data Fetch procedure, successful operation

The data consumer initiates the procedure by sending the XAPP SHARED DATA FETCH REQUEST message.

The message shall include:

- the *SDL Data Type ID* IE, which indicates the requested SDL data type.

The message may include:

- the *SDL Data Description* IE, which contains the detailed descriptions for the requested data.

Upon receiving the message, the data producer xApp shall check the authorization for the data consumer to access the requested data. If the requested data is available at the producer xApp, the producer xApp shall send the requested data to the data consumer in the XAPP SHARED DATA FETCH RESPONSE message.

7.2.9.3 Unsuccessful Operation

```

@startuml
Skin rose
skinparam ParticipantPadding 50
skinparam BoxPadding 10
skinparam lifelineStrategy solid
participant "Data consumer \n(Near-RT RIC platform or xApp)" as consumer
participant "Data producer xApp" as producerxApp
consumer -> producerxApp: XAPP SHARED DATA FETCH REQUEST
producerxApp -> consumer: XAPP SHARED DATA FETCH FAILURE
@enduml

```

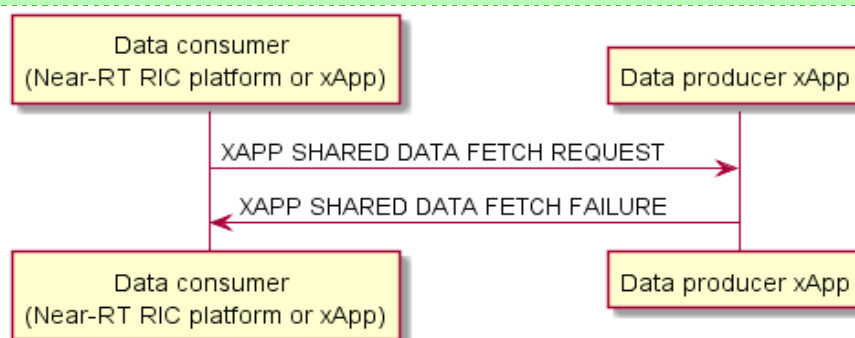


Figure 7.2.9.3-1: xApp Shared Data Fetch procedure, unsuccessful operation

If the data consumer is not authorized, or the requested data specified in the XAPP SHARED DATA FETCH REQUEST is unavailable, the producer xApp shall send an XAPP SHARED DATA FETCH FAILURE message with appropriate cause.

7.2.9.4 Abnormal Conditions

Not applicable.

7.2.10 Xapp Shared Data Subscribe procedure

7.2.10.1 General

This procedure is used by the Near-RT RIC platform or the data consumer xApp to subscribe to specific types of data shared by the data producer xApp.

7.2.10.2 Successful Operation

```
@startuml
Skin rose
skinparam ParticipantPadding 50
skinparam BoxPadding 10
skinparam lifelineStrategy solid
participant "Data consumer \n(Near-RT RIC platform or xApp)" as consumer
participant "Data producer xApp" as producerxApp
consumer -> producerxApp: XAPP SHARED DATA SUBSCRIPTION REQUEST
producerxApp --> consumer: XAPP SHARED DATA SUBSCRIPTION RESPONSE
@enduml
```

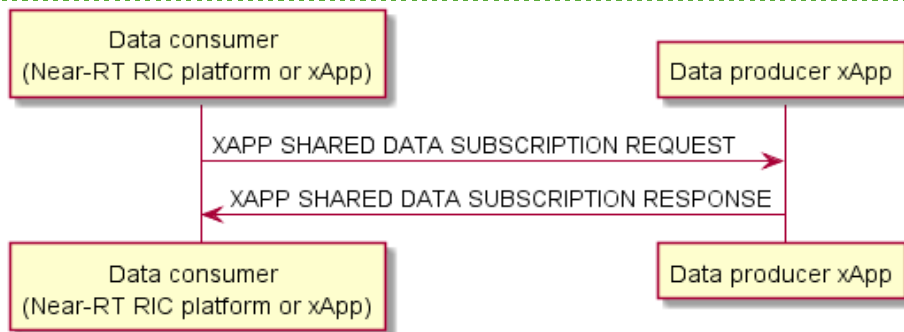


Figure 7.2.10.2-1: xApp Shared Data Subscribe procedure, successful operation

The data consumer initiates the procedure by sending the XAPP SHARED DATA SUBSCRIPTION REQUEST message.

The message shall include:

- the *SDL Data Type ID* IE, which indicates the requested SDL data type;
- the *Delivery Method* IE, which indicates the method of data delivery;
- the *Notification Destination* IE, which specifies the endpoint to receive associated notification(s);
- the *Notification Criteria* IE, which specifies the timing related to the notification(s). It shall include:
 - the *Notification Method* IE, which indicates the notification(s) are periodic or event triggered;
 - the *Notification Period* IE if the *Notification Method* IE is set to "PERIODIC", which indicates the time interval between periodic notifications;
 - the *Trigger Event Definition* IE if the *Notification Method* IE is set to "EVENT_TRIGGERED", which contains the description of the triggering event;

The message may include:

- the *SDL Data Description* IE, which contains the detailed descriptions for the requested data.

Upon receiving the message, the data producer xApp shall check the authorization for the data consumer to subscribe to the requested data. If the requested subscription can be fulfilled, the producer xApp shall prepare necessary resource for the subscription, assign a subscription ID, and send the XAPP SHARED DATA SUBSCRIPTION RESPONSE message to the data consumer.

The message shall include:

- the *Subscription ID* IE, which contains the subscription ID.

7.2.10.3 Unsuccessful Operation

```
@startuml
Skin rose
skinparam ParticipantPadding 50
skinparam BoxPadding 10
skinparam lifelineStrategy solid
participant "Data consumer \n(Near-RT RIC platform or xApp)" as consumer
participant "Data producer xApp" as producerxApp
consumer -> producerxApp: XAPP SHARED DATA SUBSCRIPTION REQUEST
producerxApp -> consumer: XAPP SHARED DATA SUBSCRIPTION FAILURE
@enduml
```

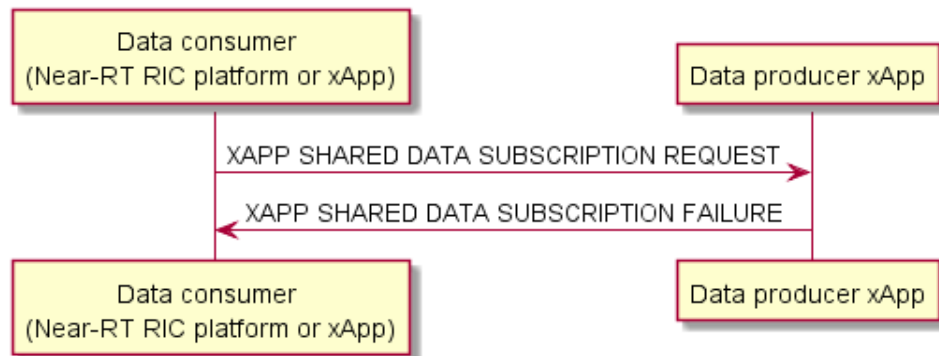


Figure 7.2.10.3-1: xApp Shared Data Subscribe procedure, unsuccessful operation

If the data consumer is not authorized, or the requested subscription cannot be fulfilled, the producer xApp shall send the XAPP SHARED DATA FETCH FAILURE message with appropriate cause.

7.2.10.4 Abnormal Conditions

Not Applicable.

7.2.11 Xapp Shared Data Subscription Delete procedure

7.2.11.1 General

This procedure is used by the Near-RT RIC platform or the data consumer xApp to delete an existing xApp shared data subscription toward the data producer xApp.

7.2.11.2 Successful Operation

```
@startuml
Skin rose
skinparam ParticipantPadding 50
skinparam BoxPadding 10
skinparam lifelineStrategy solid
participant "Data consumer \n(Near-RT RIC platform or xApp)" as consumer
participant "Data producer xApp" as producerxApp
consumer -> producerxApp: XAPP SHARED DATA SUBSCRIPTION DELETE REQUEST
producerxApp -> consumer: XAPP SHARED DATA SUBSCRIPTION DELETE RESPONSE
@enduml
```

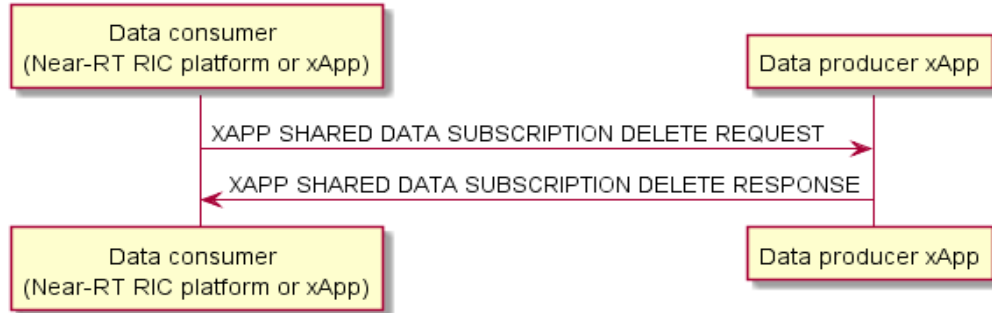


Figure 7.2.11.2-1: xApp Shared Data Subscription Delete procedure, successful operation

The data consumer initiates the procedure by sending the XAPP SHARED DATA SUBSCRIPTION DELETE message.

The message shall include:

- the *Subscription ID* IE, which indicates the subscription to delete;

Upon receiving the message, the data producer xApp shall check the authorization for the data consumer. If the subscription is present, the data producer xApp shall release the resource for the subscription, and send the XAPP SHARED DATA SUBSCRIPTION DELETE RESPONSE message to the data consumer.

7.2.11.3 Unsuccessful Operation

```

@startuml
Skin rose
skinparam ParticipantPadding 50
skinparam BoxPadding 10
skinparam lifelineStrategy solid
participant "Data consumer \n(Near-RT RIC platform or xApp)" as consumer
participant "Data producer xApp" as producerxApp
consumer -> producerxApp: XAPP SHARED DATA SUBSCRIPTION DELETE REQUEST
producerxApp -> consumer: XAPP SHARED DATA SUBSCRIPTION DELETE FAILURE
@enduml
  
```

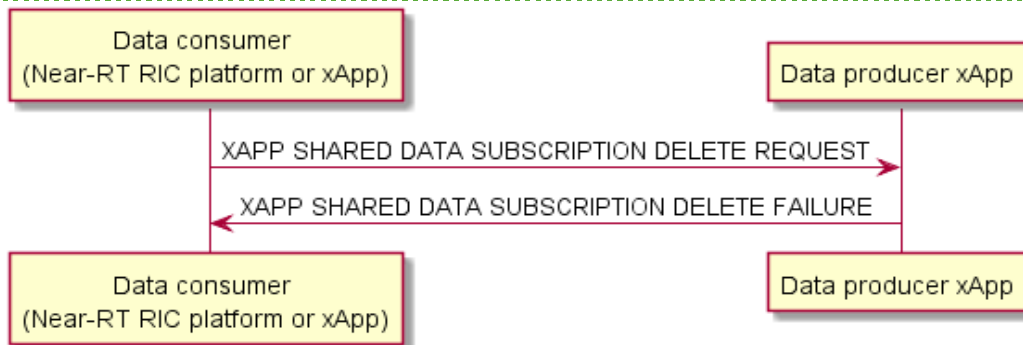


Figure 7.2.11.3-1: xApp Shared Data Subscription Delete procedure, unsuccessful operation

If the data consumer is not authorized, or the subscription indicated by the *Subscription ID* IE is not present, the data producer xApp shall send the XAPP SHARED DATA SUBSCRIPTION DELETE FAILURE message with appropriate cause.

7.2.11.4 Abnormal Conditions

Not applicable.

7.2.12 Xapp Shared Data Push procedure

7.2.12.1 General

This procedure is used by the data producer xApp to push the subscribed SDL data to the data consumer according to an existing subscription. The procedure is used when the *Delivery Method* IE set to "PUSH_DELTA" or "PUSH_FULL" in the xApp shared data subscription request.

7.2.12.2 Successful Operation

```
@startuml
Skin rose
skinparam ParticipantPadding 50
skinparam BoxPadding 10
skinparam lifelineStrategy solid
participant "Data consumer \n(Near-RT RIC platform or xApp)" as consumer
participant "Data producer xApp" as producerxApp
producerxApp -> consumer: XAPP SHARED DATA PUSH
@enduml
```

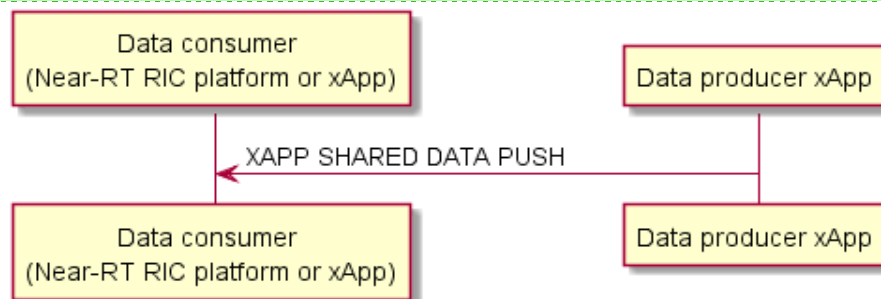


Figure 7.2.12.2-1: xApp Shared Data Push procedure, successful operation

The data producer xApp initiates the procedure by sending the XAPP SHARED DATA PUSH message.

The message shall include:

- the *Subscription ID* IE, which indicates the associated subscription;
- the *SDL Data Type ID* IE, which indicates the SDL data Type carried in the message;
- the *SDL Data Block* IE, which carries the SDL data and shall include:
 - the *SDL Data Unit* IE, which contains a single item of the SDL data;
 - the *Modification Type* IE, which indicates type of change (added, modified, removed, etc.) associated with the SDL data in the *SDL Data Unit* IE.

7.2.12.3 Unsuccessful Operation

Not applicable.

7.2.12.4 Abnormal Conditions

Not applicable.

7.3 Elements for SDL API Communication

7.3.1 General

Sub clauses 7.3.2 and 7.3.3 describe the structure of the messages and information elements required for the SDL API in tabular format.

7.3.2 Message Functional Definition and Content

7.3.2.1 SDL CLIENT REGISTRATION REQUEST

7.3.2.2 SDL CLIENT REGISTRATION RESPONSE

7.3.2.3 SDL CLIENT REGISTRATION FAILURE

7.3.2.4 SDL CLIENT DE-REGISTRATION REQUEST

7.3.2.5 SDL CLIENT DE-REGISTRATION RESPONSE

7.3.2.6 SUBSCRIBE INFORMATION REQUEST

This message is sent by an xApp to the Near-RT RIC platform to subscribe to the interested information.

Direction: xApp -> Near-RT RIC platform

IE/Group Name	Presence	Range	IE type and reference	Semantics description	Criticality	Assigned Criticality
Message Type	M		Error! Reference source not found.		YES	Reject
Procedure Transaction ID	M		10.1.1		YES	Reject
SDL Data Type ID	M		7.3.3.21		YES	Reject
Delivery Method	M		7.3.3.4		YES	Reject
SDL Data Description	O		7.3.3.22	This IE may be absent, if a default SDL Data Description is available for the SDL Type ID and used in this request.	YES	Reject
Notification Destination	M		10.1.9		YES	Reject
Notification Criteria	M		7.3.3.24		YES	Reject

7.3.2.7 SUBSCRIBE INFORMATION RESPONSE

This message is sent by the Near-RT RIC platform after successful acceptance of an xApp's request to subscribe to the information in the SDL.

Direction: Near-RT RIC platform -> xApp

IE/Group Name	Presence	Range	IE type and reference	Semantics description	Criticality	Assigned Criticality
Message Type	M		Error! Reference source not found.		YES	Reject
Procedure Transaction ID	M		10.1.1		YES	Reject
Subscription ID	M		7.3.3.26		YES	Reject

7.3.2.8 SUBSCRIBE INFORMATION FAILURE

This message is sent by the Near-RT RIC platform after failure to accept an xApp's request to subscribe to information in the SDL.

Direction: Near-RT RIC platform -> xApp

IE/Group Name	Presence	Range	IE type and reference	Semantics description	Criticality	Assigned Criticality
Message Type	M		Error! Reference source not found.		YES	Reject
Procedure Transaction ID	M		10.1.1		YES	Reject
Subscription Reject Cause	M		7.3.3.9			
Time to Wait	O		7.3.3.13	If absent, no time to wait is required.		

7.3.2.9 INFORMATION PUSH

This message is sent by the Near-RT RIC platform to an xApp containing the information subscribed by the xApp.

Direction: Near-RT RIC platform -> xApp

IE/Group Name	Presence	Range	IE type and reference	Semantics description	Criticality	Assigned Criticality
Message Type	M		7.3.3.1		YES	Reject
Procedure Transaction ID	M		10.1.1		YES	Reject
Subscription ID	M		7.3.3.26		YES	Reject
SDL Data Type ID	M		7.3.3.21			
SDL Data Block		<i>0..1</i>			YES	Reject
> SDL Data Block Item		<i>1..< maxnoof SDLData Units ></i>			-	-
>> Modification Type	C		7.3.3.10	Shall be present when the Delivery Method IE in the subscription request is set to "PUSH_DELTA".	-	-
>> SDL Data Unit	M		7.3.3.23		-	-

Range bound	Explanation
<i>maxnoofSDLDataUnits</i>	Maximum number of SDL Data Units that can be carried in an SDL Data Block. Value is 1048576.

7.3.2.10 INFORMATION UPDATE NOTIFY

This message is sent by the Near-RT RIC platform to inform an xApp that some of the information subscribed by the xApp has been updated.

Direction: Near-RT RIC platform -> xApp

IE/Group Name	Presence	Range	IE type and reference	Semantics description	Criticality	Assigned Criticality
Message Type	M		7.3.3.1		YES	Reject
Procedure Transaction ID	M		10.1.1		YES	Reject
Subscription ID	M		7.3.3.26		YES	Reject
SDL Data Type ID	M		7.3.3.21			
SDL Update Notify Block		0..1			YES	Reject
> SDL Update Notify Block Item		1..< <i>maxnoof SDLUpdateNotify Units</i> >			-	-
>> Modification Type	M		7.3.3.10		-	-
>> SDL Update Notify Unit	M		7.3.3.27		-	-

Range bound	Explanation
<i>maxnoofSDLUpdateNotifyUnits</i>	Maximum number of SDL Update Notify Units that can be carried in an SDL Update Notify Block. Value is 1048576.

7.3.2.11 INFORMATION FETCH REQUEST

This message is sent by an xApp to the Near-RT RIC platform to request certain information subscribed by the xApp.

Direction: xApp -> Near-RT RIC platform

IE/Group Name	Presence	Range	IE type and reference	Semantics description	Criticality	Assigned Criticality
Message Type	M		7.3.3.1		YES	Reject
Procedure Transaction ID	M		10.1.1		YES	Reject
SDL Data Type ID	M		7.3.3.21		YES	Reject
SDL Data Description	O		7.3.3.22	This IE may be absent when a default definition of it exists and is used in the request.	YES	Reject

7.3.2.12 INFORMATION FETCH RESPONSE

This message is sent by the Near-RT RIC platform to an xApp containing the information requested by the xApp.

Direction: Near-RT RIC platform -> xApp

IE/Group Name	Presence	Range	IE type and reference	Semantics description	Criticality	Assigned Criticality
Message Type	M		7.3.3.1		YES	Reject
Procedure Transaction ID	M		10.1.1		YES	Reject
SDL Data Type ID	M		7.3.3.21		YES	Reject
SDL Data Block		0..1			YES	Reject
> SDL Data Block Item		1..< <i>maxnoof SDLData Units</i> >			-	-
>> SDL Data Unit	M		7.3.3.23		-	-

7.3.2.13 INFORMATION FETCH FAILURE

This message is sent by the Near-RT RIC platform to an xApp if the Information fetch request failed.

Direction: Near-RT RIC platform -> xApp

IE/Group Name	Presence	Range	IE type and reference	Semantics description	Criticality	Assigned Criticality
Message Type	M		7.3.3.1		YES	Reject
Procedure Transaction ID	M		10.1.1		YES	Reject
Fetch Failure Cause	M		7.3.3.18	Cause for failure	YES	Reject

7.3.2.14 STORE DATA REQUEST

7.3.2.15 STORE DATA RESPONSE

7.3.2.16 STORE DATA FAILURE

7.3.2.17 MODIFY DATA REQUEST

7.3.2.18 MODIFY DATA RESPONSE

7.3.2.19 MODIFY DATA FAILURE

7.3.2.20 XAPP SHARED DATA FETCH REQUEST

This message is sent by the Near-RT RIC platform or a data consumer xApp to fetch data shared by the data producer xApp.

Direction: Near-RT RIC platform / Data consumer xApp -> Data producer xApp

IE/Group Name	Presence	Range	IE type and reference	Semantics description	Criticality	Assigned Criticality
Message Type	M		Error! Reference source not found.		YES	Reject
Procedure Transaction ID	M		10.1.1		YES	Reject
SDL Data Type ID	M		7.3.3.21		YES	Reject
SDL Data Description	O		7.3.3.22	This IE may be absent when a default definition of it exists and is used in the request.	YES	Reject

7.3.2.21 XAPP SHARED DATA FETCH RESPONSE

This message is sent by the data producer xApp containing the fetched data requested by the Near-RT RIC platform or the data consumer xApp.

Direction: Data producer xApp -> Near-RT RIC platform / Data consumer xApp

IE/Group Name	Presence	Range	IE type and reference	Semantics description	Criticality	Assigned Criticality
---------------	----------	-------	-----------------------	-----------------------	-------------	----------------------

Message Type	M		Error! Reference source not found.		YES	Reject
Procedure Transaction ID	M		10.1.1		YES	Reject
SDL Data Type ID	M		7.3.3.21		YES	Reject
SDL Data Block		<i>0..1</i>			YES	Reject
> SDL Data Block Item		<i>1..<maxnoof SDLData Units></i>			-	-
>> SDL Data Unit	M		7.3.3.23		-	-

Range bound	Explanation
<i>maxnoofSDLDataUnits</i>	Maximum number of SDL Data Block Units that can be carried in an SDL Data Block. Value is 1048576.

7.3.2.22 XAPP SHARED DATA FETCH FAILURE

This message is sent by the data producer xApp to the Near-RT RIC platform or the data consumer xApp if the xApp shared data fetch request fails.

Direction: Data producer xApp -> Near-RT RIC platform / Data consumer xApp

IE/Group Name	Presence	Range	IE type and reference	Semantics description	Criticality	Assigned Criticality
Message Type	M		7.3.3.1		YES	Reject
Procedure Transaction ID	M		10.1.1	Procedure Transaction ID	YES	Reject
Fetch Failure Cause	M		7.3.3.18	Cause for failure	YES	Reject

7.3.2.23 XAPP SHARED DATA SUBSCRIPTION REQUEST

This message is sent by the Near-RT RIC platform or a data consumer xApp to subscribe to data shared by the data producer xApp.

Direction: Near-RT RIC platform / Data consumer xApp -> Data producer xApp

IE/Group Name	Presence	Range	IE type and reference	Semantics description	Criticality	Assigned Criticality
Message Type	M		Error! Reference source not found.		YES	Reject
Procedure Transaction ID	M		10.1.1		YES	Reject
SDL Data Type ID	M		7.3.3.21		YES	Reject
SDL Data Description	O		7.3.3.22	This IE may be absent, if a default SDL Data Description is available for the SDL Type ID and used in this request.	YES	Reject
Delivery Method	M		7.3.3.4			
Notification Destination	M		10.1.9		YES	Reject
Notification Criteria	M		7.3.3.24		YES	Reject

7.3.2.24 XAPP SHARED DATA SUBSCRIPTION RESPONSE

This message is sent by the data producer xApp to indicate success of the subscription request.

Direction: Data producer xApp -> Near-RT RIC platform / Data consumer xApp

IE/Group Name	Presence	Range	IE type and reference	Semantics description	Criticality	Assigned Criticality
Message Type	M		Error! Reference source not found.		YES	Reject
Procedure Transaction ID	M		10.1.1		YES	Reject
Subscription ID	M		7.3.3.26		YES	Reject

7.3.2.25 XAPP SHARED DATA SUBSCRIPTION FAILURE

This message is sent by the data producer xApp to indicate failure of the subscription request.

Direction: Data producer xApp -> Near-RT RIC platform / Data consumer xApp

IE/Group Name	Presence	Range	IE type and reference	Semantics description	Criticality	Assigned Criticality
Message Type	M		Error! Reference source not found.		YES	Reject
Procedure Transaction ID	M		10.1.1		YES	Reject
Subscription Reject Cause	M		7.3.3.9		YES	Reject

7.3.2.26 XAPP SHARED DATA PUSH

This message is sent by the data producer xApp to push its shared data to the subscriber (data consumer xApp or the Near-RT RIC platform).

Direction: Data producer xApp -> Near-RT RIC platform / Data consumer xApp

IE/Group Name	Presence	Range	IE type and reference	Semantics description	Criticality	Assigned Criticality
Message Type	M		7.3.3.1		YES	Reject
Subscription ID	M		7.3.3.26	The same Subscription ID as in the subscription response.	YES	Reject
SDL Data Type ID	M		7.3.3.21		YES	Reject
SDL Data Block		0..1			YES	Reject
> SDL Data Block Item		1..< maxnoof SDLData Units >			-	-
>> Modification Type	C		7.3.3.10	Shall be present when the <i>Delivery Method</i> IE in the subscription request is set to "PUSH_DELTA".	-	-
>> SDL Data Unit	M		7.3.3.23		-	-

Range bound	Explanation
<i>maxnoofSDLDataUnits</i>	Maximum number of SDL Data Units that can be carried in an SDL Data Block. Value is 1048576.

7.3.2.27 XAPP SHARED DATA SUBSCRIPTION DELETE REQUEST

This message is sent by the Near-RT RIC platform or a data consumer xApp to delete an existing subscription to the xApp shared data.

Direction: Near-RT RIC platform / Data consumer xApp -> Data producer xApp

IE/Group Name	Presence	Range	IE type and reference	Semantics description	Criticality	Assigned Criticality
Message Type	M		Error! Reference source not found.		YES	Reject
Procedure Transaction ID	M		10.1.1		YES	Reject
Subscription ID	M		7.3.3.26	The same Subscription ID as in the subscription response.	YES	Reject

7.3.2.28 XAPP SHARED DATA SUBSCRIPTION DELETE RESPONSE

This message is sent by the data producer xApp to indicate successful deletion of an existing subscription to the xApp shared data.

Direction: Data producer xApp -> Near-RT RIC platform / Data consumer xApp

IE/Group Name	Presence	Range	IE type and reference	Semantics description	Criticality	Assigned Criticality
Message Type	M		Error! Reference source not found.		YES	Reject
Procedure Transaction ID	M		10.1.1		YES	Reject

7.3.2.29 XAPP SHARED DATA SUBSCRIPTION DELETE FAILURE

This message is sent by the data producer xApp to indicate failure of the subscription request.

Direction: Data producer xApp -> Near-RT RIC platform / Data consumer xApp

IE/Group Name	Presence	Range	IE type and reference	Semantics description	Criticality	Assigned Criticality
Message Type	M		Error! Reference source not found.		YES	Reject
Procedure Transaction ID	M		10.1.1		YES	Reject
Subscription Reject Cause	M		7.3.3.9		YES	Reject

7.3.3 Information Element Definitions

7.3.3.1 Message Type

The Message Type IE identifies the message being sent. It is mandatory for all messages.

IE/Group Name	Presence	Range	IE Type and Reference	Semantics Description
Message Type				
>Procedure Type	M		CHOICE (SDL Subscription Information, SDL Information Push, SDL Information Update Notify, SDL Information Fetch, Xapp Shared Data Fetch, Xapp Shared Data Subscribe, Xapp Shared Data Subscription Delete, Xapp Shared Data Push, ...)	
>Type of Message	M		CHOICE (Request, Response, Failure, ...)	Refer to Table 7.1.2-3 for applicable choices for each procedure type

7.3.3.2 Void

7.3.3.3 Void

7.3.3.4 Delivery Method

The Delivery Method IE indicates the delivery method in which the xApp wants the information to be delivered.

IE/Group Name	Presence	Range	IE type and reference	Semantics description
Delivery Method	M		ENUMERATED (UPDATE_NOTIFY, PUSH_DELTA, PUSH_FULL, ...)	

Table 7.3.3.4-1: Meaning of Delivery Method

Value	Description
UPDATE_NOTIFY	Notifies only the summary of change in the data specified by the <i>SDL Data Description</i> IE. It uses the SDL Information Update Notify procedure for the notification.
PUSH_DELTA	Notifies the data that has changed since the last notification associated with the same subscription, from the data specified by the <i>SDL Data Description</i> IE. It uses the SDL Information Push procedure for the notification.
PUSH_FULL	Notifies all the data specified by the <i>SDL Data Description</i> IE. It uses the SDL Information Push procedure for the notification.

7.3.3.5 Void

7.3.3.6 Void

7.3.3.7 Void

7.3.3.8 Void

7.3.3.9 Subscription Reject Cause

The Subscription Reject Cause IE indicates the reason the SDL rejected the subscription.

IE/Group Name	Presence	Range	IE type and reference	Semantics description
Subscription Reject Cause	M		ENUMERATED (SDL reached capacity, Information type not supported, Invalid filter, Unspecified, ...)	

7.3.3.10 Modification Type

The Modification Type IE informs the type of modification that the SDL underwent.

IE/Group Name	Presence	Range	IE type and reference	Semantics description
Modification Type	M		ENUMERATED (Added, Modified, Removed,)	

7.3.3.11 Void

7.3.3.12 Void

7.3.3.13 Time to wait

This IE defines the minimum allowed waiting times.

IE/Group Name	Presence	Range	IE type and reference	Semantics description
Time to wait	M		INTEGER (1..300)	Unit in seconds.

7.3.3.14 Void

7.3.3.15 Void

7.3.3.16 Void

7.3.3.17 Void

7.3.3.18 Fetch Failure Cause

IE/Group Name	Presence	Range	IE type and reference	Semantics description
Fetch Failure Cause	M		ENUMERATED (Invalid filter, Information type not supported, No information present, Unspecified, ...)	

7.3.3.19 Void

7.3.3.20 Void

7.3.3.21 SDL Data Type ID

IE/Group Name	Presence	Range	IE type and reference	Semantics description
SDL Data Type ID	M		STRING	Uniquely identifies a SDL data type. It shall be constructed with 3 fields concatenated by ":" (colon), as "{namespace}:{name}:{version}". Clause 7.3.3.28 collects the known SDL Data Type IDs per data category.

7.3.3.22 SDL Data Description

IE/Group Name	Presence	Range	IE type and reference	Semantics description
CHOICE <i>Data Category</i>				
> <i>E2 Node Information</i>				
>> SDL Data Description For E2 Node Information	M		7.3.3.28.1.2	
> <i>RAN Analytics Information</i>				
>> SDL Data Description For RAI	M		7.3.3.28.2.2	

7.3.3.23 SDL Data Unit

IE/Group Name	Presence	Range	IE type and reference	Semantics description
CHOICE <i>Data Category</i>				
> <i>E2 Node Information</i>				
>> SDL Data Unit For E2 Node Information	M		7.3.3.28.1.3	
> <i>RAN Analytics Information</i>				
>> SDL Data Unit For RAI	M		7.3.3.28.2.3	

7.3.3.24 Notification Criteria

IE/Group Name	Presence	Range	IE type and reference	Semantics description
Notification Method	M		ENUMERATED (PERIODIC, EVENT_TRIGGERED, ...)	
Notification Period	C		INTEGER	Indicates the periodicity of notifications, in units of milliseconds. Shall be present when Notification Method is set to "PERIODIC".
Trigger Event Definition	C		7.3.3.25	Shall be present when Notification Method is set to "EVENT_TRIGGERED".

7.3.3.25 Trigger Event Definition

IE/Group Name	Presence	Range	IE type and reference	Semantics description
CHOICE <i>Data Category</i>				
> <i>E2 Node Information</i>				
>> Trigger Event Definition For E2 Node Information	M		7.3.3.28.1.4	
> <i>RAN Analytics Information</i>				
>> Trigger Event Definition For RAI	M		7.3.3.28.2.4	

7.3.3.26 Subscription ID

IE/Group Name	Presence	Range	IE type and reference	Semantics description
Subscription ID	M		STRING	Uniquely identifies a SDL data subscription among all the SDL subscriptions created by a consumer.

7.3.3.27 SDL Update Notify Unit

IE/Group Name	Presence	Range	IE type and reference	Semantics description
CHOICE <i>Data Category</i>				
> <i>E2 Node Information</i>				

>> SDL Update Notify Unit For E2 Node Information	M		7.3.3.28.1.5	
---	---	--	--------------	--

7.3.3.28 SDL Data Type specific IE Definitions

7.3.3.28.1 E2 Node Information related IEs

7.3.3.28.1.1 SDL Data Type ID

To construct the *SDL Data Type ID* IE for E2 Node Information,

- the *namespace* field is "ORAN";
- the *name* field is the text value of the *Information Type* IE in clause 7.3.3.28.1.6, i.e., "E2NodeList", "E2NodeComponentList", "E2NodeRanFunctionList", etc.;
- the *version* field is "1.0.0".

EXAMPLE: "ORAN: E2NodeList:1.0.0"

7.3.3.28.1.2 SDL Data Description IE for E2 Node Information

IE Name	Presence	Range	Data type and reference	Semantics description
Information Type	M		7.3.3.28.1.6	
> Filter List		0..1		Specifies the subset of the data to be subscribed for the Information Type. Multiple filters can be specified, in which case, a logical AND shall be performed across all filters.
>> Filter Item		0..<maxFilterID>		
>>> Filter Definition	M		7.3.3.28.1.7	

Range bound	Explanation
maxnoofFilters	Maximum number of Filter Items to be contained in the Filter List. Value is 256.

7.3.3.28.1.3 SDL Data Unit IE for E2 Node Information

IE Name	Presence	Range	IE type and reference	Semantics description
Information Unit	M		7.3.3.28.1.11	

7.3.3.28.1.4 Trigger Event Definition IE For E2 Node Information

IE Name	Presence	Range	IE type and reference	Semantics description
Notify On Data Change Flag	O		ENUMERATED (TRUE, ...)	When the IE is present and set to "TRUE", the notifications shall be triggered when the E2 Node information described by the <i>SDL Data Description</i> IE

				changes since the last notification associated with the same subscription.
--	--	--	--	--

7.3.3.28.1.5 SDL Update Notify IE For E2 Node Information

IE Name	Presence	Range	IE type and reference	Semantics description
Notification Unit			7.3.3.28.1.12	

7.3.3.28.1.6 Information Type

The IE indicates the type of E2 Node information the xApp requests.

IE Name	Presence	Range	IE type and reference	Semantics description
Information Type	M		ENUMERATED (E2NodeList, E2NodeComponentConfig, E2NodeRanFunction, ...)	

7.3.3.28.1.7 Filter Definition

The IE defines a filter on the E2 Node information that the xApp requests. A Filter Definition is satisfied when the logical statement “X p Y” is True, where:

- X is the filter type, represented by the Filter Type IE;
- p is the filter operator, represented by the Filter Condition IE;
- Y is the set of the target values, represented by the Filter Condition Value IE.

IE/Group Name	Presence	Range	IE type and reference	Semantics description
Filter Type	M		7.3.3.28.1.8	
Filter Condition	M		7.3.3.28.1.9	
Filter Condition Value	M		7.3.3.28.1.10	

7.3.3.28.1.8 Filter Type

The IE defines the filtering condition type to identify the E2 Node information the xApp request.

IE/Group Name	Presence	Range	IE type and reference	Semantics description
Filter Type	M		ENUMERATED (E2NodeID, ComponentType, ServiceModel, ...)	Valid values of Filter Type IE are depending on the type of E2 Node Information indicated in the Information Type IE. See table 7.3.3.28.1.8-1 for the valid settings.

Table 7.3.3.28.1.8-1: Valid Filter Type IE settings

Filter Type	Information Type		
	E2NodeList	E2NodeComponentConfig	E2NodeRanFunction
E2NodeID	NO	YES	YES
ComponentType	YES	YES	NO
ServiceModel	YES	NO	YES

7.3.3.28.1.9

Filter Condition

The IE defines the filtering condition to identify the E2 Node information the xApp requests.

IE/Group Name	Presence	Range	IE type and reference	Semantics description
Filter Condition	M		ENUMERATED (ContainedIn, NotContainedIn, Equals, GreaterThan, LessThan, ...)	

7.3.3.28.1.10

Filter Condition Value

The IE defines the filtering condition value to identify the information the xApp requests.

IE/Group Name	Presence	Range	IE type and reference	Semantics description
CHOICE <i>Filter Condition Value</i>				
> <i>E2NodeID</i>				
>> E2 Node ID List		1		
>>> E2 Node Item		1..65536		
>>>>E2 Node ID	M		7.3.3.14	
>>>>E2 Node State	O		7.3.3.28.1.13	
> <i>ComponentType</i>				
>> E2 Node Component Type List		1		
>>> E2 Node Component Type Item		1..65536		
>>>>E2 Node Component Type	M		<i>E2 Node Component Interface Type</i> IE, clause 9.2.26 of [3]	
> <i>ServiceModel</i>				
>> RAN Function OID List		1		
>>> RAN Function OID Item		1..65536		
>>>>RAN function OID	M		<i>RAN Function OID IE</i> , clause 9.2.31 of [3]	

7.3.3.28.1.11

Information Unit

The IE contains the E2 Node information that the Near-RT RIC platform sends to the xApp.

IE/Group Name	Presence	Range	IE type and reference	Semantics description
CHOICE Information Unit	M		Contents depend on Information Type. See following table for allowed settings.	
> <i>E2 Node List Unit</i>				
>>E2 Node ID	M		10.1.6	
>>E2 Node State	M		7.3.3.28.1.13	
> <i>E2 Node Component Config Unit</i>				
>>E2 Node ID	M		10.1.6	
>>> E2 Node Component Config List		1..<maxofE2nodeComponents>		Value of "maxofE2nodeComponents" defined in 9.1.2.2 of [3].
>>>>E2 Node Component Type	M		<i>E2 Node Component Interface Type</i> IE, clause 9.2.26 of [3]	
>>>>E2 Node Component ID	O		Clause 9.2.32 of [3]	
>>>>E2 Node Component Configuration	M		Clause 9.2.27 of [3]	

>>> Last Modified E2 Node Version	M		INTEGER	The Near-RT RIC platform maintains a version number for the E2 Node Information of each E2 Node. Incremented whenever any RAN Function or Component config changes. This is the version number when the E2 Node Information changed last time.
> <i>Ran Function Unit</i>				
>>E2 Node ID	M		10.1.6	
>>>RAN Function List		1..<maxofRANfunctionID>		Value of "maxofRANfunctionIDs" defined in 9.1.2.2 of [3].
>>> RAN Function ID	M		10.1.7	
>>> RAN Function Definition	M		Clause 9.2.23 of [3].	
>>> RAN Function OID	M		Clause 9.2.31 of [3].	
>>> Last Modified E2 Node Version	M		INTEGER	The Near-RT RIC platform maintains a version number for the E2 Node Information of each E2 Node. Incremented whenever any RAN Function or Component config changes. This is the version number when the E2 Node Information changed last time.

Information Type	Information Unit
E2NodeList	E2 Node List Unit
E2NodeComponentConfig	E2 Node Component Config Unit
E2NodeRanFunction	Ran Function Unit

7.3.3.28.1.12 Notification Unit

The IE contains the summary of E2 Node Information updates sent to the xApp.

IE/Group Name	Presence	Range	IE type and reference	Semantics description
E2 Node ID	M		10.1.6	
Current E2 Node Version	M		INTEGER	The Near-RT RIC platform maintains the version number for E2 Node Information of each E2 Node. Incremented whenever any RAN Function or Component config changes.

				This is the current version number of E2 Node Information for the E2 Node.
E2 Node State	O		7.3.3.28.1.13	

7.3.3.28.1.13 E2 Node State

IE/Group Name	Presence	Range	IE type and reference	Semantics description
E2 Node State	M		ENUMERATED (CONNECTED, RESET, DISCONNECTED, ...)	

E2 Node State	Meaning
CONNECTED	E2 Node has completed E2 Setup procedure. Near-RT RIC platform has at least one validated E2NodeComponentConfig and E2NodeRanFunction record available for xApps using SDL API. xApp may use E2 related API to request E2 Subscription and E2 Control procedures
RESET	E2 connection to E2 Node has been reset using E2AP Reset procedure. Near-RT RIC platform has at least one validated E2NodeComponentConfig and E2NodeRanFunction record available for xApps using SDL API. Previous E2 Subscriptions have been deleted E2 related API services not available
DISCONNECTED	E2 connection to E2 Node is not available xApp shall delete records for E2 Node

7.3.3.28.2 RAN Analytics Information related IEs

7.3.3.28.2.1 SDL Data Type ID

To construct the *SDL Data Type ID* IE for RAN Analytics Information,

- the *namespace* field is "ORAN";
- the *name* and *version* fields are the *RAI Type ID* and the *Version* defined in Table 6.1-1 of Y1TD [11], respectively.

EXAMPLE: "ORAN:RAN performance analytics:1.0.0"

7.3.3.28.2.2 SDL Data Description For RAI

IE/Group Name	Presence	Range	IE type and reference	Semantics description
Target Entity List		1		Represents the "targetEntities" attribute in clause 6.2.2.5.2.2 and 6.3.2.2.2.3.1 of Y1AP [10].
> Target Entity Item		1..<maxn oofTarget Entities>		
>> Target Entity	M		TargetEntity, clause 6 of Y1TD [11]	
Filter Paramters	O		FilterParameters, clause 6 of Y1TD [11]	Represents the "filterParameters" attribute in clause 6.2.2.5.2.2 and 6.3.2.2.2.3.1 of Y1AP [10].
Expected Validity Period Relative List		0..1		Represents the "expectedValidityPeriods"

				Relative" attribute in clause 6.2.2.5.2.2 of Y1AP [10].
> Expected Validity Period Relative Item		1..<maxnoofValidityPeriods>		
>> Expected Validity Period Relative	M		ValidityPeriodRelative, clause 6.2.2.5.2.6 of Y1AP [10]	
Expected Validity Period List		0..1		Represents the "expectedValidityPeriods" attribute in clause 6.3.2.2.3.1 of Y1AP [10].
> Expected Validity Period Item		1..<maxnoofValidityPeriods>		
>> Expected Validity Period	M		ValidityPeriod, clause 6.2.2.5.2.7 of Y1AP [10]	

Range bound	Explanation
maxnoofTargetEntities	Maximum number of target entities that can be carried in a target entity list. Value is 2048.
maxnoofValidityPeriods	Maximum number of validity periods that can be carried in a validity period list. Value is 512.

7.3.3.28.2.3 SDL Data Unit For RAI

IE Name	Presence	Range	IE type and reference	Semantics description
RAI Report	M		RaiReport, clause 6.2.2.5.2.5 of Y1AP [10]	Represents a single RAI report in clause 6.2 and 6.3 of Y1AP [10].

7.3.3.28.2.4 Trigger Event Definition For RAI

IE Name	Presence	Range	IE type and reference	Semantics description
Notification Event Trigger	M		NotificationTriggerEvent, clause 6 of Y1TD [11]	Represents the "notificationTriggerEvent" attribute in clause 6.2.2.5.2.3 of Y1AP [10].

7.3.4 Message and Information Element Abstract Syntax

7.3.4.1 Solution 1: SCTP with Protocol Buffers

7.3.4.1.1 Usage of SCTP with Protocol Buffers

This solution uses Protocol Buffers as message encoding, and SCTP as the transport protocol.

The PDU on the SCTP connection uses the structure as defined in clause 10.1.8 where its API Payload IE serves as a container for individual API messages in clause 7.3.4.1.2 which can be SdlInformationApiPayload or XappDataSharingApiPayload.

Table 7.3.4.1.1-1 lists the SDL APIs abstract syntax included in this document.

Table 7.3.4.1.1-1: List of SDL APIs abstract syntax

API Name / Contents	Clause	Version	File Name
SDL_Information_API	7.3.4.1.2.1	2.0.0	sdl_information_api_ver_2_0_0.proto
Xapp_Data_Sharing_API	7.3.4.1.2.2	1.0.0	xapp_data_sharing_api_ver_1_0_0.proto
SDL API common information elements	7.3.4.1.3	2.0.0	sdl_apis_common_ies_ver_2_0_0.proto
Near-RT RIC API common information elements	10.2.1	2.0.0	near_rt_ric_apis_common_ies_ver_2_0_0.proto

7.3.4.1.2 API Definitions

7.3.4.1.2.1 SDL_Information_API

```
syntax = "proto3";

package ricapi.sdl_information_api.v2;

import "near_rt_ric_apis_common_ies_ver_2_0_0.proto";
import "sdl_apis_common_ies_ver_2_0_0.proto";

// top-level PDU
message SdlInformationApiPayload {
    oneof procedure_type {
        SdlSubscribeInformationProc sdl_subscribe_information_proc = 1;
        SdlInformationFetchProc sdl_information_fetch_proc = 2;
        SdlInformationPushProc sdl_information_push_proc = 3;
        SdlInformationUpdateNotifyProc sdl_information_update_notify_proc = 4;
    }
}

// ----- procedures of SDL_Information_API -----
message SdlSubscribeInformationProc {
    oneof type_of_message {
        SubscribeInformationRequest subscribe_information_request = 1;
        SubscribeInformationResponse subscribe_information_response = 2;
        SubscribeInformationFailure subscribe_information_failure = 3;
    }
}

message SdlInformationFetchProc {
    oneof type_of_message {
        InformationFetchRequest information_fetch_request = 1;
        InformationFetchResponse information_fetch_response = 2;
        InformationFetchFailure information_fetch_failure = 3;
    }
}

message SdlInformationPushProc {
    oneof type_of_message {
        InformationPush information_push = 1;
    }
}

message SdlInformationUpdateNotifyProc {
    oneof type_of_message {
        InformationUpdateNotify information_update_notify = 1;
    }
}

// ----- messages of SDL_Information_API -----
// ----- messages of SDL Subscribe Information procedure -----
message SubscribeInformationRequest {
    optional ricapi.near_rt_ric_apis_common_ies.v2.ProcedureTransactionId procedure_transaction_id = 1;
    optional ricapi.sdl_apis_common_ies.v2.SdlDataTypeId sdl_data_type_id = 2;
    optional ricapi.sdl_apis_common_ies.v2.DeliveryMethod delivery_method = 3;
    optional ricapi.sdl_apis_common_ies.v2.SdlDataDescription sdl_data_description = 4;
    optional ricapi.near_rt_ric_apis_common_ies.v2.EndpointInformation notification_destination = 5;
```

```

    optional ricapi.sdl_apis_common_ies.v2.NotificationCriteria notification_criteria = 6;
}

message SubscribeInformationResponse {
    optional ricapi.near_rt_ric_apis_common_ies.v2.ProcedureTransactionId procedure_transaction_id = 1;
    optional ricapi.sdl_apis_common_ies.v2.SubscriptionId subscription_id = 2;
}

message SubscribeInformationFailure {
    optional ricapi.near_rt_ric_apis_common_ies.v2.ProcedureTransactionId procedure_transaction_id = 1;
    optional ricapi.sdl_apis_common_ies.v2.SubscriptionRejectCause subscription_reject_cause = 2;
    optional uint32 time_to_wait = 3;
}

// ----- messages of SDL Information Fetch procedure -----
message InformationFetchRequest {
    optional ricapi.near_rt_ric_apis_common_ies.v2.ProcedureTransactionId procedure_transaction_id = 1;
    optional ricapi.sdl_apis_common_ies.v2.SdlDataTypeId sdl_data_type_id = 2;
    optional ricapi.sdl_apis_common_ies.v2.SdlDataDescription sdl_data_description = 3;
}

message InformationFetchResponse {
    optional ricapi.near_rt_ric_apis_common_ies.v2.ProcedureTransactionId procedure_transaction_id = 1;
    optional ricapi.sdl_apis_common_ies.v2.SdlDataTypeId sdl_data_type_id = 2;
    repeated SdlDataBlockItem sdl_data_block = 3;

    message SdlDataBlockItem {
        optional ricapi.sdl_apis_common_ies.v2.SdlDataUnit sdl_data_unit = 1;
    }
}

message InformationFetchFailure {
    optional ricapi.near_rt_ric_apis_common_ies.v2.ProcedureTransactionId procedure_transaction_id = 1;
    optional ricapi.sdl_apis_common_ies.v2.FetchFailureCause fetch_failure_cause = 2;
}

// ----- messages of SDL Information Push procedure -----
message InformationPush {
    optional ricapi.near_rt_ric_apis_common_ies.v2.ProcedureTransactionId procedure_transaction_id = 4;
    optional ricapi.sdl_apis_common_ies.v2.SubscriptionId subscription_id = 1;
    optional ricapi.sdl_apis_common_ies.v2.SdlDataTypeId sdl_data_type_id = 2;
    repeated SdlDataBlockItem sdl_data_block = 3;

    message SdlDataBlockItem {
        optional ricapi.sdl_apis_common_ies.v2.ModificationType modification_type = 1;
        optional ricapi.sdl_apis_common_ies.v2.SdlDataUnit sdl_data_unit = 2;
    }
}

// ----- messages of SDL Information Update Notify procedure -----
message InformationUpdateNotify {
    optional ricapi.near_rt_ric_apis_common_ies.v2.ProcedureTransactionId procedure_transaction_id = 4;
    optional ricapi.sdl_apis_common_ies.v2.SubscriptionId subscription_id = 1;
    optional ricapi.sdl_apis_common_ies.v2.SdlDataTypeId sdl_data_type_id = 2;
    repeated SdlUpdateNotifyBlockItem sdl_update_notify_block = 3;

    message SdlUpdateNotifyBlockItem {
        optional ricapi.sdl_apis_common_ies.v2.ModificationType modification_type = 1;
        optional ricapi.sdl_apis_common_ies.v2.SdlUpdateNotifyUnit sdl_update_notify_unit = 2;
    }
}

```

7.3.4.1.2.2 Xapp_Data_Sharing_API

```

syntax = "proto3";

package ricapi.xapp_data_sharing_api.v1;

import "sdl_apis_common_ies_ver_2_0_0.proto";
import "near_rt_ric_apis_common_ies_ver_2_0_0.proto";

// top-level PDU

```

```

message XappDataSharingApiPayload {
    oneof procedure_type {
        XappSharedDataFetchProc xapp_shared_data_fetch_proc = 1;
        XappSharedDataSubscribeProc xapp_shared_data_subscribe_proc = 2;
        XappSharedDataSubscriptionDeleteProc xapp_shared_data_subscription_delete_proc = 3;
        XappSharedDataPushProc xapp_shared_data_push_proc = 4;
    }
}

// ----- procedures of Xapp_Data_Sharing_API -----
message XappSharedDataFetchProc {
    oneof type_of_message {
        XappSharedDataFetchRequest xapp_shared_data_fetch_request = 1;
        XappSharedDataFetchResponse xapp_shared_data_fetch_response = 2;
        XappSharedDataFetchFailure xapp_shared_data_fetch_failure = 3;
    }
}

message XappSharedDataSubscribeProc {
    oneof type_of_message {
        XappSharedDataSubscriptionRequest xapp_shared_data_subscription_request = 1;
        XappSharedDataSubscriptionResponse xapp_shared_data_subscription_response = 2;
        XappSharedDataSubscriptionFailure xapp_shared_data_subscription_failure = 3;
    }
}

message XappSharedDataSubscriptionDeleteProc {
    oneof type_of_message {
        XappSharedDataSubscriptionDeleteRequest xapp_shared_data_subscription_delete_request = 1;
        XappSharedDataSubscriptionDeleteResponse xapp_shared_data_subscription_delete_response = 2;
        XappSharedDataSubscriptionDeleteFailure xapp_shared_data_subscription_delete_failure = 3;
    }
}

message XappSharedDataPushProc {
    oneof type_of_message {
        XappSharedDataPush xapp_shared_data_push = 1;
    }
}

// ----- messages of Xapp_Data_Sharing_API -----
// ----- messages of Xapp Shared Data Fetch procedure -----
message XappSharedDataFetchRequest {
    optional ricapi.near_rt_ric_apis_common_ies.v2.ProcedureTransactionId procedure_transaction_id = 1;
    optional ricapi.sdl_apis_common_ies.v2.SdlDataTypeId sdl_data_type_id = 2;
    optional ricapi.sdl_apis_common_ies.v2.SdlDataDescription sdl_data_description = 3;
}

message XappSharedDataFetchResponse {
    optional ricapi.near_rt_ric_apis_common_ies.v2.ProcedureTransactionId procedure_transaction_id = 1;
    optional ricapi.sdl_apis_common_ies.v2.SdlDataTypeId sdl_data_type_id = 2;
    repeated SdlDataBlockItem sdl_data_block = 3;

    message SdlDataBlockItem {
        optional ricapi.sdl_apis_common_ies.v2.SdlDataUnit sdl_data_unit = 1;
    }
}

message XappSharedDataFetchFailure {
    optional ricapi.near_rt_ric_apis_common_ies.v2.ProcedureTransactionId procedure_transaction_id = 1;
    optional ricapi.sdl_apis_common_ies.v2.FetchFailureCause fetch_failure_cause = 2;
}

// ----- messages of Xapp Shared Data Subscribe procedure -----
message XappSharedDataSubscriptionRequest {
    optional ricapi.near_rt_ric_apis_common_ies.v2.ProcedureTransactionId procedure_transaction_id = 1;
    optional ricapi.sdl_apis_common_ies.v2.SdlDataTypeId sdl_data_type_id = 2;
    optional ricapi.sdl_apis_common_ies.v2.SdlDataDescription sdl_data_description = 3;
    optional ricapi.sdl_apis_common_ies.v2.DeliveryMethod delivery_method = 4;
    optional ricapi.near_rt_ric_apis_common_ies.v2.EndpointInformation notification_destination = 5;
    optional ricapi.sdl_apis_common_ies.v2.NotificationCriteria notification_criteria = 6;
}

message XappSharedDataSubscriptionResponse {

```

```

    optional ricapi.near_rt_ric_apis_common_ies.v2.ProcedureTransactionId procedure_transaction_id = 1;
    optional ricapi.sdl_apis_common_ies.v2.SubscriptionId subscription_id = 2;
}

message XappSharedDataSubscriptionFailure {
    optional ricapi.near_rt_ric_apis_common_ies.v2.ProcedureTransactionId procedure_transaction_id = 1;
    optional ricapi.sdl_apis_common_ies.v2.SubscriptionRejectCause subscription_reject_cause = 2;
}

// ----- messages of Xapp Shared Data Push procedure -----
message XappSharedDataPush {
    optional ricapi.sdl_apis_common_ies.v2.SubscriptionId subscription_id = 1;
    optional ricapi.sdl_apis_common_ies.v2.SdlDataTypeId sdl_data_type_id = 2;
    repeated SdlDataBlockItem sdl_data_block = 3;

    message SdlDataBlockItem {
        optional ricapi.sdl_apis_common_ies.v2.ModificationType modification_type = 1;
        optional ricapi.sdl_apis_common_ies.v2.SdlDataUnit sdl_data_unit = 2;
    }
}

// ----- messages of Xapp Shared Data Subscription Delete procedure -----
message XappSharedDataSubscriptionDeleteRequest {
    optional ricapi.near_rt_ric_apis_common_ies.v2.ProcedureTransactionId procedure_transaction_id = 1;
    optional ricapi.sdl_apis_common_ies.v2.SubscriptionId subscription_id = 2;
}

message XappSharedDataSubscriptionDeleteResponse {
    optional ricapi.near_rt_ric_apis_common_ies.v2.ProcedureTransactionId procedure_transaction_id = 1;
}

message XappSharedDataSubscriptionDeleteFailure {
    optional ricapi.near_rt_ric_apis_common_ies.v2.ProcedureTransactionId procedure_transaction_id = 1;
    optional ricapi.sdl_apis_common_ies.v2.SubscriptionRejectCause subscription_reject_cause = 2;
}

```

7.3.4.1.3 Information Element Definitions

```

syntax = "proto3";

package ricapi.sdl_apis_common_ies.v2;

import "near_rt_ric_apis_common_ies_ver_2_0_0.proto";

// ----- Information Elements -----

// -----A-----
// From E2AP Clause 9.2.32
message AmfName {
    optional string value = 1;
}

// -----D-----
enum DeliveryMethod {
    DELIVERY_METHOD_UPDATE_NOTIFY = 0;
    DELIVERY_METHOD_PUSH_DELTA = 1;
    DELIVERY_METHOD_PUSH_FULL = 2;
}

// -----E-----
// From E2AP Clause 9.2.27
message E2NodeComponentConfiguration {
    optional bytes e2_node_component_request_part = 1;
    optional bytes e2_node_component_response_part = 2;
}

// From E2AP Clause 9.2.32
message E2NodeComponentId {
    oneof e2_node_component_interface_type {
        NgInterfaceType ng = 1;
        XnInterfaceType xn = 2;
    }
}

```

```

        E1InterfaceType e1 = 3;
        F1InterfaceType f1 = 4;
        W1InterfaceType w1 = 5;
        S1InterfaceType s1 = 6;
        X2InterfaceType x2 = 7;
    }

    message NgInterfaceType {
        optional AmfName amf_name = 1;
    }

    message XnInterfaceType {
        optional GlobalNgRanNodeId global_ng_ran_node_id = 1;
    }

    message E1InterfaceType {
        optional near_rt_ric_apis_common_ies.v2.GnbCuUpId gnb_cu_up_id = 1;
    }

    message F1InterfaceType {
        optional near_rt_ric_apis_common_ies.v2.GnbDuId gnb_du_id = 1;
    }

    message W1InterfaceType {
        optional near_rt_ric_apis_common_ies.v2.NgEnbDuId ng_enb_du_id = 1;
    }

    message S1InterfaceType {
        optional MmeName mme_name = 1;
    }

    message X2InterfaceType {
        optional near_rt_ric_apis_common_ies.v2.GlobalEnbId global_enb_id = 1;
        optional near_rt_ric_apis_common_ies.v2.GlobalEnGnbId global_en_gnb_id = 2;
    }
}

// From E2AP Clause 9.2.26
enum E2NodeComponentType {
    E2_NODE_COMPONENT_TYPE_NG = 0;
    E2_NODE_COMPONENT_TYPE_XN = 1;
    E2_NODE_COMPONENT_TYPE_E1 = 2;
    E2_NODE_COMPONENT_TYPE_F1 = 3;
    E2_NODE_COMPONENT_TYPE_W1 = 4;
    E2_NODE_COMPONENT_TYPE_S1 = 5;
    E2_NODE_COMPONENT_TYPE_X2 = 6;
}

enum E2NodeState {
    E2_NODE_STATE_CONNECTED = 0;
    E2_NODE_STATE_RESET = 1;
    E2_NODE_STATE_DISCONNECTED = 2;
}

// -----F-----
enum FetchFailureCause {
    FETCH_FAILURE_CAUSE_UNSPECIFIED = 0;
    FETCH_FAILURE_CAUSE_INVALID_FILTER = 1;
    FETCH_FAILURE_CAUSE_INFORMATION_TYPE_NOT_SUPPORTED = 2;
    FETCH_FAILURE_CAUSE_NO_INFORMATION_PRESENT = 3;
}

enum FilterCondition {
    FILTER_CONDITION_CONTAINED_IN = 0;
    FILTER_CONDITION_NOT_CONTAINED_IN = 1;
    FILTER_CONDITION_EQUALS = 2;
    FILTER_CONDITION_GREATER_THAN = 3;
    FILTER_CONDITION_LESS_THAN = 4;
}

message FilterConditionValue {
    oneof filter_condition_value_type {

```

```

    E2NodeIdList e2_node_id_list = 1;
    E2NodeComponentTypeList e2_node_component_type_list = 2;
    RanFunctionOidList ran_function_oid_list = 3;
}

message E2NodeIdList {
    repeated E2NodeIdItem list = 1;

    message E2NodeIdItem {
        optional near_rt_ric_apis_common_ies.v2.GlobalE2NodeId e2_node_id = 1;
        optional E2NodeState e2_node_state = 2;
    }
}

message E2NodeComponentTypeList {
    repeated E2NodeComponentTypeItem list = 1;

    message E2NodeComponentTypeItem {
        optional E2NodeComponentType e2_node_component_type = 1;
    }
}

message RanFunctionOidList {
    repeated RanFunctionOidItem list = 1;

    message RanFunctionOidItem {
        optional RanFunctionOid ran_function_oid = 1;
    }
}

message FilterDefinition {
    optional FilterType filter_type = 1;
    optional FilterCondition filter_condition = 2;
    optional FilterConditionValue filter_condition_value = 3;
}

enum FilterType {
    FILTER_TYPE_E2_NODE_ID = 0;
    FILTER_TYPE_COMPONENT_TYPE = 1;
    FILTER_TYPE_SERVICE_MODEL = 2;
}

// -----G-----

// From E2AP Clause 9.2.32
message GlobalNgRanNodeId {
    oneof ng_ran_node_choice {
        near_rt_ric_apis_common_ies.v2.GlobalGnbId global_gnb_id = 1;
        near_rt_ric_apis_common_ies.v2.GlobalNgEnbId global_ng_enb_id = 2;
    }
}

// -----I-----
enum InformationType {
    INFORMATION_TYPE_E2NODELIST = 0;
    INFORMATION_TYPE_E2NODECOMPONENTCONFIG = 1;
    INFORMATION_TYPE_E2NODERANFUNCTION = 2;
}

message InformationUnit {
    oneof information_unit_type {
        E2NodeListUnit e2_node_list_unit = 1;
        E2NodeComponentConfigUnit e2_node_component_config_unit = 2;
        RanFunctionUnit ran_function_unit = 3;
    }

    message E2NodeListUnit {
        optional near_rt_ric_apis_common_ies.v2.GlobalE2NodeId e2_node_id = 1;
        optional E2NodeState e2_node_state = 2;
    }

    message E2NodeComponentConfigUnit {
        optional near_rt_ric_apis_common_ies.v2.GlobalE2NodeId e2_node_id = 1;

```



```

    repeated E2NodeComponentConfigItem e2_node_component_config_list = 2;

    message E2NodeComponentConfigItem {
        optional E2NodeComponentType e2_node_component_type = 1;
        optional E2NodeComponentId e2_node_component_id = 2;
        optional E2NodeComponentConfiguration e2_node_component_configuration = 3;
        optional uint32 last_modified_e2_node_version = 4;
    }
}

message RanFunctionUnit {
    optional near_rt_ric_apis_common_ies.v2.GlobalE2NodeId e2_node_id = 1;
    repeated RanFunctionItem ran_function_list = 2;

    message RanFunctionItem {
        optional near_rt_ric_apis_common_ies.v2.RanFunctionId ran_function_id = 1;
        optional RanFunctionDefinition ran_function_definition = 2;
        optional RanFunctionOid ran_function_oid = 3;
        optional uint32 last_modified_e2_node_version = 4;
    }
}

// -----M-----
enum ModificationType {
    TYPE_ADDED = 0;
    TYPE_MODIFIED = 1;
    TYPE_REMOVED = 2;
}

// From E2AP Clause 9.2.32
message MmeName {
    optional string value = 1;
}

// -----N-----
message NotificationCriteria {
    optional NotificationMethod notification_method = 1;
    optional uint32 notification_period = 2;
    optional TriggerEventDefinition trigger_event_definition = 3;

    enum NotificationMethod {
        NOTIFICATION_METHOD_PERIODIC = 0;
        NOTIFICATION_METHOD_EVENT_TRIGGERED = 1;
    }
}

message NotificationUnit {
    optional near_rt_ric_apis_common_ies.v2.GlobalE2NodeId e2_node_id = 1;
    optional uint32 current_e2_node_version = 2;
    optional E2NodeState e2_node_state = 3;
}

// -----R-----
// From E2AP Clause 9.2.23
message RanFunctionDefinition {
    optional bytes value = 1;
}

// From E2AP Clause 9.2.31
message RanFunctionOid {
    optional string value = 1;
}

// -----S-----
message SdlDataDescription {
    oneof data_category {
        SdlDataDescriptionForE2NodeInformation sdl_data_description_for_e2_node_information = 1;
        SdlDataDescriptionForRai sdl_data_description_for_rai = 2;
    }
}

message SdlDataDescriptionForE2NodeInformation {

```

```

    optional InformationType information_type = 1;
    repeated FilterItem filter_list = 2;

    message FilterItem {
        optional FilterDefinition filter_definition = 1;
    }
}

message SdlDataDescriptionForRai {
    repeated TargetEntityItem target_entity_list = 1;
    optional bytes filter_parameters = 2;
    repeated ExpectedValidityPeriodRelativeItem expected_validity_period_relative_list = 3;
    repeated ExpectedValidityPeriodItem expected_validity_period_list = 4;

    message TargetEntityItem {
        optional bytes target_entity = 1;
    }

    message ExpectedValidityPeriodRelativeItem {
        optional bytes expected_valitivity_period_relative = 1;
    }

    message ExpectedValidityPeriodItem {
        optional bytes expected_valitivity_period = 1;
    }
}

message SdlDataTypeId {
    optional string value = 1;
}

message SdlDataUnit {
    oneof data_category {
        SdlDataUnitForE2NodeInformation sdl_data_unit_for_e2_node_information = 1;
        SdlDataUnitForRai sdl_data_unit_for_rai = 2;
    }
}

message SdlDataUnitForE2NodeInformation {
    optional InformationUnit information_unit = 1;
}

message SdlDataUnitForRai {
    optional bytes rai_report = 1;
}

message SdlUpdateNotifyUnit {
    oneof data_category {
        SdlUpdateNofityUnitForE2NodeInformation sdl_update_notify_unit_for_e2_node_information = 1;
    }
}

message SdlUpdateNofityUnitForE2NodeInformation {
    optional NotificationUnit notification_unit = 1;
}

message SubscriptionId {
    optional string subscription_id = 1;
}

enum SubscriptionRejectCause {
    SUBSCRIPTION_REJECT_CAUSE_UNSPECIFIED = 0;
    SUBSCRIPTION_REJECT_CAUSE_SDL_REACHED_CAPACITY = 1;
    SUBSCRIPTION_REJECT_CAUSE_INFORMATION_TYPE_NOT_SUPPORTED = 2;
    SUBSCRIPTION_REJECT_CAUSE_INVALID_FILTER = 3;
}

// -----T-----

message TimeToWait {
    optional uint32 value = 1;
}

```

```

message TriggerEventDefinition {
    oneof data_category {
        TriggerEventDefinitionForE2NodeInformation trigger_event_definition_for_e2_node_information = 1;
        TriggerEventDefinitionForRai trigger_event_definition_for_rai = 2;
    }
}

message TriggerEventDefinitionForE2NodeInformation {
    optional EnumValue notify_on_data_change = 1;

    enum EnumValue {
        ENUM_VALUE_TRUE = 0;
    }
}

message TriggerEventDefinitionForRai {
    optional bytes notification_event_trigger = 1;
}

```

7.3.4.2 Solution 2: gRPC with Protocol Buffers

7.3.4.2.1 Usage of gRPC with Protocol Buffers

This solution uses Protocol Buffers as message encoding, also as the Interface Definition Language (IDL) for gRPC [5].

Table 7.3.4.2.1-1 lists the mapping of enablement APIs and corresponding gRPC services.

Table 7.3.4.2.1-1: Mapping of SDL APIs and gRPC services

API Name	gRPC service(s)
SDL_Information_API	Sdl_Information_API, SDL_Information_API_Callback
Xapp_Data_Sharing_API	Xapp_Data_Sharing_API, Xapp_Data_Sharing_API_Callback

A procedure of an SDL API is mapped to a *gRPC method*.

Table 7.3.4.1.1-1 lists the SDL APIs abstract syntax included in this document.

Table 7.3.4.2.1-1: List of SDL APIs abstract syntax

API Name / Contents	Clause	Version	File Name
SDL_Information_API	7.3.4.2.2.1	1.0.0	sdl_information_api_grpc_ver_1_0_0.proto
Xapp_Data_Sharing_API	7.3.4.2.2.2	1.0.0	xapp_data_sharing_api_grpc_ver_1_0_0.proto
SDL API common information elements	7.3.4.1.3	2.0.0	sdl_apis_common_ies_ver_2_0_0.proto
Near-RT RIC API common information elements	10.2.1	2.0.0	near_rt_ric_apis_common_ies_ver_2_0_0.proto

7.3.4.2.2 API Definitions

7.3.4.2.2.1 SDL_Information_API

```

syntax = "proto3";

package ricapi.sdl_information_api_grpc.v1;

import "sdl_information_api_ver_2_0_0.proto";

// gRPC service and rpc definition
service Sdl_Information_API {
    rpc SdlSubscribeInformationProc(SdlSubscribeInformationInitMsg) returns
(SdlSubscribeInformationOutMsg) {};
    rpc SdlInformationFetchProc(SdlInformationFetchInitMsg) returns (SdlInformationFetchOutMsg) {};
}

```

```

}

message SdlSubscribeInformationInitMsg {
    optional ricapi.sdl_information_api.v2.SubscribeInformationRequest subscribe_information_request = 1;
}

message SdlSubscribeInformationOutMsg {
    oneof type_of_message {
        ricapi.sdl_information_api.v2.SubscribeInformationResponse subscribe_information_response = 1;
        ricapi.sdl_information_api.v2.SubscribeInformationFailure subscribe_information_failure = 2;
    }
}

message SdlInformationFetchInitMsg {
    optional ricapi.sdl_information_api.v2.InformationFetchRequest information_fetch_request = 1;
}

message SdlInformationFetchOutMsg {
    oneof type_of_message {
        ricapi.sdl_information_api.v2.InformationFetchResponse information_fetch_response = 1;
        ricapi.sdl_information_api.v2.InformationFetchFailure information_fetch_failure = 2;
    }
}

service Sdl_Information_API_Callback {
    rpc SdlInformationPushProc(SdlInformationPushInitMsg) returns (SdlInformationPushOutMsg) {};
    rpc SdlInformationUpdateNotifyProc(SdlInformationUpdateNotifyInitMsg) returns
(SdlInformationUpdateNoiftyNotifyInitMsg) {};
}

message SdlInformationPushInitMsg {
    optional ricapi.sdl_information_api.v2.InformationPush information_push = 1;
}

message SdlInformationPushOutMsg {
}

message SdlInformationUpdateNotifyInitMsg {
    optional ricapi.sdl_information_api.v2.InformationUpdateNotify information_update_notify = 1;
}

message SdlInformationUpdateNoiftyNotifyInitMsg {
}

```

7.3.4.2.2.2 Xapp_Data_Sharing_API

```

syntax = "proto3";

package ricapi.xapp_data_sharing_api_grpc.v1;

import "xapp_data_sharing_api_ver_1_0_0.proto";

// gRPC service and rpc definition
service Xapp_Data_Sharing_API {
    rpc XappSharedDataFetchProc(XappSharedDataFetchInitMsg) returns (XappSharedDataFetchOutMsg) {};
    rpc XappSharedDataSubscribeProc(XappSharedDataSubscribeInitMsg) returns
(XappSharedDataSubscribeOutMsg) {};
    rpc XappSharedDataSubscriptionDeleteProc(XappSharedDataSubscriptionDeleteInitMsg) returns
(XappSharedDataSubscriptionDeleteOutMsg) {};
}

message XappSharedDataFetchInitMsg {
    optional ricapi.xapp_data_sharing_api.v1.XappSharedDataFetchRequest xapp_shared_data_fetch_request =
1;
}

message XappSharedDataFetchOutMsg {
    oneof type_of_message {
        ricapi.xapp_data_sharing_api.v1.XappSharedDataFetchResponse xapp_shared_data_fetch_response = 1;
        ricapi.xapp_data_sharing_api.v1.XappSharedDataFetchFailure xapp_shared_data_fetch_failure = 2;
    }
}

message XappSharedDataSubscribeInitMsg {
    optional ricapi.xapp_data_sharing_api.v1.XappSharedDataSubscriptionRequest
xapp_shared_data_subscription_request = 1;
}

```

```

}

message XappSharedDataSubscribeOutMsg {
    oneof type_of_message {
        ricapi.xapp_data_sharing_api.v1.XappSharedDataSubscriptionResponse
    }
    xapp_shared_data_subscription_response = 1;
    ricapi.xapp_data_sharing_api.v1.XappSharedDataSubscriptionFailure
    xapp_shared_data_subscription_failure = 2;
}

message XappSharedDataSubscriptionDeleteInitMsg {
    optional ricapi.xapp_data_sharing_api.v1.XappSharedDataSubscriptionDeleteRequest
    xapp_shared_data_subscription_delete_request = 1;
}

message XappSharedDataSubscriptionDeleteOutMsg {
    oneof type_of_message {
        ricapi.xapp_data_sharing_api.v1.XappSharedDataSubscriptionDeleteResponse
    }
    xapp_shared_data_subscription_delete_response = 2;
    ricapi.xapp_data_sharing_api.v1.XappSharedDataSubscriptionDeleteFailure
    xapp_shared_data_subscription_delete_failure = 3;
}

service Xapp_Data_Sharing_API_Callback {
    rpc XappSharedDataPushProc(XappSharedDataPushInitMsg) returns (XappSharedDataPushOutMsg) {};
}

message XappSharedDataPushInitMsg {
    optional ricapi.xapp_data_sharing_api.v1.XappSharedDataPush xapp_shared_data_push = 1;
}

message XappSharedDataPushOutMsg {
}

```

7.3.5 Message Transfer Syntax

7.3.5.1 Solution 1: SCTP with Protocol Buffers

The message transfer syntax for Solution 1 is serialized Protocol Buffers [4].

7.3.5.2 Solution 2: gRPC with Protocol Buffers

The message transfer syntax for Solution 2 is serialized Protocol Buffers [4].

8 Management API

8.1 Overview

8.1.1 Protocol Stack

In this version of specification, the default protocol stack for the Management APIs is shown on Figure 8.1.1-1.

Note: The security protocols are specified in [6].

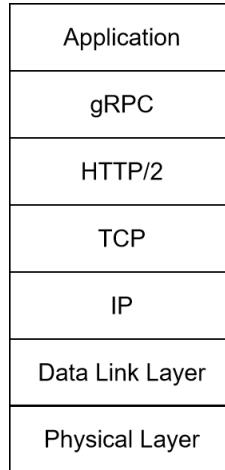


Figure 8.1.1-1: Protocol stack for Management APIs

The Management APIs use gRPC [5] with Protocol Buffers [4] as the application layer serialization protocol.

8.1.2 List of Management APIs, procedures and messages

The Management API procedures introduced in clause 9.4 of [2] are categorized under the Management APIs. These APIs include:

- xApp Registration API: enabling registration of xApps with the Near-RT RIC platform.

These APIs are summarized in Table 8.1.2-1.

Table 8.1.2-1: List of Management APIs

API Name	Description
Mgmt_xApp_Registration_API	Near-RT RIC platform service for xApp registration

The table 8.1.2-2 lists the API procedures for each Management API.

Table 8.1.2-2: List of Management API procedures

API Name	API procedure	Communication type
Mgmt_xApp_Registration_API	xApp Registration	Request/Response

The table 8.1.2-3 lists the messages for each management API procedure.

Table 8.1.2-3: List of Management API messages

API Procedure	Initiated by	Initiating Message	Outcome Message	
			Successful Outcome	Unsuccessful Outcome
xApp Registration	xApp	XAPP REGISTRATION REQUEST	XAPP REGISTRATION SUCCESS	XAPP REGISTRATION FAILURE

8.2 Management API Procedures

8.2.1 xApp Registration Procedure

8.2.1.1 General

This procedure is used by an xApp who wants to register in Near-RT RIC platform.

8.2.1.2 Successful Operation

```
@startuml
Skin rose
skinparam ParticipantPadding 50
skinparam BoxPadding 10
skinparam lifelineStrategy solid
participant "xApp" as app
participant "Near-RT RIC Platform" as near
app -> near: xAPP REGISTRATION (Request)
near -> app: xAPP REGISTRATION (Success)
@enduml
```

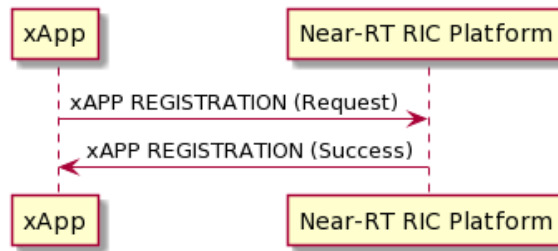


Figure 8.2.1.2-1: xApp Registration procedure, successful operation

The xApp initiates the procedure by sending an xApp Registration (Request) message to the Near-RT-RIC platform. The message contains a unique *Procedure Transaction ID* IE assigned by the initiating xApp, the optional *xApp Description* IE, and the *Required APIs and Data Types* IE that specifies the APIs and data types needed for normal operation of the xApp.

At reception of the xApp Registration (Request) message, the Near-RT-RIC platform shall:

- Check xApp's compatibility with the Near-RT RIC platform, using information in the *Required APIs and Data Types* IE.
- Assign an xApp ID and create the profile for the xApp.

If the registration request is successfully fulfilled, the Near-RT RIC platform shall send an xApp Registration (Success) message back to the xApp.

In the xApp Registration (Success) message, the Near-RT RIC platform shall include the associated *Procedure Transaction ID* IE and the *xApp ID* IE.

8.2.1.3 Unsuccessful Operation

```
@startuml
Skin rose
skinparam ParticipantPadding 50
skinparam BoxPadding 10
skinparam lifelineStrategy solid
participant "xApp" as app
participant "Near-RT RIC Platform" as near
app -> near: xAPP REGISTRATION (Request)
near -> app: xAPP REGISTRATION (Failure)
@enduml
```

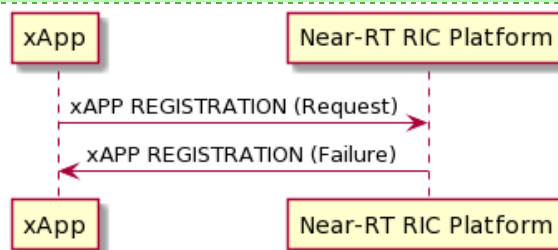


Figure 8.2.1.3-1: xApp Registration procedure, unsuccessful operation

If the registration request fails, the Near-RT RIC platform shall send an xApp Registration (Failure) message back to the xApp. The message shall include the associated *Procedure Transaction ID* IE replicated from the corresponding request and appropriate cause.

8.2.1.4 Abnormal Conditions

Not applicable.

8.3 Elements for Management API Communication

8.3.1 General

8.3.2 Message Functional Definition and Content

8.3.2.1 xAPP REGISTRATION REQUEST

This message is sent by an xApp to the Near-RT RIC platform for xApp Registration.

Direction: xApp -> Near-RT RIC platform

IE/Group Name	Presence	Range	IE type and reference	Semantics description	Criticality	Assigned Criticality
Message Type	M		8.3.3.1		YES	Reject
Procedure Transaction ID	M		10.1.1	Value assigned by the initiating xApp.	YES	Reject
xApp Description	O		8.3.3.2		YES	Ignore
Required APIs and Data Types	M				YES	Reject
> List of Required APIs		0.. <maxnoofAPIs>		The API names and versions required by the xApp	-	-
>> API Name	M		10.1.3		-	-
>> API Version	M		10.1.4		-	-
> List of A1 Policy Types		0.. <maxnoofA1PolicyTypes>		The A1 Policy Types required by the xApp	-	-
>> A1 Policy Type ID	M		8.3.3.3		-	-
> List of E2 Service Models		0.. <maxnoofE2SMs>		The E2 Service Models required by the xApp	-	-
>> E2 Service Model Name	M		8.3.3.4		-	-
>> E2 Service Model Version	M		8.3.3.5		-	-

Range bound	Explanation
maxnoofAPIs	Maximum no. of APIs supported by the xApp. Value is 1024.
maxnoofA1PolicyTypes	Maximum no. of A1 Policy Types supported by the xApp. Value is 1024.
maxnoofE2SMs	Maximum no. of E2SMs supported by the xApp. Value is 1024.

8.3.2.2 xAPP REGISTRATION SUCCESS

This message is sent by the Near-RT RIC platform after the xApp's registration request is successfully accepted.

Direction: Near-RT RIC platform -> xApp

IE/Group Name	Presence	Range	IE type and reference	Semantics description	Criticality	Assigned Criticality
Message Type	M		8.3.3.1		YES	Reject
Procedure Transaction ID	M		10.1.1	Value replicated from corresponding request.	YES	Reject
xApp ID	M		10.1.2		YES	Reject

8.3.2.3 xAPP REGISTRATION FAILURE

This message is sent by the Near-RT RIC platform after the xApp's registration request is failed.

Direction: Near-RT RIC platform -> xApp

IE/Group Name	Presence	Range	IE type and reference	Semantics description	Criticality	Assigned Criticality
Message Type	M		8.3.3.1		YES	Reject
Procedure Transaction ID	M		10.1.1	Value replicated from corresponding request.	YES	Reject
Cause	M		8.3.3.6		YES	Reject

8.3.2.4 xAPP CONFIGURATION REQUEST

8.3.2.5 xAPP CONFIGURATION RESPONSE

8.3.2.6 xAPP CONFIGURATION FAILURE

8.3.3 Information Element Definitions

8.3.3.1 Message Type

This IE uniquely identifies the message being sent. It is mandatory for all messages.

IE/Group Name	Presence	Range	IE type and reference	Semantics description
Message Type				
>Procedure Type	M		CHOICE (xApp Registration, ...)	
>Type of Message	M		CHOICE (Request, Success, Failure, ...)	Refer to Table 8.1.2-3 for applicable choices for each procedure type

8.3.3.2 xApp Description

This IE provides generic information related to the xApp such as its name, vendor and purpose.

IE/Group Name	Presence	Range	IE type and reference	Semantics description
xApp Description	M		STRING	

8.3.3.3 A1 Policy Type ID

Defined in 10.1.5.

8.3.3.4 E2 Service Model Name

This IE specifies the name of E2 Service Model supported by the xApp.

IE/Group Name	Presence	Range	IE type and reference	Semantics description
E2 Service Model Name	M		STRING	The E2SM short name as in Table 5-1 of [8]

8.3.3.5 E2 Service Model Version

This IE is used to indicate the version of E2 Service Model supported by the xApp.

IE/Group Name	Presence	Range	IE type and reference	Semantics description
E2 Service Model Version	M		STRING	

8.3.3.6 Cause

This IE is used to indicate the reason for a particular event for the Management APIs.

IE/Group Name	Presence	Range	IE type and reference	Semantics description
Cause	M		ENUMERATED (Unspecified, API Not Supported, A1 Policy Type Not Supported, E2 Service Model Not Supported, ...)	

8.3.4 Message and Information Element Abstract Syntax

8.3.4.1 Solution 1: gRPC with Protocol Buffers

8.3.4.1.1 Usage of gRPC with Protocol Buffers

This solution uses Protocol Buffers as message encoding, and also as the Interface Definition Language (IDL) for gRPC [5].

Table 8.3.4.1.1-1 lists the mapping of management APIs and corresponding gRPC services.

Table 8.3.4.1.1-1: Mapping of management APIs and gRPC services

API Name	gRPC service(s)
Mgmt_Xapp_Registration_API	Mgmt_Xapp_Registration_API

A procedure of an Management API is mapped to a *gRPC method*.

Table 8.3.4.1.1-2 lists the Management APIs abstract syntax included in this document.

Table 8.3.4.1.1-2: List of Management APIs abstract syntax

API Name / Contents	Clause	Version	File Name
Mgmt_Xapp_Registration_API	8.3.4.1.2.1	1.0.0	mgmt_xapp_registration_api_ver_1_0_0.proto
Management API common information elements	8.3.4.1.3	1.0.0	mgmt_apis_common_ies_ver_1_0_0.proto
Near-RT RIC API common information elements	10.2.1	2.0.0	near_rt_ric_apis_common_ies_ver_2_0_0.proto

8.3.4.1.2 API Definitions

8.3.4.1.2.1 Mgmt_Xapp_Registration_API

```
syntax = "proto3";

package ricapi.mgmt_xapp_registration_api.v1;

import "near_rt_ric_apis_common_ies_ver_2_0_0.proto";
import "mgmt_apis_common_ies_ver_1_0_0.proto";

// gRPC service and rpc definition
service mgmt_Xapp_Registration_API {
    rpc XappRegistrationProc(XappRegistrationInitMsg) returns (XappRegistrationOutMsg) {};
}

message XappRegistrationInitMsg {
    optional XappRegistrationRequest xapp_registration_request = 1;
}

message XappRegistrationOutMsg {
    oneof type_of_message {
        XappRegistrationSuccess xapp_registration_success = 1;
        XappRegistrationFailure xapp_registration_failure = 2;
    }
}

// ----- messages of Mgmt_Xapp_Registration_API -----
// ----- messages of xApp Registration procedure -----

message XappRegistrationRequest {
    optional near_rt_ric_apis_common_ies.v2.ProcedureTransactionId procedure_transaction_id = 1;
    optional mgmt_apis_common_ies.v1.XappDescription xapp_description = 2;
    optional RequiredApisAndDataTypes required_apis_and_data_types = 3;

    message RequiredApisAndDataTypes {
        repeated RequiredApiItem list_of_required_apis = 1;
        repeated AlPolicyTypeItem list_of_al_policy_types = 2;
        repeated E2ServiceModelItem list_of_e2_service_models = 3;

        message RequiredApiItem {
            optional near_rt_ric_apis_common_ies.v2.ApiName api_name = 1;
            optional near_rt_ric_apis_common_ies.v2.ApiVersion api_version = 2;
        }

        message AlPolicyTypeItem {
            optional near_rt_ric_apis_common_ies.v2.AlPolicyTypeId al_policy_type_id = 1;
        }

        message E2ServiceModelItem {
            optional mgmt_apis_common_ies.v1.E2ServiceModelName e2_service_model_name = 1;
            optional mgmt_apis_common_ies.v1.E2ServiceModelVersion e2_service_model_version = 2;
        }
    }
}
```

```
message XappRegistrationSuccess {
    optional near_rt_ric_apis_common_ies.v2.ProcedureTransactionId procedure_transaction_id = 1;
    optional near_rt_ric_apis_common_ies.v2.XappId xapp_id = 2;
}

message XappRegistrationFailure {
    optional near_rt_ric_apis_common_ies.v2.ProcedureTransactionId procedure_transaction_id = 1;
    optional mgmt_apis_common_ies.v1.Cause cause = 2;
}
```

8.3.4.1.3 Information Element Definitions

```
syntax = "proto3";

package ricapi.mgmt_apis_common_ies.v1;

import "near_rt_ric_apis_common_ies_ver_2_0_0.proto";

// ----- Information Elements -----
// -----C-----
enum Cause {
    UNSPECIFIED = 0;
    API_NOT_SUPPORTED = 1;
    A1_POLICY_TYPE_NOT_SUPPORTED = 2;
    E2_SERVICE_MODEL_NOT_SUPPORTED = 3;
}

// -----E-----
message E2ServiceModelName {
    optional string value = 1;
}

message E2ServiceModelVersion {
    optional string value = 1;
}

// -----X-----
message XappDescription {
    optional string value = 1;
}
```

8.3.5 Message Transfer Syntax

8.3.5.1 Solution 1: gRPC with Protocol Buffers

The message transfer syntax for Solution 1 is serialized Protocol Buffers [4].

9 Enablement API

9.1 Overview

9.1.1 Protocol Stack

In this version of specification, the default protocol stack for the Enablement APIs is shown on Figure 9.1.1-1.

Note: The security protocols are specified in [6].

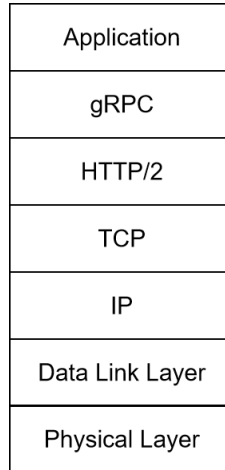


Figure 9.1.1-1: Protocol stack for Enablement APIs

The Enablement APIs use gRPC [5] with Protocol Buffers [4] as the application layer serialization protocol.

9.1.2 List of Enablement APIs, procedures and messages

The enablement API procedures introduced in clause 9.5 of [2] are categorized under the enablement APIs. These APIs include:

- Discovery API: enabling discovery of the Near-RT RIC APIs by xApps.
- Events API: enabling subscription/un-subscription to API-related events and event notifications.
- Registration API: enabling registration of the Near-RT RIC APIs by xApps.

These APIs are summarized in Table 9.1.2-1.

Table 9.1.2-1: List of Enablement APIs

API Name	Description
Enabl_Discovery_API	Near-RT RIC platform service for API discovery
Enabl_Events_API	Near-RT RIC platform service for API related events
Enabl_Registration_API	Near-RT RIC platform service for API registration

The table 9.1.2-2 lists the API procedures for each enablement API.

Table 9.1.2-2: List of enablement API procedures

API Name	API procedure	Communication type
Enabl_Discovery_API	API Discovery	Request/Response
Enabl_Events_API	API Event Subscription	Request/Response
	API Event Subscription Delete	Request/Response
	API Event Notification	Notify
Enabl_Registration_API	API Registration	Request/Response
	API Registration Update	Request/Response
	API Deregistration	Request/Response

The table 9.1.2-3 lists the messages for each enablement API procedure.

Table 9.1.2-3: List of enablement API messages

API Procedure	Initiated by	Initiating Message	Outcome Message	
			Successful Outcome	Unsuccessful Outcome

API Discovery	xApp	API DISCOVERY REQUEST	API DISCOVERY RESPONSE	API DISCOVERY FAILURE
API Event Subscription	xApp	API EVENT SUBSCRIPTION REQUEST	API EVENT SUBSCRIPTION RESPONSE	API EVENT SUBSCRIPTION FAILURE
API Event Subscription Delete	xApp	API EVENT SUBSCRIPTION DELETE REQUEST	API EVENT SUBSCRIPTION DELETE RESPONSE	API EVENT SUBSCRIPTION DELETE FAILURE
API Event Notification	Near-RT RIC platform	API EVENT NOTIFICATION	API EVENT NOTIFICATION ACK	-
API Registration	xApp	API REGISTRATION REQUEST	API REGISTRATION RESPONSE	API REGISTRATION FAILURE
API Registration Update	xApp	API REGISTRATION UPDATE REQUEST	API REGISTRATION UPDATE RESPONSE	API REGISTRATION UPDATE FAILURE
API Deregistration	xApp	API DEREGISTRATION REQUEST	API DEREGISTRATION RESPONSE	API DEREGISTRATION FAILURE

9.2 Enablement API Procedures

9.2.1 Enabl_Discovery_API Procedures

9.2.1.1 General

The Enabl_Discovery_API allows an xApp to discover service APIs available at the Near-RT RIC platform.

The API procedure defined for Enabl_Discovery_API is shown in table 9.2.1.1-1.

Table 9.2.1.1-1: Procedure of the Enabl_Discovery_API

API Procedure	Description	Initiated by
API Discovery	This procedure is used by an xApp to discover Near-RT RIC APIs available at the Near-RT RIC platform.	xApp

9.2.1.2 API Discovery procedure

9.2.1.2.1 Successful Operation

```

@startuml
skin rose
skinparam ParticipantPadding 50
skinparam BoxPadding 10
skinparam defaultFontSize 12
skinparam lifelineStrategy solid

Participant xApp as "xApp"
Participant RICP as "Near-RT RIC Platform"

xApp -> RICP : 1. API DISCOVERY REQUEST
RICP -> xApp : 2. API DISCOVERY RESPONSE

@enduml

```

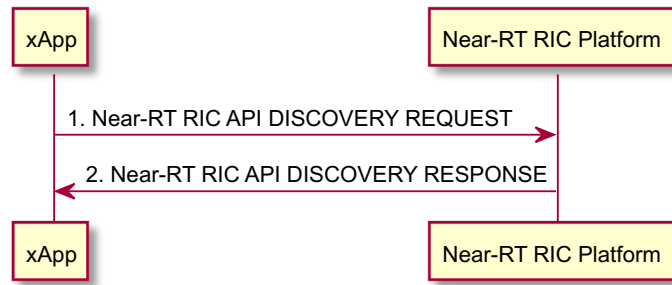


Figure 9.2.1.2.1-1: API Discovery procedure, successful operation

To discover Near-RT RIC APIs available at the API Enablement function, the xApp shall send an API DISCOVERY REQUEST message with the *xApp ID* IE and the *Query Parameters* IE to the Near-RT RIC platform.

Upon receiving the message, the Near-RT RIC platform shall:

1. verify the identity of the xApp and check if it is authorized to discover the Near-RT RIC APIs;
2. if the xApp is authorized to discover the Near-RT RIC APIs, the Near-RT RIC platform shall:
 - a. search the API registry for APIs matching the query criteria in the *Query Parameters* IE;
 - b. apply the discovery policy, if any, on the search results and filter the search results to obtain the list of service API description;
 - c. return the filtered search results in the API DISCOVERY RESPONSE message.

9.2.1.2.2 Unsuccessful Operation

```

@startuml
skin rose
skinparam ParticipantPadding 50
skinparam BoxPadding 10
skinparam defaultFontSize 12
skinparam lifelineStrategy solid

Participant xApp as "xApp"
Participant RICP as "Near-RT RIC Platform"

xApp ->> RICP : 1. API DISCOVERY REQUEST
RICP -->>xApp : 2. API DISCOVERY FAILURE

@enduml
  
```

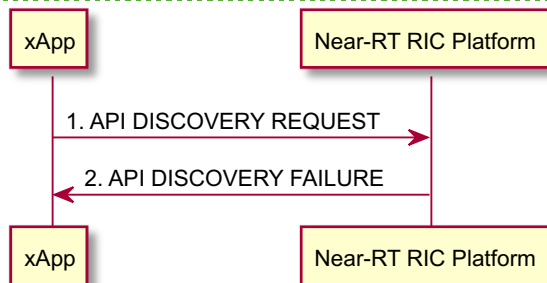


Figure 9.2.1.2.2-1: API Discovery procedure, unsuccessful operation

If the Near-RT RIC platform rejects the request it shall respond with the API DISCOVERY FAILURE message with a proper cause. If the message contains the *Time to Wait* IE the xApp shall wait at least the indicated time before reinitiating the request to the Near-RT RIC platform.

9.2.1.2.3 Abnormal Conditions

Not applicable.

9.2.2 Enabl_Events_API Procedures

9.2.2.1 General

The Enabl_Events_API allows an xApp to subscribe to and unsubscribe from events related to Near-RT RIC APIs and to receive notifications of the events.

Such events includes:

- Availability of Near-RT RIC API: event related to availability of Near-RT RIC APIs (e.g. active, inactive)
- Update of Near-RT RIC API: event related to change in Near-RT RIC API information

The API procedures defined for the Enabl_Events_API are shown in table 9.2.2.1-1.

Table 9.2.2.1-1: Operations of the Enabl_Events_API

API Procedure	Description	Initiated by
API Event Subscription	This procedure is used by xApp to subscribe for receiving Near-RT RIC API related events.	xApp
API Event Subscription Delete	This procedure is used by xApp to unsubscribe from receiving Near-RT RIC API related events.	xApp
API Event Notification	This procedure is used by Near-RT RIC to send a notification of Near-RT RIC API related event to an xApp.	Near-RT RIC platform

9.2.2.2 API Event Subscription procedure

9.2.2.2.1 Successful Operation

```

@startuml
skin rose
skinparam ParticipantPadding 50
skinparam BoxPadding 10
skinparam defaultFontSize 12
skinparam lifelineStrategy solid

Participant xApp as "xApp"
Participant RICP as "Near-RT RIC Platform"

xApp -> RICP : 1. API EVENT SUBSCRIPTION REQUEST
RICP -> xApp : 2. API EVENT SUBSCRIPTION RESPONSE

@enduml

```

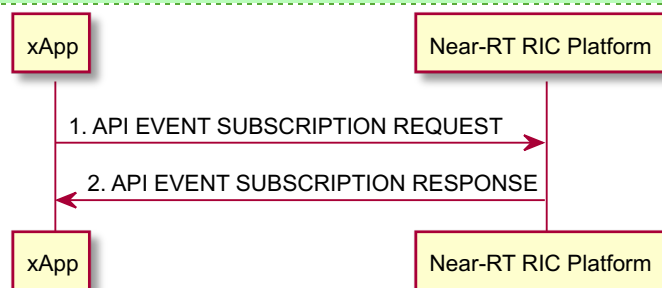


Figure 9.2.2.2.1-1: API Event Subscription procedure, successful operation

To subscribe to Near-RT RIC API related events, the xApp shall send an API EVENT SUBSCRIPTION REQUEST message to the Near-RT RIC platform. The message shall include the *xApp ID* IE, the *List of Events* IE, the *Event Notification Requirements* IE and the *Notification Destination* IE.

The xApp may also include additional event filter(s) in the *List of Event Filters* IE. Such event filters include:

- API ID(s) in the *API ID Filter* IE, if the event type is NEAR_RT_RIC_API_AVAILABLE, NEAR_RT_RIC_API_UNAVAILABLE or NEAR_RT_RIC_API_UPDATE;

Upon receiving the message, the API Enablement function shall:

1. verify the identity of the xApp and check if the xApp is authorized to subscribe to the Near-RT RIC API related events specified in the request message;
2. if the xApp is authorized to subscribe to the Near-RT RIC API related events, the Near-RT RIC platform shall prepare necessary resource for the subscription and associated notifications, according to the contents in the *List of Events* IE and the *Event Notification Requirements* IE, and send an API EVENT SUBSCRIPTION RESPONSE message back to the xApp. The response message shall include the assigned subscription ID in the *Subscription ID* IE.

9.2.2.2.2 Unsuccessful Operation

```
@startuml
skin rose
skinparam ParticipantPadding 50
skinparam BoxPadding 10
skinparam defaultFontSize 12
skinparam lifelineStrategy solid

Participant xApp as "xApp"
Participant RICP as "Near-RT RIC Platform"

xApp -> RICP : 1. API EVENT SUBSCRIPTION REQUEST
RICP -> xApp : 2. API EVENT SUBSCRIPTION FAILURE

@enduml
```

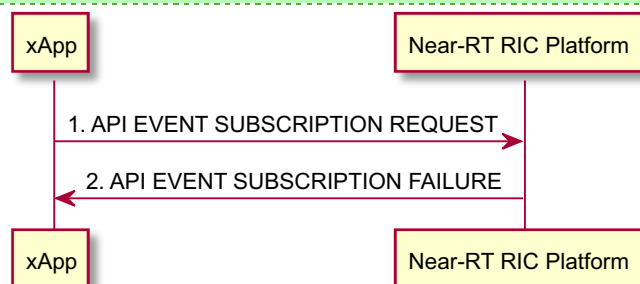


Figure 9.2.2.2.2-1: API Event Subscription procedure, unsuccessful operation

If the Near-RT RIC platform rejects the request it shall respond with the API EVENT SUBSCRIPTION FAILURE message with proper cause. If the message contains *Time to Wait* IE the xApp shall wait at least the indicated time before reinitiating the request to the Near-RT RIC platform.

9.2.2.2.3 Abnormal conditions

If the Near-RT RIC platform receives an API EVENT SUBSCRIPTION REQUEST message containing *Event Type* IE that is not supported by the Near-RT RIC platform, or the optional filter definition is invalid, the Near-RT RIC platform shall send the API EVENT SUBSCRIPTION FAILURE message to the xApp with an appropriate cause value.

9.2.2.3 API Event Subscription Delete procedure

9.2.2.3.1 Successful Operation

```
@startuml
skin rose
skinparam ParticipantPadding 50
skinparam BoxPadding 10
skinparam defaultFontSize 12
skinparam lifelineStrategy solid
```

```

Participant xApp as "xApp"
Participant RICP as "Near-RT RIC Platform"

xApp -> RICP : 1. API EVENT SUBSCRIPTION DELETE REQUEST
RICP -> xApp : 2. API EVENT SUBSCRIPTION DELETE RESPONSE
@enduml

```

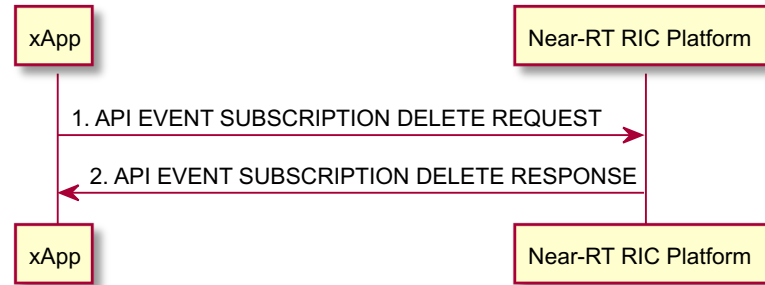


Figure 9.2.2.3.1-1: API Event Subscription Delete procedure, successful operation

To unsubscribe from Near-RT RIC API related events, the xApp shall send an API EVENT SUBSCRIPTION DELETE REQUEST message to the Near-RT RIC platform with the *xApp ID* IE and the *Subscription ID* IE.

Upon receiving the message, the Near-RT RIC platform shall:

1. verify the identity of the xApp and check if the xApp is authorized to unsubscribe from the Near-RT RIC API related events associated with the subscription ID; and
2. if the xApp is authorized to unsubscribe from the Near-RT RIC API related events, the Near-RT RIC platform shall delete the resource for the subscription and reply API EVENT SUBSCRIPTION DELETE RESPONSE to the xApp.

9.2.2.3.2 Unsuccessful Operation

```

@startuml
skin rose
skinparam ParticipantPadding 50
skinparam BoxPadding 10
skinparam defaultFontSize 12
skinparam lifelineStrategy solid

Participant xApp as "xApp"
Participant RICP as "Near-RT RIC Platform"

xApp -> RICP : 1. API EVENT SUBSCRIPTION DELETE REQUEST
RICP -> xApp : 2. API EVENT SUBSCRIPTION DELETE FAILURE
@enduml

```

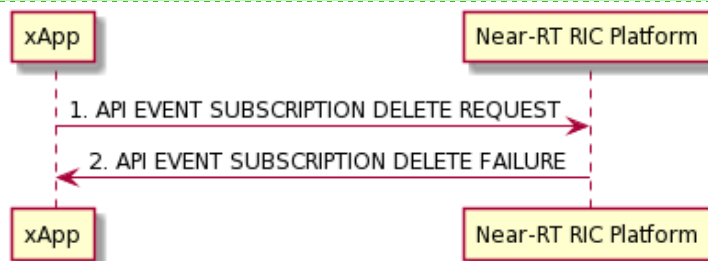


Figure 9.2.2.3.2-1: API Event Subscription Delete procedure, unsuccessful operation

If the Near-RT RIC platform rejects the request it shall respond with the API EVENT SUBSCRIPTION DELETE FAILURE message with proper cause. If the message contains *Time To Wait* IE the xApp shall wait at least the indicated time before reinitiating the request to the Near-RT RIC platform.

9.2.2.3.3 Abnormal conditions

Not applicable.

9.2.2.4 API Event Notification procedure

9.2.2.4.1 Successful Operation

```
@startuml
skin rose
skinparam ParticipantPadding 50
skinparam BoxPadding 10
skinparam defaultFontSize 12
skinparam lifelineStrategy solid

Participant xApp as "xApp"
Participant RICP as "Near-RT RIC Platform"

RICP -> xApp : 1. API EVENT NOTIFICATION
RICP <-- xApp : 2. API EVENT NOTIFICATION ACK

@enduml
```

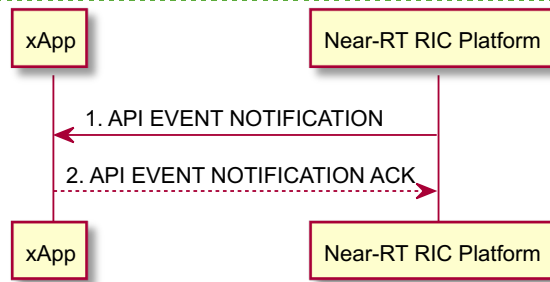


Figure 9.2.2.4.1-1: API Event Notification procedure, successful operation

To notify a Near-RT RIC API related event, the Near-RT RIC platform shall send an API EVENT NOTIFICATION message using the *Notification Destination* IE received in the API Event Subscription procedure.

The Near-RT RIC platform may include details of the event in the *Event Detail* IE. Such details include:

- if the event type is Near-RT_RIC_API_AVAILABLE or Near-RT_RIC_API_UNAVAILABLE, the API IDs in the *List of API IDs* IE;
- if the event is Near-RT_RIC_API_UPDATE, the updated API information in the *List of API Information* IE;

Upon receiving the message, the xApp may reply an API EVENT NOTIFICATION ACK message.

9.2.2.4.2 Unsuccessful Operation

Not applicable.

9.2.2.4.3 Abnormal conditions

Not applicable.

9.2.3 Enabl_Registration_API Procedures

9.2.3.1 General

The Enabl_Registration_API allows an xApp to register its service APIs with the Near-RT RIC platform.

The API procedure defined for Enabl_Registration_API is shown in table 9.2.3.1-1.

Table 9.2.3.1-1: Procedure of the Enabl_Discovery_API

API Procedure	Description	Initiated by
API Registration	This procedure is used by an xApp to register its Near-RT RIC APIs.	xApp
API Registration Update	This procedure is used by an xApp to update its registered Near-RT RIC APIs.	xApp
API Deregistration	This procedure is used by an xApp to deregister its registered Near-RT RIC APIs.	xApp

9.2.3.2 API Registration procedure

9.2.3.2.1 Successful Operation

```

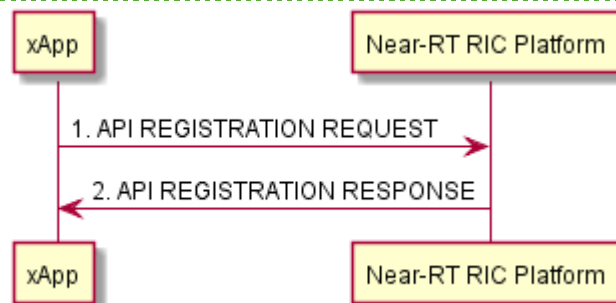
@startuml
skin rose
skinparam ParticipantPadding 50
skinparam BoxPadding 10
skinparam defaultFontSize 12
skinparam lifelineStrategy solid

Participant xApp as "xApp"
Participant RICP as "Near-RT RIC Platform"

xApp -> RICP : 1. API REGISTRATION REQUEST
RICP -> xApp : 2. API REGISTRATION RESPONSE

@enduml

```


Figure 9.2.3.2.1-1: API Registration procedure, successful operation

To register Near-RT RIC APIs with the Near-RT RIC platform, the xApp shall send an API REGISTRATION REQUEST message. The message shall include:

- the *List of API Information to Register* IE, which contain the API information of the Near-RT RIC APIs to be registered.

Upon receiving the message, the Near-RT RIC platform shall verify the authorization of the xApp, assign an API ID for each successfully registered API, and store the associated API information. The Near-RT RIC platform shall reply with an API REGISTRATION RESPONSE message. The message shall include:

- the *List of API Information Successfully Registered* IE, which contains the API information of the successfully registered APIs;
- the *List of API Information Failed To Register* IE if any API is not successfully registered, which contains the corresponding API information and cause.

9.2.3.2.2 Unsuccessful Operation

```

@startuml
skin rose
skinparam ParticipantPadding 50

```

```

skinparam BoxPadding 10
skinparam defaultFontSize 12
skinparam lifelineStrategy solid

Participant xApp as "xApp"
Participant RICP as "Near-RT RIC Platform"

xApp -> RICP : 1. API REGISTRATION REQUEST
RICP -> xApp : 2. API REGISTRATION FAILURE

@enduml

```

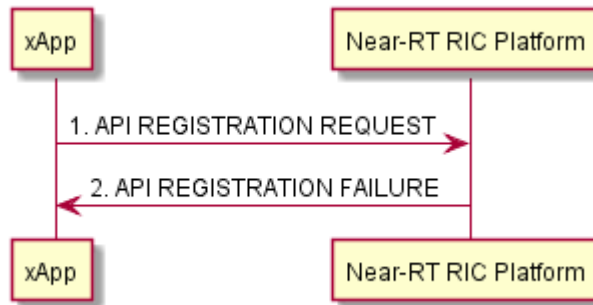


Figure 9.2.3.2.2-1: API Registration procedure, unsuccessful operation

If the Near-RT RIC platform rejects the request it shall respond with the API REGISTRATION FAILURE message. The message shall include the *List of API Information Failed To Register* IE, which contains the corresponding API information and cause.

9.2.3.2.3 Abnormal Conditions

Not applicable.

9.2.3.3 API Registration Update procedure

9.2.3.3.1 Successful Operation

```

@startuml
skin rose
skinparam ParticipantPadding 50
skinparam BoxPadding 10
skinparam defaultFontSize 12
skinparam lifelineStrategy solid

Participant xApp as "xApp"
Participant RICP as "Near-RT RIC Platform"

xApp -> RICP : 1. API REGISTRATION UPDATE REQUEST
RICP -> xApp : 2. API REGISTRATION UPDATE RESPONSE

@enduml

```

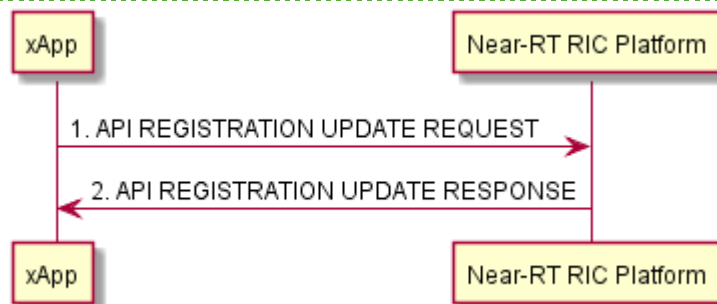


Figure 9.2.3.3.1-1: API Registration Update procedure, successful operation

To update its Near-RT RIC APIs that have been registered at the Near-RT RIC platform, the xApp shall send an API REGISTRATION UPDATE REQUEST message. The message shall include:

- the *List of API Information to Update* IE, which contain the API information of the Near-RT RIC APIs to be updated.

Upon receiving the message, the Near-RT RIC platform shall verify the authorization of the xApp, and update the stored API information with the new API information associated with the same API ID. The Near-RT RIC platform shall reply with an API REGISTRATION UPDATE RESPONSE message. The message shall include:

- the *List of API IDs Successfully Updated* IE, which contains the API IDs of the successfully updated APIs;
- the *List of API IDs Failed To Update* IE if any API is not successfully updated, which contains the corresponding API ID and cause.

9.2.3.3.2 Unsuccessful Operation

```
@startuml
skin rose
skinparam ParticipantPadding 50
skinparam BoxPadding 10
skinparam defaultFontSize 12
skinparam lifelineStrategy solid

Participant xApp as "xApp"
Participant RICP as "Near-RT RIC Platform"

xApp -> RICP : 1. API REGISTRATION UPDATE REQUEST
RICP -> xApp : 2. API REGISTRATION UPDATE FAILURE

@enduml
```

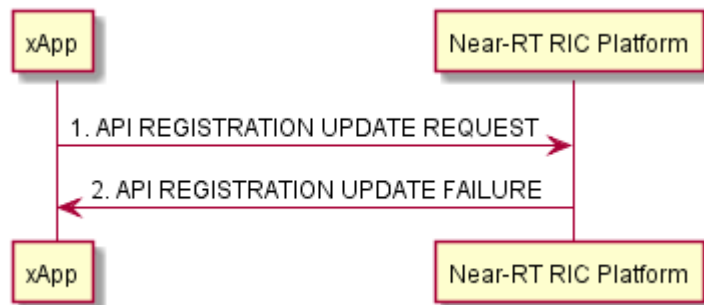


Figure 9.2.3.3.2-1: API Registration Update procedure, unsuccessful operation

If the Near-RT RIC platform rejects the request it shall respond with the API REGISTRATION UPDATE FAILURE message. The message shall include the *List of API IDs Failed To Update* IE, which contains the API IDs that are not updated and the corresponding cause.

9.2.3.3.3 Abnormal Conditions

Not applicable.

9.2.3.4 API Deregistration procedure

9.2.3.4.1 Successful Operation

```
@startuml
skin rose
skinparam ParticipantPadding 50
skinparam BoxPadding 10
skinparam defaultFontSize 12
skinparam lifelineStrategy solid

Participant xApp as "xApp"
Participant RICP as "Near-RT RIC Platform"
```

```

xApp -> RICP : 1. API DEREGISTRATION REQUEST
RICP -> xApp : 2. API DEREGISTRATION RESPONSE

@enduml

```

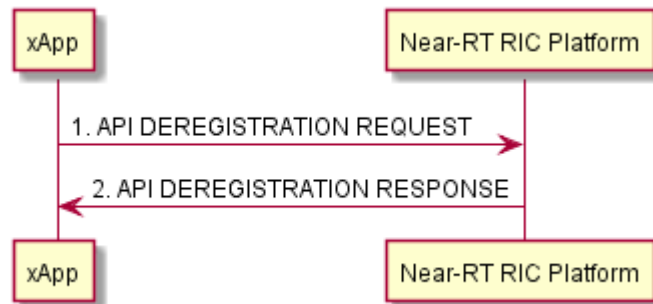


Figure 9.2.3.4.1-1: API Deregistration procedure, successful operation

To deregister its Near-RT RIC APIs from the Near-RT RIC platform, the xApp shall send an API DEREGISTRATION REQUEST message. The message shall include the *List of API IDs To Deregister* IE which indicates the API IDs of the APIs to be deregistered.

Upon receiving the message, the Near-RT RIC platform shall verify the authorization of the xApp, and delete the stored API information associated with the API IDs in the request. The Near-RT RIC platform shall reply with an API DEREGISTRATION RESPONSE message. The message shall include:

- the *List of API IDs Successfully Deregistered* IE, which contains the API IDs of the successfully deregistered APIs;
- the *List of API IDs Failed To Deregister* IE if any API is not successfully deregistered, which contains the corresponding API IDs and cause.

9.2.3.4.2 Unsuccessful Operation

```

@startuml
skin rose
skinparam ParticipantPadding 50
skinparam BoxPadding 10
skinparam defaultFontSize 12
skinparam lifelineStrategy solid

Participant xApp as "xApp"
Participant RICP as "Near-RT RIC Platform"

xApp -> RICP : 1. API DEREGISTRATION REQUEST
RICP -> xApp : 2. API DEREGISTRATION FAILURE

@enduml

```

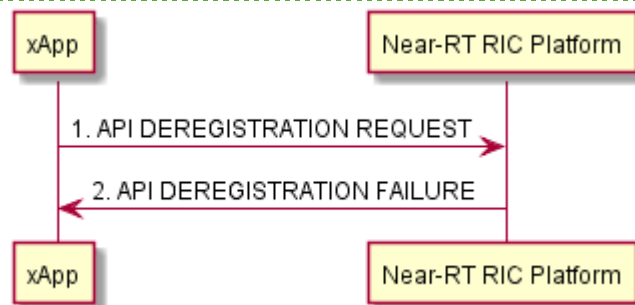


Figure 9.2.3.4.2-1: API Deregistration procedure, unsuccessful operation

If the Near-RT RIC platform rejects the request it shall respond with the API DEREGISTRATION FAILURE message. The message shall include the *List of API IDs Failed To Deregister* IE, which contains the API IDs that are not deregistered and the corresponding cause.

9.2.3.4.3 Abnormal Conditions

Not applicable.

9.3 Elements for Enablement API Communication

9.3.1 General

9.3.2 Message Functional Definition and Content

9.3.2.1 API DISCOVERY REQUEST

This message is sent by an xApp to the Near-RT RIC platform to discover the available Near-RT RIC APIs.

Direction: xApp -> Near-RT RIC platform

IE/Group Name	Presence	Range	IE type and reference	Semantics description	Criticality	Assigned Criticality
Message Type	M		9.3.3.1			
xApp ID	M		10.1.2			
Procedure Transaction ID	M		10.1.1			
Query Parameters		0..1				
>API Name	O		9.3.3.6			
>API Version	O		9.3.3.7			
>Protocol	O		9.3.3.8			
>Data Format	O		9.3.3.9			
>API Category	O		9.3.3.10			

9.3.2.2 API DISCOVERY RESPONSE

This message is sent by Near-RT RIC platform to the xApp to carry the information of discovered Near-RT RIC APIs.

Direction: Near-RT RIC platform -> xApp

IE/Group Name	Presence	Range	IE type and reference	Semantics description	Criticality	Assigned Criticality
Message Type	M		9.3.3.1			
xApp ID	M		10.1.2			
Procedure Transaction ID	M		10.1.1			
List of API Information		0..<maxnoofAPIInformation>		Description of the Near-RT RIC APIs as published by the provider(s).		
>API Information	M		9.3.3.2			

Range bound	Explanation
maxnoofAPIInformation	Maximum no. of API information items supported in an API list. Value is 1024.

9.3.2.3 API DISCOVERY FAILURE

This message is sent by Near-RT RIC platform to the xApp for the case of failure of the discovery of the available near-RT RIC APIs.

Direction: Near-RT RIC platform -> xApp

IE/Group Name	Presence	Range	IE type and reference	Semantics description	Criticality	Assigned Criticality
Message Type	M		9.3.3.1			
xApp ID	M		10.1.2			
Procedure Transaction ID	M		10.1.1			
Cause	M		9.3.3.4			
Time to Wait	O		9.3.3.5			

9.3.2.4 API EVENT SUBSCRIPTION REQUEST

This message is sent by an xApp to the Near-RT RIC platform to subscribe for events related to Near-RT RIC APIs.

Direction: xApp -> Near-RT RIC platform

IE/Group Name	Presence	Range	IE type and reference	Semantics description	Criticality	Assigned Criticality
Message Type	M		9.3.3.1			
xApp ID	M		10.1.2			
Procedure Transaction ID	M		10.1.1			
List of Events		1.. <maxnofAPIEventFilter>				
>Event Type	M		9.3.3.13			
>API ID Filter		0.. <maxnofAPIIDFilters>		API identifiers that the event subscriber wants to know in the interested event.		
>>API ID	M		9.3.3.11			
Event Notification Requirements		1		Represents the reporting requirements of the event subscription.		
>Notification Method	M		ENUMERATED (On Event, Periodic, ...)			
>Notification Period	O		INTEGER (1.. 300, ...)	The duration between notifications expressed in unit of second. Shall be present when the <i>Notification Method</i> IE is set to "Periodic".		
Notification Destination	M		10.1.9			

Range bound	Explanation
maxnoofAPIEventFilter	Maximum no. of API event filters supported in the list. Value is 64.
maxnoofAPIIDFilters	Maximum no. of API ID filters supported in the list. Value is 1024.

9.3.2.5 API EVENT SUBSCRIPTION RESPONSE

This message is sent by Near-RT RIC platform to the xApp as successful response to API event subscription request.

Direction: Near-RT RIC platform -> xApp

IE/Group Name	Presence	Range	IE type and reference	Semantics description	Criticality	Assigned Criticality
Message Type	M		9.3.3.1			
xApp ID	M		10.1.2			
Procedure Transaction ID	M		10.1.1			
Subscription ID	M		9.3.3.3			

9.3.2.6 API EVENT SUBSCRIPTION FAILURE

This message is sent by Near-RT RIC platform to the xApp for the case of failure of the API event subscription request.

Direction: Near-RT RIC platform -> xApp

IE/Group Name	Presence	Range	IE type and reference	Semantics description	Criticality	Assigned Criticality
Message Type	M		9.3.3.1			
xApp ID	M		10.1.2			
Procedure Transaction ID	M		10.1.1			
Cause	M		9.3.3.4			
Time to Wait	O		9.3.3.5			

9.3.2.7 API EVENT SUBSCRIPTION DELETE REQUEST

This message is sent by an xApp to the Near-RT RIC platform to unsubscribe for events related to Near-RT RIC APIs.

Direction: xApp -> Near-RT RIC platform

IE/Group Name	Presence	Range	IE type and reference	Semantics description	Criticality	Assigned Criticality
Message Type	M		9.3.3.1			
xApp ID	M		10.1.2			
Procedure Transaction ID	M		10.1.1			
Subscription ID	M		9.3.3.3			

9.3.2.8 API EVENT SUBSCRIPTION DELETE RESPONSE

This message is sent by Near-RT RIC platform to the xApp as successful response to API event subscription delete request.

Direction: Near-RT RIC platform -> xApp

IE/Group Name	Presence	Range	IE type and reference	Semantics description	Criticality	Assigned Criticality
Message Type	M		9.3.3.1			
xApp ID	M		10.1.2			
Procedure Transaction ID	M		10.1.1			

Subscription ID	M		9.3.3.3			
-----------------	---	--	---------	--	--	--

9.3.2.9 API EVENT SUBSCRIPTION DELETE FAILURE

This message is sent by Near-RT RIC platform to the xApp for the case of failure of the API event subscription delete request.

Direction: Near-RT RIC platform -> xApp

IE/Group Name	Presence	Range	IE type and reference	Semantics description	Criticality	Assigned Criticality
Message Type	M		9.3.3.1			
xApp ID	M		10.1.2			
Procedure Transaction ID	M		10.1.1			
Cause	M		9.3.3.4			
Time to Wait	O		9.3.3.5			

9.3.2.10 API EVENT NOTIFICATION

This message is sent to a subscriber xApp from the Near-RT RIC platform to notify on an API event (based on subscription).

Direction: Near-RT RIC platform -> xApp

IE/Group Name	Presence	Range	IE type and reference	Semantics description	Criticality	Assigned Criticality
Message Type	M		9.3.3.1			
Subscription ID	M		9.3.3.4			
Procedure Transaction ID	M		10.1.1			
Event Type	M		9.3.3.13			
Event Detail		0.. 1				
>List of API IDs		0.. <maxnoofAPIIDs>		The API IDs which are available/unavailable in the relevant event.		
>>API ID	M		9.3.3.11			
>List of API Information		0.. <maxnoofAPIInformation>		Description of the Near-RT RIC APIs which are to be updated.		
>>API Information	M		9.3.3.2			

Range bound	Explanation
maxnoofAPIIDs	Maximum no. of API IDs supported in an API list. Value is 1024.
maxnoofAPIInformation	Maximum no. of API information items supported in an API list. Value is 1024.

9.3.2.11 API EVENT NOTIFICATION ACK

This message is optionally sent by the xApp to Near-RT RIC platform as acknowledgement to an API event notification.

Direction: xApp -> Near-RT RIC platform

IE/Group Name	Presence	Range	IE type and reference	Semantics description	Criticality	Assigned Criticality
Message Type	M		9.3.3.1			
Subscription ID	M		9.3.3.3			

Procedure Transaction ID	M		10.1.1			
--------------------------	---	--	--------	--	--	--

9.3.2.12 API REGISTRATION REQUEST

This message is sent by an xApp to the Near-RT RIC platform to register its Near-RT RIC APIs.

Direction: xApp -> Near-RT RIC platform

IE/Group Name	Presence	Range	IE type and reference	Semantics description	Criticality	Assigned Criticality
Message Type	M		9.3.3.1		YES	Reject
Procedure Transaction ID	M		10.1.1		YES	Reject
List of API Information To Register		1..<maxnoofAPIInformation>			YES	Reject
> API Information	M		9.3.3.2		-	-

Range bound	Explanation
maxnoofAPIInformation	Maximum no. of API information items supported in an API list. Value is 1024.

9.3.2.13 API REGISTRATION RESPONSE

This message is sent by Near-RT RIC platform to the xApp to indicate success of the registration of Near-RT RIC APIs.

Direction: Near-RT RIC platform -> xApp

IE/Group Name	Presence	Range	IE type and reference	Semantics description	Criticality	Assigned Criticality
Message Type	M		9.3.3.1		YES	Reject
Procedure Transaction ID	M		10.1.1		YES	Reject
List of API Information Successfully Registered		1..<maxnoofAPIInformation>		The list of successfully registered APIs	YES	Reject
>API Information	M		9.3.3.2		-	-
List of API Information Failed To Register		0..<maxnoofAPIInformation>		The list of APIs that are not successfully registered	YES	Reject
>API Information	M		9.3.3.2		-	-
>Cause	M		9.3.3.4		-	-

Range bound	Explanation
maxnoofAPIInformation	Maximum no. of API information items supported in an API list. Value is 1024.

9.3.2.14 API REGISTRATION FAILURE

This message is sent by Near-RT RIC platform to the xApp to indicate failure of the registration of Near-RT RIC APIs.

Direction: Near-RT RIC platform -> xApp

IE/Group Name	Presence	Range	IE type and reference	Semantics description	Criticality	Assigned Criticality
Message Type	M		9.3.3.1		YES	Reject
Procedure Transaction ID	M		10.1.1		YES	Reject
List of API Information Failed To Register		1..<maxnoofAPIInformation>		The list of APIs that are not successfully registered.	YES	Reject
> API Information	M		9.3.3.2		-	-
> Cause	M		9.3.3.4		-	-

Range bound	Explanation
maxnoofAPIInformation	Maximum no. of API information items supported in an API list. Value is 1024.

9.3.2.15 API REGISTRATION UPDATE REQUEST

This message is sent by an xApp to the Near-RT RIC platform to update its registered Near-RT RIC APIs.

Direction: xApp -> Near-RT RIC platform

IE/Group Name	Presence	Range	IE type and reference	Semantics description	Criticality	Assigned Criticality
Message Type	M		9.3.3.1		YES	Reject
Procedure Transaction ID	M		10.1.1		YES	Reject
List of API Information To Update		1..<maxnoofAPIInformation>			YES	Reject
> API Information	M		9.3.3.2		-	-

Range bound	Explanation
maxnoofAPIInformation	Maximum no. of API information items supported in an API list. Value is 1024.

9.3.2.16 API REGISTRATION UPDATE RESPONSE

This message is sent by Near-RT RIC platform to the xApp to indicate success of the registration of Near-RT RIC APIs.

Direction: Near-RT RIC platform -> xApp

IE/Group Name	Presence	Range	IE type and reference	Semantics description	Criticality	Assigned Criticality
Message Type	M		9.3.3.1		YES	Reject
Procedure Transaction ID	M		10.1.1		YES	Reject
List of API IDs Successfully Updated		1..<maxnoofAPIIDs>		The list of successfully updated APIs	YES	Reject
>API ID	M		9.3.3.11		-	-
List of API IDs Failed To Update		0..<maxnoofAPIIDs>		The list of APIs that are not successfully updated.	YES	Reject
>API ID	M		9.3.3.11		-	-

>Cause	M		9.3.3.4		-	-
--------	---	--	---------	--	---	---

Range bound	Explanation
maxnoofAPIIDs	Maximum no. of API IDs supported in an API list. Value is 1024.

9.3.2.17 API REGISTRATION UPDATE FAILURE

This message is sent by Near-RT RIC platform to the xApp to indicate failure of the registration of Near-RT RIC APIs.

Direction: Near-RT RIC platform -> xApp

IE/Group Name	Presence	Range	IE type and reference	Semantics description	Criticality	Assigned Criticality
Message Type	M		9.3.3.1		YES	Reject
Procedure Transaction ID	M		10.1.1		YES	Reject
List of API IDs Failed to Update		1..<maxnoofAPIIDs>		The list of APIs that are not successfully updated.	YES	Reject
> API ID	M		9.3.3.11		-	-
> Cause	M		9.3.3.4		-	-

Range bound	Explanation
maxnoofAPIIDs	Maximum no. of API IDs supported in an API list. Value is 1024.

9.3.2.18 API DEREGISTRATION REQUEST

This message is sent by an xApp to the Near-RT RIC platform to deregister its Near-RT RIC APIs.

Direction: xApp -> Near-RT RIC platform

IE/Group Name	Presence	Range	IE type and reference	Semantics description	Criticality	Assigned Criticality
Message Type	M		9.3.3.1		YES	Reject
Procedure Transaction ID	M		10.1.1		YES	Reject
List of API IDs To Deregister		1..<maxnoofAPIIDs>			YES	Reject
> API ID	M		9.3.3.11		-	-

Range bound	Explanation
maxnoofAPIIDs	Maximum no. of API IDs supported in an API list. Value is 1024.

9.3.2.19 API DEREGISTRATION RESPONSE

This message is sent by Near-RT RIC platform to the xApp to indicate success of the deregistration of Near-RT RIC APIs.

Direction: Near-RT RIC platform -> xApp

IE/Group Name	Presence	Range	IE type and reference	Semantics description	Criticality	Assigned Criticality
Message Type	M		9.3.3.1		YES	Reject
Procedure Transaction ID	M		10.1.1		YES	Reject

List of API IDs Successfully Deregistered		1..<maxnoofAPIIDs>		The list of successfully deregistered APIs	YES	Reject
>API ID	M		9.3.3.11		-	-
List of API IDs Failed To Deregister		0..<maxnoofAPIIDs>		The list of APIs that are not successfully deregistered	YES	Reject
>API ID	M		9.3.3.11		-	-
>Cause	M		9.3.3.4		-	-

Range bound	Explanation
maxnoofAPIIDs	Maximum no. of API IDs supported in an API list. Value is 1024.

9.3.2.20 API DEREGISTRATION FAILURE

This message is sent by Near-RT RIC platform to the xApp to indicate failure of the deregistration of Near-RT RIC APIs.

Direction: Near-RT RIC platform -> xApp

IE/Group Name	Presence	Range	IE type and reference	Semantics description	Criticality	Assigned Criticality
Message Type	M		9.3.3.1		YES	Reject
Procedure Transaction ID	M		10.1.1		YES	Reject
List of API IDs Failed To Deregister		1..<maxnoofAPIInformation>		The list of APIs that are not successfully deregistered.	YES	Reject
> API ID	M		9.3.3.11		-	-
> Cause	M		9.3.3.4		-	-

Range bound	Explanation
maxnoofAPIIDs	Maximum no. of API IDs supported in an API list. Value is 1024.

9.3.3 Information Element Definitions

9.3.3.1 Message Type

This IE identifies the message being sent. It is mandatory for all messages.

IE/Group Name	Presence	Range	IE Type and Reference	Semantics Description
Message Type				
>Procedure Type	M		CHOICE (API Discovery, API Event Subscription, API Event Subscription Delete, API Event Notification, API Registration, API Registration Update, API Deregistration, ...)	
>Type of Message	M		CHOICE (Request, Response, Failure, ...)	Refer to Table 9.1.2-3 for applicable choices for each procedure type

9.3.3.2 API Information

This IE indicates the information of a Near-RT RIC API.

IE/Group Name	Presence	Range	IE type and reference	Semantics description
API Name	M		9.3.3.6	
API ID	C		9.3.3.11	Assigned by the Near-RT RIC platform. Shall be present when the API Information is associated with a registered API, or used to update a registered API.
API Category	M		9.3.3.10	
Description	O		STRING	Text description of the API
List of API Profiles		1..<maxnoofAPIProfiles>		
>API Version	M		9.3.3.7	
>Protocol	O		9.3.3.8	
>Data Format	O		9.3.3.9	
>Endpoint Information	O		10.1.9	
>List of Supported Data Types		0..<maxnoofSupportedDataTypes>		Shall be present for SDL APIs.
>> Data Type ID	M		STRING	For the API information of SDL APIs, this IE is populated with the SDL Data Type ID specified in clause 7.3.3.21.

Range bound	Explanation
maxnoofSupportedDataTypes	Maximum no. of supported data types in an API profile. Value is 1024.

9.3.3.3 Subscription ID

The IE uniquely identifies a subscription to API-related event(s).

IE/Group Name	Presence	Range	IE type and reference	Semantics description
Subscription ID	M		STRING	Identifier of the subscription resource to which the notification is related

9.3.3.4 Cause

This IE is used to indicate the reason for a particular event for the Enablement APIs.

IE/Group Name	Presence	Range	IE type and reference	Semantics description
Cause	M		ENUMERATED (Unspecified, System busy, Duplicate API Information, Unknown API ID, Invalid Data Type, Unknown Subscription ID,...)	

The meaning of the different cause values is described in the following table.

Cause value	Meaning
Unspecified	Sent when none of the below cause values applies.
System busy	The system cannot process the request due to temporary overload.
Duplicate API Information	The same API information has been registered.
Unknown API ID	The API ID is unknown.
Invalid Data Type	The supported data type in the API information is invalid.
Unknown Subscription ID	The Subscription ID for API event is unknown.

9.3.3.5 Time to Wait

This IE specifies the time that xApp shall wait before reinitiating a request.

IE/Group Name	Presence	Range	IE type and reference	Semantics description
Time to Wait	M		INTEGER (1.. 300, ...)	Unit in second

9.3.3.6 API Name

Defined in 10.1.3.

9.3.3.7 API Version

Defined in 10.1.4.

9.3.3.8 Protocol

IE/Group Name	Presence	Range	IE type and reference	Semantics description
Protocol	M		ENUMERATED (gRPC, SCTP, HTTP REST, ...)	Protocol used by the Near-RT RIC API

9.3.3.9 Data Format

IE/Group Name	Presence	Range	IE type and reference	Semantics description
Data Format	M		ENUMERATED (Proto3, JSON, ASN.1, ...)	Data format used by the Near-RT RIC API

9.3.3.10 API Category

IE/Group Name	Presence	Range	IE type and reference	Semantics description
API Category	M		ENUMERATED (E2 related APIs, SDL APIs, A1 related APIs, ...)	The Near-RT RIC API category to which the API belongs

9.3.3.11 API ID

IE/Group Name	Presence	Range	IE type and reference	Semantics description
API ID	M		STRING	API identifier assigned by the Near-RT RIC platform to the published Near-RT RIC API

9.3.3.12 Void

9.3.3.13 Event Type

IE/Group Name	Presence	Range	IE type and reference	Semantics description
---------------	----------	-------	-----------------------	-----------------------

Event Type	M		ENUMERATED (NEAR_RT_RIC_API_AVAILABLE, NEAR_RT_RIC_API_UNAVAILABLE, NEAR_RT_RIC_API_API_UPDATE , ...)	
------------	---	--	---	--

Table 9.3.3.13-1: Meaning of Event Types

value	Description
NEAR_RT_RIC_API_AVAILABLE	Events related to the availability of APIs after the APIs are published.
NEAR_RT_RIC_API_UNAVAILABLE	Events related to the unavailability of APIs after the APIs are unpublished.
NEAR_RT_RIC_API_API_UPDATE	Events related to change in API information

9.3.4 Message and Information Element Abstract Syntax

9.3.4.1 Solution 1: gRPC with Protocol Buffers

9.3.4.1.1 Usage of gRPC with Protocol Buffers

This solution uses Protocol Buffers as message encoding, and also as the Interface Definition Language (IDL) for gRPC [5].

Table 9.3.4.1.1-1 lists the mapping of enablement APIs and corresponding gRPC services.

Table 9.3.4.1.1-1: Mapping of enablement APIs and gRPC services

API Name	gRPC service(s)
Enabl_Discovery_API	Enabl_Discovery_API
Enabl_Events_API	Enabl_Events_API, Enabl_Events_API_Notification
Enabl_Registration_API	Enabl_Registration_API

A procedure of an Enablement API is mapped to a *gRPC method*.

Table 9.3.4.1.1-2 lists the Enablement APIs abstract syntax included in this document.

Table 9.3.4.1.1-2: List of Enablement APIs abstract syntax

API Name / Contents	Clause	Version	File Name
Enabl_Discovery_API	9.3.4.1.2.1	2.0.0	enabl_discovery_api_ver_2_0_0.proto
Enabl_Events_API	9.3.4.1.2.2	2.0.0	enabl_events_api_ver_2_0_0.proto
Enabl_Registration_API	9.3.4.1.2.3	1.0.0	enabl_registration_api_ver_1_0_0.proto
Enablement API common information elements	9.3.4.1.3	2.0.0	enabl_apis_common_ies_ver_2_0_0.proto
Near-RT RIC API common information elements	10.2.1	2.0.0	near_rt_ric_apis_common_ies_ver_2_0_0.proto

9.3.4.1.2 API Definitions

9.3.4.1.2.1 Enabl_Discovery_API

```
syntax = "proto3";

package ricapi.enabl_discovery_api.v2;

import "near_rt_ric_apis_common_ies_ver_2_0_0.proto";
import "enabl_apis_common_ies_ver_2_0_0.proto";
```

```
// gRPC service and rpc definition
service Enabl_Discovery_API {
    rpc ApiDiscoveryProc(ApiDiscoveryInitMsg) returns (ApiDiscoveryOutMsg) {};
}

message ApiDiscoveryInitMsg {
    optional ApiDiscoveryRequest api_discovery_request = 1;
}

message ApiDiscoveryOutMsg {
    oneof type_of_message {
        ApiDiscoveryResponse api_discovery_response = 1;
        ApiDiscoveryFailure api_discovery_failure = 2;
    }
}

// ----- messages of Enabl_Discovery_API -----
// ----- messages of API Discovery procedure -----
message ApiDiscoveryRequest {
    optional near_rt_ric_apis_common_ies.v2.XappId xapp_id = 1;
    optional near_rt_ric_apis_common_ies.v2.ProcedureTransactionId procedure_transaction_id = 3;
    optional QueryParameters qps = 2;

    message QueryParameters {
        optional near_rt_ric_apis_common_ies.v2.ApiName api_name = 1;
        optional near_rt_ric_apis_common_ies.v2.ApiVersion api_version = 2;
        optional enabl_apis_common_ies.v2.Protocol protocol = 3;
        optional enabl_apis_common_ies.v2.DataFormat data_format = 4;
        optional enabl_apis_common_ies.v2.ApiCategory api_category = 5;
    };
};

message ApiDiscoveryResponse {
    optional near_rt_ric_apis_common_ies.v2.XappId xapp_id = 1;
    optional near_rt_ric_apis_common_ies.v2.ProcedureTransactionId procedure_transaction_id = 3;
    repeated enabl_apis_common_ies.v2.ApiInformation list_of_api_information = 2;
};

message ApiDiscoveryFailure {
    optional near_rt_ric_apis_common_ies.v2.XappId xapp_id = 1;
    optional near_rt_ric_apis_common_ies.v2.ProcedureTransactionId procedure_transaction_id = 4;
    optional enabl_apis_common_ies.v2.Cause cause = 2;
    optional uint32 time_to_wait = 3;
};
```

9.3.4.1.2.2 Enabl_Events_API

```
syntax = "proto3";

package ricapi.enabl_events_api.v2;

import "near_rt_ric_apis_common_ies_ver_2_0_0.proto";
import "enabl_apis_common_ies_ver_2_0_0.proto";

// gRPC service and rpc definition
service Enabl_Events_API {
    rpc ApiEventSubscriptionProc(ApiEventSubscriptionInitMsg) returns (ApiEventSubscriptionOutMsg) {};
    rpc ApiEventSubscriptionDeleteProc(ApiEventSubscriptionDeleteInitMsg) returns
(ApiEventSubscriptionDeleteOutMsg) {};
}

message ApiEventSubscriptionInitMsg {
    optional ApiEventSubscriptionRequest api_event_subscription_request = 1;
}

message ApiEventSubscriptionOutMsg {
    oneof type_of_message {
        ApiEventSubscriptionResponse api_event_subscription_response = 1;
        ApiEventSubscriptionFailure api_event_subscription_failure = 2;
    }
}
```

```

message ApiEventSubscriptionDeleteInitMsg {
    optional ApiEventSubscriptionDeleteRequest api_event_subscription_delete_request = 1;
}

message ApiEventSubscriptionDeleteOutMsg {
    oneof type_of_message {
        ApiEventSubscriptionDeleteResponse api_event_subscription_delete_response = 1;
        ApiEventSubscriptionDeleteFailure api_event_subscription_delete_failure = 2;
    }
}

service Enabl_Events_API_Notification {
    rpc ApiEventNotificationProc(ApiEventNotificationInitMsg) returns (ApiEventNotificationOutMsg) {};
}

message ApiEventNotificationInitMsg {
    optional ApiEventNotification api_event_notification = 1;
}

message ApiEventNotificationOutMsg {
    oneof type_of_message {
        ApiEventNotificationACK api_event_notification_ack = 1;
    }
}

// ----- messages of Enabl_Events_API -----
// ----- messages of API Event Subscription procedure -----
message ApiEventSubscriptionRequest {
    optional near_rt_ric_apis_common_ies.v2.XappId xapp_id = 1;
    optional near_rt_ric_apis_common_ies.v2.ProcedureTransactionId procedure_transaction_id = 5;
    repeated EventItem list_of_events = 2;
    optional EventNotificationRequirements event_notification_requirements = 3;
    optional near_rt_ric_apis_common_ies.v2.EndpointInformation notification_destination = 4;

    message EventItem {
        optional enabl_apis_common_ies.v2.EventType event_type = 1;
        repeated ApiIdFilterItem api_id_filter = 2;

        message ApiIdFilterItem {
            optional enabl_apis_common_ies.v2.ApiId api_id = 1;
        }
    }

    message EventNotificationRequirements {
        optional NotificationMethod notification_method = 1;
        optional uint32 notification_period = 2;

        enum NotificationMethod {
            NOTIFICATION_METHOD_ON_EVENT = 0;
            NOTIFICATION_METHOD_PERIODIC = 1;
        }
    }
};

message ApiEventSubscriptionResponse {
    optional near_rt_ric_apis_common_ies.v2.XappId xapp_id = 1;
    optional near_rt_ric_apis_common_ies.v2.ProcedureTransactionId procedure_transaction_id = 3;
    optional enabl_apis_common_ies.v2.SubscriptionId subscription_id = 2;
}

message ApiEventSubscriptionFailure {
    optional near_rt_ric_apis_common_ies.v2.XappId xapp_id = 1;
    optional near_rt_ric_apis_common_ies.v2.ProcedureTransactionId procedure_transaction_id = 4;
    optional enabl_apis_common_ies.v2.Cause cause = 2;
    optional uint32 time_to_wait = 3;
}

// ----- messages of API Event Subscription Delete procedure -----
message ApiEventSubscriptionDeleteRequest {
    optional near_rt_ric_apis_common_ies.v2.XappId xapp_id = 1;
    optional near_rt_ric_apis_common_ies.v2.ProcedureTransactionId procedure_transaction_id = 3;
    optional enabl_apis_common_ies.v2.SubscriptionId subscription_id = 2;
};

```

```

message ApiEventSubscriptionDeleteResponse {
    optional near_rt_ric_apis_common_ies.v2.XappId xapp_id = 1;
    optional near_rt_ric_apis_common_ies.v2.ProcedureTransactionId procedure_transaction_id = 3;
    optional enabl_apis_common_ies.v2.SubscriptionId subscription_id = 2;
}

message ApiEventSubscriptionDeleteFailure {
    optional near_rt_ric_apis_common_ies.v2.XappId xapp_id = 1;
    optional near_rt_ric_apis_common_ies.v2.ProcedureTransactionId procedure_transaction_id = 4;
    optional enabl_apis_common_ies.v2.Cause cause = 2;
    optional uint32 time_to_wait = 3;
}

// ----- messages of API Event Notification procedure -----
message ApiEventNotification {
    optional enabl_apis_common_ies.v2.SubscriptionId subscription_id = 1;
    optional near_rt_ric_apis_common_ies.v2.ProcedureTransactionId procedure_transaction_id = 4;
    optional enabl_apis_common_ies.v2.EventType event_type = 2;
    optional EventDetail event_detail = 3;

    message EventDetail {
        repeated enabl_apis_common_ies.v2.ApiId list_of_api_ids = 1;
        repeated enabl_apis_common_ies.v2.ApiInformation list_of_api_information = 2;
    };
};

message ApiEventNotificationACK {
    optional enabl_apis_common_ies.v2.SubscriptionId subscription_id = 1;
    optional near_rt_ric_apis_common_ies.v2.ProcedureTransactionId procedure_transaction_id = 2;
};

```

9.3.4.1.2.3 Enabl_Registration_API

```

syntax = "proto3";

package ricapi.enabl_registration_api_v1;

import "near_rt_ric_apis_common_ies_ver_2_0_0.proto";
import "enabl_apis_common_ies_ver_2_0_0.proto";

// gRPC service and rpc definition
service Enabl_Registration_API {
    rpc ApiRegistrationProc(ApiRegistrationInitMsg) returns (ApiRegistrationOutMsg) {};
    rpc ApiRegistrationUpdateProc(ApiRegistrationUpdateInitMsg) returns (ApiRegistrationUpdateOutMsg) {};
    rpc ApiDeregistrationProc(ApiDeregistrationInitMsg) returns (ApiDeregistrationOutMsg) {};
}

message ApiRegistrationInitMsg {
    optional ApiRegistrationRequest api_registration_request = 1;
}

message ApiRegistrationOutMsg {
    oneof type_of_message {
        ApiRegistrationResponse api_registration_response = 1;
        ApiRegistrationFailure api_registration_failure = 2;
    }
}

message ApiRegistrationUpdateInitMsg {
    optional ApiRegistrationUpdateRequest api_registration__update_request = 1;
}

message ApiRegistrationUpdateOutMsg {
    oneof type_of_message {
        ApiRegistrationUpdateResponse api_registration_update_response = 1;
        ApiRegistrationUpdateFailure api_registration_update_failure = 2;
    }
}

message ApiDeregistrationInitMsg {
    optional ApiDeregistrationRequest api_deregistration_request = 1;
}

```

```

}

message ApiDeregistrationOutMsg {
    oneof type_of_message {
        ApiDeregistrationResponse api_deregistration_response = 1;
        ApiDeregistrationFailure api_deregistration_failure = 2;
    }
}

// ----- messages of Enabl_Registration_API -----
// ----- messages of API Registration procedure -----
message ApiRegistrationRequest {
    optional near_rt_ric_apis_common_ies.v2.ProcedureTransactionId procedure_transaction_id = 1;
    repeated ApiInformationToRegisterItem list_of_api_information_to_register = 2;

    message ApiInformationToRegisterItem {
        optional enabl_apis_common_ies.v2.ApiInformation api_information = 1;
    }
}

message ApiRegistrationResponse {
    optional near_rt_ric_apis_common_ies.v2.ProcedureTransactionId procedure_transaction_id = 1;
    repeated ApiInformationSuccessfullyRegisteredItem list_of_api_information_successfully_registered = 2;
    repeated ApiInformationFailedToRegisterItem list_of_api_information_failed_to_register = 3;

    message ApiInformationSuccessfullyRegisteredItem {
        optional enabl_apis_common_ies.v2.ApiInformation api_information = 1;
    }

    message ApiInformationFailedToRegisterItem {
        optional enabl_apis_common_ies.v2.ApiInformation api_information = 1;
        optional enabl_apis_common_ies.v2.Cause cause = 2;
    }
}

message ApiRegistrationFailure {
    optional near_rt_ric_apis_common_ies.v2.ProcedureTransactionId procedure_transaction_id = 1;
    repeated ApiInformationFailedToRegisterItem list_of_api_information_failed_to_register = 2;

    message ApiInformationFailedToRegisterItem {
        optional enabl_apis_common_ies.v2.ApiInformation api_information = 1;
        optional enabl_apis_common_ies.v2.Cause cause = 2;
    }
}

// ----- messages of API Registration Update procedure -----
message ApiRegistrationUpdateRequest {
    optional near_rt_ric_apis_common_ies.v2.ProcedureTransactionId procedure_transaction_id = 1;
    repeated ApiInformationToUpdateItem list_of_api_information_to_update = 2;

    message ApiInformationToUpdateItem {
        optional enabl_apis_common_ies.v2.ApiInformation api_information = 1;
    }
}

message ApiRegistrationUpdateResponse {
    optional near_rt_ric_apis_common_ies.v2.ProcedureTransactionId procedure_transaction_id = 1;
    repeated ApiIdSuccessfullyUpdatedItem list_of_api_ids_successfully_updated = 2;
    repeated ApiIdFailedToUpdateItem list_of_api_ids_failed_to_update = 3;

    message ApiIdSuccessfullyUpdatedItem {
        optional enabl_apis_common_ies.v2.ApiId api_id = 1;
    }

    message ApiIdFailedToUpdateItem {
        optional enabl_apis_common_ies.v2.ApiId api_id = 1;
        optional enabl_apis_common_ies.v2.Cause cause = 2;
    }
}

message ApiRegistrationUpdateFailure {
    optional near_rt_ric_apis_common_ies.v2.ProcedureTransactionId procedure_transaction_id = 1;
    repeated ApiIdFailedToUpdateItem list_of_api_ids_failed_to_update = 2;
}

```

```

message ApiIdFailedToUpdateItem {
    optional enabl_apis_common_ies.v2.ApiId api_id = 1;
    optional enabl_apis_common_ies.v2.Cause cause = 2;
}

// ----- messages of API Deregistration procedure -----
message ApiDeregistrationRequest {
    optional near_rt_ric_apis_common_ies.v2.ProcedureTransactionId procedure_transaction_id = 1;
    repeated ApiIdToDeregisterItem list_of_api_ids_to_deregister = 2;

    message ApiIdToDeregisterItem {
        optional enabl_apis_common_ies.v2.ApiId api_id = 1;
    }
}

message ApiDeregistrationResponse {
    optional near_rt_ric_apis_common_ies.v2.ProcedureTransactionId procedure_transaction_id = 1;
    repeated ApiIdSuccessfullyDeregisteredItem list_of_api_ids_successfully_deregistered = 2;
    repeated ApiIdFailedToDeregisterItem list_of_api_ids_failed_to_deregister = 3;

    message ApiIdSuccessfullyDeregisteredItem {
        optional enabl_apis_common_ies.v2.ApiId api_id = 1;
    }

    message ApiIdFailedToDeregisterItem {
        optional enabl_apis_common_ies.v2.ApiId api_id = 1;
        optional enabl_apis_common_ies.v2.Cause cause = 2;
    }
}

message ApiDeregistrationFailure {
    optional near_rt_ric_apis_common_ies.v2.ProcedureTransactionId procedure_transaction_id = 1;
    repeated ApiIdFailedToDeregisterItem list_of_api_ids_failed_to_deregister = 2;

    message ApiIdFailedToDeregisterItem {
        optional enabl_apis_common_ies.v2.ApiId api_id = 1;
        optional enabl_apis_common_ies.v2.Cause cause = 2;
    }
}

```

9.3.4.1.3 Information Element Definitions

```

syntax = "proto3";

package ricapi.enabl_apis_common_ies.v2;

import "near_rt_ric_apis_common_ies_ver_2_0_0.proto";

// ----- Information Elements -----
// -----A-----
enum ApiCategory {
    APICATEGORY_E2_RELATED = 0;
    APICATEGORY_SDL = 1;
    APICATEGORY_A1_RELATED = 2;
};

message ApiId {
    optional string value = 1;
}

message ApiInformation {
    optional near_rt_ric_apis_common_ies.v2.ApiName api_name = 1;
    optional ApiId api_id = 2;
    optional ApiCategory api_category = 3;
    optional string description = 4;
    repeated ApiProfile list_of_api_profiles = 5;

    message ApiProfile {
        optional near_rt_ric_apis_common_ies.v2.ApiVersion api_version = 1;
        optional Protocol protocol = 2;
    }
}

```

```

        optional DataFormat data_format = 3;
        optional near_rt_ric_apis_common_ies.v2.EndpointInformation endpoint_information = 4;
        repeated SupportedDataTypeItem list_of_supported_data_types = 5;
        message SupportedDataTypeItem {
            optional string data_type_id = 1;
        };
};

// -----C-----
enum Cause {
    UNSPECIFIED = 0;
    SYSTEM_BUSY = 1;
    DUPLICATE_API_INFORMATION = 2;
    UNKNOWN_API_ID = 3;
    INVALID_DATA_TYPE = 4;
    UNKNOWN_SUBSCRIPTION_ID = 5;
}

// -----D-----
enum DataFormat {
    DATA_FORMAT_PROTO3 = 0;
    DATA_FORMAT_JSON = 1;
    DATA_FORMAT_ASN1 = 2;
};

enum EventType {
    EVENT_TYPE_NEAR_RT_RIC_API_AVAILABLE = 0;
    EVENT_TYPE_NEAR_RT_RIC_API_UNAVAILABLE = 1;
    EVENT_TYPE_NEAR_RT_RIC_API_UPDATE = 2;
}

// -----P-----
enum Protocol {
    PROTOCOL_GRPC = 0;
    PROTOCOL_SCTP = 1;
    PROTOCOL_HTTP = 2;
};

// -----S-----
message SubscriptionId {
    optional string value = 1;
}

```

9.3.5 Message Transfer Syntax

9.3.5.1 Solution 1: gRPC with Protocol Buffers

The message transfer syntax for Solution 1 is serialized Protocol Buffers [4].

10 Common definitions across APIs

10.1 Information element definitions

10.1.1 PTI

The Procedure Transaction Identity (PTI) IE allows identifying a procedure among all ongoing parallel procedures of the same type initiated by the same RICAPI peer. The PTI value is assigned to the initiating message by the procedure initiator, and subsequent messages shall include the same PTI value as in the initiating message, if any. PTI is released when the procedure is completed.

IE/Group Name	Presence	Range	IE Type and Reference	Semantics Description
PTI	M		INTEGER (0..2 ³² -1)	

10.1.2 xApp ID

The xApp ID IE uniquely identifies an xApp.

IE/Group Name	Presence	Range	IE type and reference	Semantics description
xApp ID	M		STRING	The format of the xApp ID shall be a Universally Unique Identifier (UUID) version 4, as described in [9].

10.1.3 API Name

The API Name IE indicates the Name of each Near-RT RIC API.

IE/Group Name	Presence	Range	IE type and reference	Semantics description
API Name	M		ENUMERATED (E2_Subscription_API, E2_Indication_API, E2_Control_API, E2_Guidance_API, A1_Policy_API, A1_EI_API, SDL_Information_API, Xapp_Data_Sharing_API, ...)	Contains the Near-RT RIC API name.

The table 10.1.3-1 lists the API Names.

Table 10.1.3-1: List of API Names

API Category	API Name
E2 related APIs	E2_Subscription_API, E2_Indication_API, E2_Control_API, E2_Guidance_API
A1 related APIs	A1_Policy_API, A1_EI_API
SDL APIs	SDL_Information_API, Xapp_Data_Sharing_API

10.1.4 API Version

The API Version IE indicates the Version of each Near-RT RIC API.

IE/Group Name	Presence	Range	IE type and reference	Semantics description
API Version	M		STRING	Contains the Near-RT RIC API Version.

10.1.5 A1 Policy Type ID

The A1 Policy Type ID IE identifies the type of one A1 policy.

IE/Group Name	Presence	Range	IE Type and Reference	Semantics Description
A1 Policy Type ID	M		STRING	Applicable values defined in [7].

10.1.6 Global E2 Node ID

Defined in 9.2.6 of [3].

10.1.7 RAN Function ID

Defined in 9.2.8 of [3].

10.1.8 SCTP API PDU

The SCTP API PDU IE serves as the outer layer encapsulation for the payload when SCTP is used as the transport protocol.

IE/Group Name	Presence	Range	IE Type and Reference	Semantics Description
API Name	M		10.1.3	
API Version	M		10.1.4	
API Payload	M		OCTET STRING	

10.1.9 Endpoint Information

IE/Group Name	Presence	Range	IE type and reference	Semantics description
Domain Name	O		STRING	(NOTE1)
IPv4 Address	O		STRING	(NOTE1)
IPv6 Address	O		STRING	(NOTE1)
Port	O		INTEGER	
NOTE1: At least one of these IEs shall be present.				

10.2 Information element abstract syntax

10.2.1 Abstract syntax in Protocol Buffers

Table 10.2.1-1 lists the common IEs abstract syntax included in this document.

Table 10.2.1-1: List of common IEs abstract syntax

API Name / Contents	Clause	Version	File Name
Near-RT RIC API common information elements	10.2.1	2.0.0	near_rt_ric_apis_common_ies_ver_2_0_0.proto

```

syntax = "proto3";

package ricapi.near_rt_ric_apis_common_ies.v2;

// Information elements used across different API categories

// ----- Information Elements -----
// -----A-----
// From A1TD Clause 5.2
message A1PolicyTypeId {
    optional string value = 1;
}

enum ApiName {
    APINAME_E2_SUBSCRIPTION_API = 0;
    APINAME_E2_INDICATION_API = 1;
    APINAME_E2_CONTROL_API = 2;
}

```

```

    APINAME_E2_GUIDANCE_API = 3;
    APINAME_A1_POLICY_API = 4;
    APINAME_A1_EI_API = 5;
    APINAME_SDL_INFORMATION_API = 6;
    APINAME_XAPP_DATA_SHARING_API = 7;
};

message ApiVersion {
    optional string value = 1;
}

// -----B-----
// BitString definition
message BitString {
    optional bytes value = 1;
    optional uint32 len = 2;
}

// -----E-----
message EndpointInformation {
    optional string domain_name = 1;
    optional string ipv4_address = 2;
    optional string ipv6_address = 3;
    optional uint32 port = 4;
};

// -----G-----
// From E2AP Clause 9.2.6, "Global E2 Node ID"
message GlobalE2NodeId {
    oneof node_type {
        GlobalE2NodeGnbId gnb = 1;
        GlobalE2NodeEnGnbId en_gnb = 2;
        GlobalE2NodeNgEnbId ng_enb = 3;
        GlobalE2NodeEnbId enb = 4;
    }

    message GlobalE2NodeGnbId {
        optional GlobalGnbId global_gnb_id = 1;
        optional GlobalEnGnbId global_en_gnb_id = 2;
        optional GnbCuUpId gnb_cu_up_id = 3;
        optional GnbDuId gnb_du_id = 4;
    }

    message GlobalE2NodeEnGnbId {
        optional GlobalEnGnbId global_en_gnb_id = 1;
        optional GnbCuUpId en_gnb_cu_up_id = 2;
        optional GnbDuId en_gnb_du_id = 3;
    }

    message GlobalE2NodeNgEnbId {
        optional GlobalNgEnbId global_ng_enb_id = 1;
        optional GlobalEnbId global_enb_id = 2;
        optional NgEnbDuId ng_enb_du_id = 3;
    }

    message GlobalE2NodeEnbId {
        optional GlobalEnbId global_enb_id = 1;
    }
}

// From E2AP Clause 9.2.6, as part of "Global E2 Node ID"
message GlobalEnbId {
    optional PlmnIdentity plmn_identity = 1;
    oneof enb_id_choice {
        BitString macro_enb_id = 2;
        BitString home_enb_id = 3;
        BitString short_macro_enb_id = 4;
        BitString long_macro_enb_id = 5;
    }
}

// From E2AP Clause 9.2.6, as part of "Global E2 Node ID"
message GlobalEnGnbId {

```

```

    optional PlmnIdentity plmn_identity = 1;
    oneof en_gnb_id_choice {
        BitString en_gnb_id = 2;
    }
}

// From E2AP Clause 9.2.6, as part of "Global E2 Node ID"
message GlobalGnbId {
    optional PlmnIdentity plmn_identity = 1;
    oneof gnb_id_choice {
        BitString gnb_id = 2;
    }
}

// From E2AP Clause 9.2.6, as part of "Global E2 Node ID"
message GlobalNgEnbId {
    optional PlmnIdentity plmn_identity = 1;
    oneof ng_enb_id_choice {
        BitString macro_ng_enb_id = 2;
        BitString short_macro_ng_enb_id = 3;
        BitString long_macro_ng_enb_id = 4;
    }
}

// From E2AP Clause 9.2.6, as part of "Global E2 Node ID"
message GnbCuUpId {
    optional uint64 value = 1;
}

// From E2AP Clause 9.2.6, as part of "Global E2 Node ID"
message GnbDuId {
    optional uint64 value = 1;
}

// -----N-----
// From E2AP Clause 9.2.6, as part of "Global E2 Node ID"
message NgEnbDuId {
    optional uint64 value = 1;
};

// -----P-----
// From E2AP Clause 9.2.6, as part of "Global E2 Node ID"
message PlmnIdentity {
    optional bytes value = 1;
};

message ProcedureTransactionId {
    optional uint32 value = 1;
};

// -----R-----
// From E2AP Clause 9.2.8, "RAN Function ID"
message RanFunctionId {
    optional uint32 value = 1;
};

//-----S-----
// Outer layer encapsulation for SCTP based APIs
message SctpApiPdu {
    optional ApiName api_name = 1;
    optional ApiVersion api_version = 2;
    optional bytes api_payload = 3;
}

// -----X-----
message XappId {
    optional string value = 1;
}

```

Annex (informative): Change History

Date	Revision	Description
2024.04.01	02.00	Version incremented for TSC approval.
2024.04.01	01.00.06	Editorial updates to capture comments received during on-line vote.
2024.03.23	01.00.05	Enclosure of CRs: CMCC-2024.02.29-WG3-CR-0014-RICAPI-API registration service stage 3 definitions-v02.docx , NEC.AO-2024.02.07-WG3-CR-027-RICAPI- Introduction of gRPC for E2 related and SDL API - v5.docx , CMCC-2024.03.13-WG3-CR-0015-RICAPI-xApp's Endpoints for E2 Indications and E2 Subscription Delete Notifications-v01.docx agreed at RICARCH #102. NOK-2023.12.04-WG3-CR0003-RICAPI-v6.docx agreed at WG3 regular #218. Editorial updates.
2024.03.14	01.00.04	Enclosure of CRs: CICT-2024.01.19-WG3-CR-0015-RICAPI corrections v2.docx agreed at RICARCH #100. NOK-2023.02.01-WG3-CR0004-RICAPI-Timer-v3.docx agreed at 202402 Athens F2F RICARCH stream. CMCC-2024.01.27-WG3-CR-0013-RICAPI-SDL Information API stage 3 updates-v03.docx , and CMCC.AO-2024.01.21-WG3-CR-0012-RICAPI-xApp Data Sharing API and support for RAI-v04.docx agreed at RICARCH #101. Editorial updates.
2023.11.23	01.00.03	Enclosure of CRs: NEC-2023.08.17-WG3-CR-0019-RICAPI- Clarification on Data format-v02.docx agreed at RICARCH #93. CICT-2023.09.11-WG3-CR-0014-Corrections on RICAPI v1.docx agreed at 202310 Phoenix F2F RICARCH stream. Editorial updates.
2023.08.14	01.00.02	Adopt new TS template. Enclosure of CR: CMCC-2023.06.13-WG3-CR-0011-RICAPI-Correction of the BitString definition in GPB code-v02.docx agreed at RICARCH #92. Editorial updates.
2023.06.08	01.00.01	Enclosure of CR: CMCC.AO-2023.02.03-WG3-CR-0002-RICAPI-E2 Subscription Delete API Procedure (Platform initiated) descriptions and messages_v05.docx agreed at 202302 Prague F2F RICARCH stream. Editorial updates.
2022.11.20	01.00	Version incremented for TSC approval.
2022.11.16	00.06.00	Enclosure of CR: VMW.AO-2022.11.11-WG3-CR-0003-RICAPI-Improvements on Proto3 definitions-v2.zip agreed at RICARCH #79. Editorial updates.
2022.11.12	00.05.00	Enclosure of CR: CMCC.AO-2022.11.09-WG3-CR-0010-RICAPI-Collection of Common IEs across API categories under Section 10-v1.zip agreed at RICARCH TG 20221111 Ad-hoc meeting. Editorial updates.

2022.11.09	00.04.00	Copyright statement update in accordance to O-RAN alliance rules. Enclosure of CRs: CMCC.AO-2022.05.13-WG3-CR-0003-RICAPI-E2 related APIs overview_v04.docx agreed at RICARCH #76. AsialInfo.AO-2022.08.05-WG3-CR-0004-RICAPI-Cause and Criticality Diagnostics IE Definition-v04.docx, and CMCC.AO-2022.09.16-WG3-CR-0006-RICAPI-A1 related APIs overview_v01.docx agreed at RICARCH #77. CMCC.AO-2022.10.10-WG3-CR-0005-RICAPI-Proto3 Definitions for E2 related APIs-v1.zip agreed at 202210 Madrid F2F RICARCH stream. CMCC-2022.08.17-WG3-CR-0004-RICAPI-Basic contents in Section 4-v3.docx, CMCC-2022.09.16-WG3-CR-0007-RICAPI-Enablement APIs overview for Section 9.1-v2.docx, and CMCC-2022.11.01-WG3-CR-0009-RICAPI-Alignment on Message Type IEs for all API categories-v1.docx agreed at RICARCH #78. CICT.AO-2021.06.01-WG3-CR-0004-procedural descriptions and msgs applied to A1-related API_v9.zip, CMCC.AO-2022.10.25-WG3-CR-0008-SDL APIs overview and Proto3 Definitions-v3.zip, AsialInfo.AO-2022.10.11-WG3-CR-0005-RICAPI-xApp Registration Procedure Description and Messages-v3.docx, AsialInfo.AO-2022.10.20-WG3-CR-0006-RICAPI-Proto3 Definitions for Management APIs-v1.zip, and CMCC.AO-2022.08.22-WG3-CR-0005-RICAPI-Proto3 Definitions for Enablement APIs-v4.zip agreed at RICARCH TG 20221108 Ad-hoc meeting. Editorial updates.
2022.08.10	00.03.00	Clause assigned per API message. Enclosure of CR: AsialInfo.AO-2021.11.17-WG3-CR-0001-RICAPI-E2 Subscription API Procedure descriptions and messages_v09.docx agreed at RICARCH #74. Minor editorial update.
2022.06.06	00.02.00	Adoption of new O-RAN TS template. Enclosure of CRs: NOK-2022.03.21-WG3-CR-0001-RICAPI-E2NodeState-v02.docx agreed at RICARCH #67. Lenovo.AO-2022.05.13-WG3-RICAPI-Enablement API procedure descriptions and definitions_v04.docx, CICT-2022.04.22-WG3-CR-0009-Procedure Transaction Identity_v3.docx, AsialInfo.AO-2022.01.04-WG3-CR-0002-RICAPI-E2 Indication API Procedure descriptions and messages_v04.docx, and AsialInfo.AO-2022.03.04-WG3-CR-0003-RICAPI-E2 Control API Procedure descriptions and messages_v04.docx agreed at RICARCH #70. Minor editorial update.
2021.10.12	00.01.00	Enclosure of CR VMW.AO-2021.07.14-WG3-CR-0002-SDLAPI-v04.docx agreed at RICARCH #56.
2021.05.18	00.00.03	Approved TS skeleton.